RELATÓRIO COM AS LINHAS DE CÓDIGO

Sistema de identificação e autenticação biométrica facial para controle de acesso a cofre de segurança máxima

Atividades Práticas Supervisionadas – Processamento de Imagem e Visão Computacional (PIVC)

Ciência da Computação – Turma CC6P33

Gabriela Soares Batista – RA: R078FD2

Icaro G C de Souza – RA: R080870

Letícia Silva Bezkorowayny de Oliveira – RA: G908AG7

Luíza J de Souza – RA: G051HC5

Linguagem: Python 3.12 · Bibliotecas: OpenCV 4.11 e NumPy · Banco: SQLite

Total: 27 arquivos, 3929 linhas

SUMÁRIO

1 PACOTE DO SISTEMA (COFRE) 4
1.1 cofre/__init__.py 4
1.2 cofre/__main__.py 4
1.3 cofre/config.py 5
1.4 cofre/erros.py 7
1.5 cofre/aquisicao.py 7
1.6 cofre/preprocessamento.py 9
1.7 cofre/segmentacao.py 10
1.8 cofre/caracteristicas.py 11
1.9 cofre/classificacao.py 13
1.10 cofre/pipeline.py 15
1.11 cofre/prova_vida.py 16
1.12 cofre/autenticacao.py 17
1.13 cofre/politica.py 20
1.14 cofre/banco.py 20
1.15 cofre/servico.py 24
1.16 cofre/relatorios.py 27
1.17 cofre/avaliacao.py 28
1.18 cofre/interface.py 31
2 TESTES AUTOMATIZADOS 39
2.1 testes/_ambiente.py 39
2.2 testes/test_fases.py 39
2.3 testes/test_integracao.py 41
2.4 testes/test_seguranca.py 43
3 FERRAMENTAS DE EXPERIMENTOS E FIGURAS 49
3.1 ferramentas/adulterar_historico.py 49
3.2 ferramentas/baixar_orl.py 49
3.3 ferramentas/experimentos.py 50
3.4 ferramentas/gerar_figuras.py 55
3.5 ferramentas/simulacao_interface.py 60

1 PACOTE DO SISTEMA (COFRE)

1.1 cofre/__init__.py

Apresentação do pacote e da organização por fases.

```
"""Sistema de identificação e autenticação biométrica facial para o cofre do
Ministério do Meio Ambiente (APS de Processamento de Imagem e Visão Computacional).

O pacote está organizado segundo as cinco fases do processamento de imagens
digitais:

    aquisicao         -> sensor (câmera ou arquivo de imagem)
    preprocessamento  -> tons de cinza, redução de ruído, equalização
    segmentacao       -> detecção, recorte, alinhamento e máscara do rosto
    caracteristicas   -> LBP uniforme e histogramas por blocos
    classificacao     -> distância qui-quadrado e decisão de aceitação

e pelos módulos de apoio de segurança: banco, politica, autenticacao,
relatorios, avaliacao e interface.
"""

__version__ = "1.0.0"
```

1.2 cofre/__main__.py

Ponto de entrada: menu e subcomandos de linha de comando.

```
"""Ponto de entrada: `python -m cofre` abre o menu; subcomandos servem para
demonstrar partes do sistema sem câmera (arquivo de imagem, base pública)."""

import argparse
import sys

from . import avaliacao, config
from .erros import ErroCofre
from .interface import Menu
from .servico import Cofre


def _montar_parser():
    p = argparse.ArgumentParser(prog="cofre", description=__doc__)
    sub = p.add_subparsers(dest="comando")

    a = sub.add_parser("autenticar-arquivo", help="autentica a partir de uma imagem")
    a.add_argument("imagem")
    a.add_argument("--compartimento", type=int, required=True, help="id do compartimento")
    a.add_argument("--recorte", action="store_true",
                   help="a imagem já contém só o rosto (pula a detecção)")

    v = sub.add_parser("avaliar", help="mede FAR/FRR/EER em uma base de imagens")
    v.add_argument("pasta", help="pasta com uma subpasta por pessoa")
    v.add_argument("--amostras", type=int, default=5, help="imagens de cadastro por
pessoa")
    v.add_argument("--detectar", action="store_true",
                   help="roda o detector de rosto (use se as imagens forem fotos
completas)")
    v.add_argument("--csv", help="grava a curva FAR x FRR neste arquivo CSV")
    sub.add_parser("diagnostico", help="testa câmera, detecção de rosto/olhos e piscada")
    sub.add_parser("calibrar", help="mede distâncias entre as amostras de cadastro reais
(limiar)")
    sub.add_parser("retreinar", help="ação LOCAL: refaz o modelo a partir das amostras
guardadas")
    sub.add_parser("trocar-ministro", help="ação LOCAL: substitui o ministro e cadastra o
novo")
    return p


def main(argv=None):
    if not sys.stdout.isatty():
        sys.stdout.reconfigure(encoding="utf-8")
    args = _montar_parser().parse_args(argv)
```

```
    try:
        if args.comando == "avaliar":
            relatorio = avaliacao.avaliar(args.pasta, args.amostras, args.detectar)
            print(avaliacao.resumo(relatorio))
            if args.csv:
                print("CSV:", avaliacao.salvar_csv(relatorio, args.csv))
            return 0
        cofre = Cofre()
        if args.comando == "diagnostico":
            from .interface import diagnosticar_camera
            print(diagnosticar_camera(cofre))
            return 0
        if args.comando == "calibrar":

print(avaliacao.resumo_calibracao(avaliacao.calibrar_com_cadastros(cofre.banco)))
            return 0
        if args.comando == "retreinar":
            print("Esta ação refaz o modelo a partir das amostras em disco e fica
registrada no histórico.")
            if input("Digite SIM para confirmar: ").strip() != "SIM":
                print("Cancelado.")
                return 1
            print(f"Modelo refeito com {cofre.retreinar_modelo()} amostras.")
            return 0
        if args.comando == "trocar-ministro":
            atual = [u for u in cofre.banco.listar_usuarios() if u["nivel"] ==
config.NIVEL_MINISTRO]
            if atual:
                print(f"Ministro atual: {atual[0]['nome']}. Ele será substituído; a
biometria dele só "
                      "será apagada depois que o novo ministro for cadastrado com
sucesso.")
                if input("Digite SIM para confirmar: ").strip() != "SIM":
                    print("Cancelado.")
                    return 1
            else:
                print("Não há ministro ativo. O novo ministro será cadastrado por ação
local.")
            Menu(cofre).cadastrar(local=True, substituindo=atual[0] if atual else None)
            return 0
        if args.comando == "autenticar-arquivo":
            print("AVISO: autenticação por arquivo NÃO tem prova de vida (uma foto do
titular serve). "
                  "Uso apenas para demonstração e teste; o registro fica marcado no
histórico.",
                  file=sys.stderr)
            res = cofre.autenticar_arquivo(args.imagem, args.compartimento, args.recorte)
            print(("PERMITIDO" if res["permitido"] else "NEGADO"),
                  "-", res.get("usuario") or "ninguém identificado", "-", res["motivo"])
            return 0 if res["permitido"] else 2
        Menu(cofre).executar()
        return 0
    except ErroCofre as e:
        print(f"Erro: {e}")
        return 1
    except EOFError:
        print("\nEntrada encerrada.")
        return 1


if __name__ == "__main__":
    sys.exit(main())
```

1.3 cofre/config.py

Parâmetros centrais de desempenho e segurança (limiar, janelas, tempos).

```
"""Parâmetros centrais do sistema.

Todos os valores que influenciam o desempenho ou a segurança ficam aqui, para
que possam ser ajustados (e justificados no relatório) em um único lugar.
A pasta de dados pode ser trocada pela variável de ambiente COFRE_DADOS, o que
é usado pelos testes automatizados para não tocar nos dados reais.
"""

```

```
 9  import os
10  from pathlib import Path
11
12  RAIZ = Path(__file__).resolve().parent.parent
13  PASTA_DADOS = Path(os.environ.get("COFRE_DADOS", RAIZ / "dados"))
14  PASTA_AMOSTRAS = PASTA_DADOS / "amostras"
15  PASTA_RELATORIOS = PASTA_DADOS / "relatorios"
16  PASTA_TENTATIVAS = PASTA_DADOS / "tentativas_negadas"
17  ARQUIVO_BANCO = PASTA_DADOS / "cofre.db"
18  ARQUIVO_MODELO = PASTA_DADOS / "modelo.npz"
19
20  # --- Níveis de segurança (conforme o enunciado) ----------------------------
21  NIVEL_GERAL = 1       # permissão geral
22  NIVEL_DIRETOR = 2     # diretores de divisões específicas
23  NIVEL_MINISTRO = 3    # exclusivo do ministro do Meio Ambiente
24  NOMES_NIVEIS = {
25      NIVEL_GERAL: "Nível 1 - Permissão geral",
26      NIVEL_DIRETOR: "Nível 2 - Diretor de divisão",
27      NIVEL_MINISTRO: "Nível 3 - Ministro do Meio Ambiente",
28  }
29
30  # --- Aquisição -------------------------------------------------------------
31  INDICE_CAMERA = 0
32  LARGURA_CAPTURA = 640
33  ALTURA_CAPTURA = 480
34
35  # --- Segmentação -----------------------------------------------------------
36  ESCALA_DETECCAO = 1.1        # passo da pirâmide de escalas do detector Haar
37  VIZINHOS_DETECCAO = 5        # detecções vizinhas exigidas (reduz falsos positivos)
38  TAMANHO_MIN_FACE = 90        # lado mínimo (px) do rosto no quadro original
39  MARGEM_RECORTE = 0.15        # folga ao redor da caixa do rosto (fração do lado)
40  FRACAO_MIN_SEGUNDA_FACE = 0.5  # outro rosto com >= 50% da área do maior é "intruso"
41  ANGULO_MAX_ALINHAMENTO = 25.0  # graus; acima disso a detecção de olhos é descartada
42
43  # --- Pré-processamento e normalização --------------------------------------
44  TAMANHO_FACE = (100, 100)    # largura x altura do rosto normalizado
45  CLAHE_LIMITE = 2.0
46  CLAHE_BLOCO = (8, 8)
47  # Duas etapas que, na base ORL (recortes justos, luz controlada), PIORARAM o resultado
48  # (EER médio de 15,1% com as duas contra 10,5% sem elas, em 30 divisões). Foram mantidas
49  # ligadas por serem pensadas para câmera real (fundo variável, luz irregular), mas isso é
50  # uma hipótese: compare as duas configurações com a sua câmera (comando `calibrar`).
51  USAR_CLAHE = True
52  USAR_MASCARA = True
53
54  # --- Qualidade mínima da amostra -------------------------------------------
55  BRILHO_MIN = 45.0
56  BRILHO_MAX = 215.0
57  NITIDEZ_MIN = 20.0           # variância do laplaciano no recorte normalizado
58
59  # --- Extração de características -------------------------------------------
60  GRADE_BLOCOS = (8, 8)        # linhas x colunas de blocos do histograma LBP
61
62  # --- Classificação e decisão -----------------------------------------------
63  # Limiar de distância qui-quadrado: abaixo dele o rosto é aceito como o usuário
64  # previsto. O valor padrão foi escolhido a partir da avaliação em base pública
65  # (ver módulo avaliacao) e deve ser CONFERIDO com os rostos reais de quem usa o
66  # sistema (comando `python -m cofre calibrar`), editando este arquivo se preciso.
67  LIMIAR_DISTANCIA = 0.38      # ORL, 40 pessoas, conjunto aberto: 1 impostor aceito em 200
testes
68                               # (a menor distância de impostor na ORL foi 0,378). O valor
foi
69                               # escolhido NOS MESMOS dados em que foi medido e vale para
imagens
70                               # de 64x64 sem detecção; para webcam é uma estimativa de
partida.
71                               # Prioriza a segurança: aceitar intruso é pior que pedir ao
usuário
72                               # legítimo que tente de novo
73  MARGEM_MINIMA = 0.03         # folga mínima entre o 1° e o 2° usuário mais próximos
74  MELHORES_AMOSTRAS = 1        # média das k menores distâncias de cada usuário (k=1 foi o
75                               # melhor na avaliação com a base ORL: EER 17,3% contra 21,0%
com
76                               # k=2; diferença dentro da incerteza estatística, ver
relatório)
77  AMOSTRAS_POR_CADASTRO = 25
```

```
78  AMOSTRAS_MINIMAS = 5         # menos que isso o cadastro é recusado (e a calibração exige
o mesmo)
79  INTERVALO_ENTRE_AMOSTRAS = 0.35   # segundos
80
81  # --- Autenticação em tempo real ---------------------------------------------
82  QUADROS_JANELA = 12          # janela de votação (últimos N quadros)
83  QUADROS_CONFIRMACAO = 8      # votos iguais necessários na janela
84  QUADROS_RECONFIRMACAO = 3    # quadros seguidos reconhecendo o MESMO usuário ao fim da
prova de vida
85  TEMPO_MAX_AUTENTICACAO = 15  # segundos para o usuário ser reconhecido
86  TENTATIVAS_ATE_BLOQUEIO = 3
87  SEGUNDOS_BLOQUEIO = 60
88  EXIGIR_PROVA_DE_VIDA = True  # desafio de piscada (ver modulo prova_vida)
89  TEMPO_MAX_PISCADA = 8        # segundos para completar o desafio
```

1.4 cofre/erros.py

Exceções específicas do sistema.

```
 1  """Exceções específicas do sistema.
 2
 3  Cada falha previsível tem a sua própria classe, para que a interface possa
 4  explicar ao usuário o que aconteceu sem mostrar uma pilha de erro.
 5  """
 6
 7
 8  class ErroCofre(Exception):
 9      """Classe base de todos os erros do sistema."""
10
11
12  class CameraIndisponivel(ErroCofre):
13      """A câmera não pôde ser aberta (ocupada, desconectada ou sem permissão)."""
14
15
16  class ErroCaptura(ErroCofre):
17      """A câmera abriu, mas deixou de entregar quadros."""
18
19
20  class ImagemInvalida(ErroCofre):
21      """O arquivo de imagem não existe ou não pôde ser decodificado."""
22
23
24  class ModeloIndisponivel(ErroCofre):
25      """Não há modelo treinado, ou o arquivo do modelo foi alterado/corrompido."""
26
27
28  class UsuarioInvalido(ErroCofre):
29      """Dados de cadastro rejeitados (nome vazio, nível inexistente, etc.)."""
30
31
32  class RegraDeNegocio(ErroCofre):
33      """Operação proibida por uma regra do cofre (ex.: segundo ministro)."""
34
35
36  class SistemaBloqueado(ErroCofre):
37      """O terminal está em bloqueio temporário após falhas consecutivas."""
38
39      def __init__(self, segundos_restantes):
40          super().__init__(f"Terminal bloqueado por mais {segundos_restantes} s.")
41          self.segundos_restantes = segundos_restantes
```

1.5 cofre/aquisicao.py

Fase 1: câmera e leitura de arquivos de imagem.

```
 1  """Fase 1 - Aquisição da imagem (sensor).
 2
 3  O enunciado pede reconhecimento em tempo real por câmera, mas admite imagens
 4  obtidas por scanner ou arquivo. Por isso o módulo oferece as duas fontes com a
 5  mesma interface de saída: uma matriz NumPy BGR (formato nativo do OpenCV).
 6  """
 7
 8  import os
 9  import time
```

```
from pathlib import Path

import cv2
import numpy as np

from . import config
from .erros import CameraIndisponivel, ErroCaptura, ImagemInvalida


class Camera:
    """Envolve cv2.VideoCapture com abertura segura e liberação garantida."""

    def __init__(self, indice=None, largura=None, altura=None):
        self.indice = config.INDICE_CAMERA if indice is None else indice
        self.largura = largura or config.LARGURA_CAPTURA
        self.altura = altura or config.ALTURA_CAPTURA
        self._captura = None

    def abrir(self):
        # No Windows, o backend DirectShow abre mais rápido e de forma mais
        # estável do que o backend padrão (MSMF) para webcams comuns.
        backend = cv2.CAP_DSHOW if os.name == "nt" else cv2.CAP_ANY
        captura = cv2.VideoCapture(self.indice, backend)
        if not captura.isOpened():
            captura.release()
            raise CameraIndisponivel(
                f"Não foi possível abrir a câmera de índice {self.indice}. "
                "Verifique se ela está conectada, se outro programa a está "
                "usando e se o Windows permite o acesso (Configurações > "
                "Privacidade > Câmera).")
        captura.set(cv2.CAP_PROP_FRAME_WIDTH, self.largura)
        captura.set(cv2.CAP_PROP_FRAME_HEIGHT, self.altura)
        self._captura = captura
        self._aquecer()
        return self

    def _aquecer(self, quadros=5):
        """Descarta os primeiros quadros, que costumam sair escuros."""
        for _ in range(quadros):
            self._captura.read()
            time.sleep(0.03)

    def ler(self):
        if self._captura is None:
            raise ErroCaptura("A câmera não foi aberta.")
        ok, quadro = self._captura.read()
        if not ok or quadro is None:
            raise ErroCaptura("A câmera parou de enviar imagens.")
        return quadro

    def fechar(self):
        if self._captura is not None:
            self._captura.release()
            self._captura = None

    def __enter__(self):
        return self.abrir()

    def __exit__(self, *_):
        self.fechar()


def carregar_imagem(caminho, cinza=False):
    """Lê uma imagem de arquivo (scanner, foto, base pública).

    Usa np.fromfile + imdecode porque cv2.imread falha em silêncio com
    caminhos que têm acentos no Windows (ex.: 'Documentos', 'João').
    """
    caminho = Path(caminho)
    if not caminho.is_file():
        raise ImagemInvalida(f"Arquivo não encontrado: {caminho}")
    try:
        dados = np.fromfile(str(caminho), dtype=np.uint8)
        modo = cv2.IMREAD_GRAYSCALE if cinza else cv2.IMREAD_COLOR
        imagem = cv2.imdecode(dados, modo)
    except Exception as exc:  # leitura de disco ou decodificação
        raise ImagemInvalida(f"Falha ao ler {caminho}: {exc}") from exc
```

```
     if imagem is None:
         raise ImagemInvalida(f"O arquivo não é uma imagem válida: {caminho}")
     return imagem


 def salvar_imagem(caminho, imagem):
     """Grava uma imagem PNG/JPG suportando caminhos com acentos."""
     caminho = Path(caminho)
     caminho.parent.mkdir(parents=True, exist_ok=True)
     extensao = caminho.suffix or ".png"
     ok, buffer = cv2.imencode(extensao, imagem)
     if not ok:
         raise ImagemInvalida(f"Não foi possível codificar a imagem {caminho}")
     buffer.tofile(str(caminho))
     return caminho
```

1.6 cofre/preprocessamento.py

Fase 2: tons de cinza, ruído, equalização, normalização e qualidade.

```
"""Fase 2 - Pré-processamento.

Prepara o quadro e o recorte do rosto para as fases seguintes:
conversão para tons de cinza, redução de ruído, correção de iluminação
(equalização de histograma) e avaliação de qualidade da amostra.
"""

import cv2
import numpy as np

from . import config


def para_cinza(imagem):
    """Converte BGR para tons de cinza; aceita imagem que já é cinza."""
    if imagem.ndim == 2:
        return imagem
    return cv2.cvtColor(imagem, cv2.COLOR_BGR2GRAY)


def reduzir_ruido(cinza):
    """Filtro gaussiano 3x3: atenua o ruído do sensor sem borrar demais."""
    return cv2.GaussianBlur(cinza, (3, 3), 0)


def equalizar_global(cinza):
    """Equalização de histograma global (usada antes do detector Haar)."""
    return cv2.equalizeHist(cinza)


def equalizar_adaptativa(cinza):
    """CLAHE: equalização adaptativa com limite de contraste.

    Trabalha por blocos, o que corrige sombras laterais no rosto melhor do
    que a equalização global (Zuiderveld, 1994).
    """
    clahe = cv2.createCLAHE(clipLimit=config.CLAHE_LIMITE,
                            tileGridSize=config.CLAHE_BLOCO)
    return clahe.apply(cinza)


def normalizar_face(recorte_cinza):
    """Leva qualquer recorte ao tamanho padrão e (se config.USAR_CLAHE) corrige a
iluminação."""
    reduzido = cv2.resize(recorte_cinza, config.TAMANHO_FACE,
                          interpolation=cv2.INTER_AREA)
    return equalizar_adaptativa(reduzido) if config.USAR_CLAHE else reduzido


def nitidez(cinza):
    """Variância do laplaciano: quanto menor, mais desfocada a imagem."""
    return float(cv2.Laplacian(cinza, cv2.CV_64F).var())


def avaliar_qualidade(recorte_cinza):
```

```
55      """Verifica brilho e nitidez do recorte (antes da equalização).
56
57      Devolve (aprovado, lista_de_motivos). Amostras ruins pioram o modelo e,
58      na autenticação, produzem falsas rejeições; por isso são descartadas.
59      """
60      reduzido = cv2.resize(recorte_cinza, config.TAMANHO_FACE,
61                            interpolation=cv2.INTER_AREA)
62      motivos = []
63      brilho = float(np.mean(reduzido))
64      if brilho < config.BRILHO_MIN:
65          motivos.append("imagem muito escura")
66      elif brilho > config.BRILHO_MAX:
67          motivos.append("imagem muito clara")
68      if nitidez(reduzido) < config.NITIDEZ_MIN:
69          motivos.append("imagem desfocada")
70      return (len(motivos) == 0), motivos
```

1.7 cofre/segmentacao.py

Fase 3: detecção, recorte, alinhamento e máscara do rosto.

```
  1  """Fase 3 - Segmentação.
  2
  3  Separa o rosto do resto da cena em quatro passos:
  4    1. detecção do rosto com cascata de Haar (Viola e Jones, 2001);
  5    2. recorte com pequena margem;
  6    3. alinhamento: gira o recorte para deixar a linha dos olhos horizontal;
  7    4. máscara elíptica, que remove cabelo, fundo e roupa dos cantos.
  8  """
  9
 10  import math
 11
 12  import cv2
 13  import numpy as np
 14
 15  from . import config
 16  from .preprocessamento import equalizar_global
 17
 18
 19  class Detector:
 20      """Detector de rostos e olhos baseado em cascatas de Haar do OpenCV."""
 21
 22      def __init__(self):
 23          pasta = cv2.data.haarcascades
 24          self._rosto = cv2.CascadeClassifier(pasta +
"haarcascade_frontalface_default.xml")
 25          self._olho = cv2.CascadeClassifier(pasta + "haarcascade_eye.xml")
 26          if self._rosto.empty() or self._olho.empty():
 27              raise RuntimeError("Arquivos de cascata do OpenCV não encontrados.")
 28
 29      def detectar_rostos(self, cinza):
 30          """Lista de caixas (x, y, w, h) ordenada da maior para a menor."""
 31          equalizada = equalizar_global(cinza)
 32          caixas = self._rosto.detectMultiScale(
 33              equalizada,
 34              scaleFactor=config.ESCALA_DETECCAO,
 35              minNeighbors=config.VIZINHOS_DETECCAO,
 36              minSize=(config.TAMANHO_MIN_FACE, config.TAMANHO_MIN_FACE))
 37          caixas = [tuple(int(v) for v in c) for c in caixas]
 38          return sorted(caixas, key=lambda c: c[2] * c[3], reverse=True)
 39
 40      def detectar_olhos(self, recorte_cinza):
 41          """Caixas de olhos ABERTOS: no máximo um por metade do rosto.
 42
 43          Procurar em cada metade da faixa superior (55% da altura) elimina
 44          detecções em sobrancelha, nariz ou boca e garante um par esquerdo e
 45          direito para o alinhamento. Parâmetros (passo 1,05 e 4 vizinhos)
 46          escolhidos medindo, em quadros reais de webcam, a fração de quadros
 47          com os dois olhos encontrados: 99-100%, contra 38% com passo 1,1 e
 48          6 vizinhos.
 49          """
 50          altura, largura = recorte_cinza.shape[:2]
 51          faixa = equalizar_global(recorte_cinza[: int(altura * 0.55), :])
 52          lado = max(8, largura // 8)
 53          olhos = []
```

```
        for x0, x1 in ((0, largura // 2), (largura // 2, largura)):
            achados = self._olho.detectMultiScale(
                faixa[:, x0:x1], scaleFactor=1.05, minNeighbors=4, minSize=(lado, lado))
            if len(achados):
                x, y, w, h = max(achados, key=lambda o: o[2] * o[3])
                olhos.append((int(x) + x0, int(y), int(w), int(h)))
        return olhos


def ha_intruso(caixas):
    """Verdadeiro se houver um segundo rosto de tamanho comparável ao maior.

    Impede que um acompanhante entre junto (piggybacking) ou que alguém
    observe a tela por cima do ombro durante a autenticação.
    """
    if len(caixas) < 2:
        return False
    maior = caixas[0][2] * caixas[0][3]
    segundo = caixas[1][2] * caixas[1][3]
    return segundo >= config.FRACAO_MIN_SEGUNDA_FACE * maior


def recortar(cinza, caixa, margem=None):
    """Recorta a caixa do rosto com margem, respeitando os limites da imagem."""
    margem = config.MARGEM_RECORTE if margem is None else margem
    x, y, w, h = caixa
    dx, dy = int(w * margem), int(h * margem)
    x0, y0 = max(0, x - dx), max(0, y - dy)
    x1, y1 = min(cinza.shape[1], x + w + dx), min(cinza.shape[0], y + h + dy)
    return cinza[y0:y1, x0:x1]


def angulo_entre_olhos(olhos, largura_recorte):
    """Ângulo (graus) da reta que liga os dois olhos, ou None se não houver par."""
    if len(olhos) < 2:
        return None
    # fica com os dois maiores e os ordena da esquerda para a direita
    dois = sorted(sorted(olhos, key=lambda o: o[2] * o[3], reverse=True)[:2],
                  key=lambda o: o[0])
    (x1, y1, w1, h1), (x2, y2, w2, h2) = dois
    c1 = (x1 + w1 / 2.0, y1 + h1 / 2.0)
    c2 = (x2 + w2 / 2.0, y2 + h2 / 2.0)
    if abs(c2[0] - c1[0]) < 0.25 * largura_recorte:
        return None  # olhos "colados": quase certamente uma detecção falsa
    angulo = math.degrees(math.atan2(c2[1] - c1[1], c2[0] - c1[0]))
    if abs(angulo) > config.ANGULO_MAX_ALINHAMENTO:
        return None
    return angulo


def alinhar(recorte_cinza, angulo):
    """Gira o recorte em torno do centro para nivelar os olhos."""
    if angulo is None or abs(angulo) < 1.0:
        return recorte_cinza
    h, w = recorte_cinza.shape[:2]
    matriz = cv2.getRotationMatrix2D((w / 2.0, h / 2.0), angulo, 1.0)
    return cv2.warpAffine(recorte_cinza, matriz, (w, h),
                          flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE)


def mascara_eliptica(face_normalizada):
    """Zera os pixels fora de uma elipse centrada no rosto.

    O fundo e o cabelo variam de um dia para o outro e não identificam a
    pessoa; mantê-los só aumentaria a distância entre fotos do mesmo usuário.
    """
    h, w = face_normalizada.shape[:2]
    mascara = np.zeros((h, w), dtype=np.uint8)
    cv2.ellipse(mascara, (w // 2, int(h * 0.52)),
                (int(w * 0.42), int(h * 0.52)), 0, 0, 360, 255, -1)
    return cv2.bitwise_and(face_normalizada, face_normalizada, mask=mascara)
```

1.8 cofre/caracteristicas.py

Fase 4: LBP uniforme e histograma por blocos.

```python
"""Fase 4 - Extração de características.

Implementa o LBP (Local Binary Patterns) uniforme e o histograma por blocos,
descritor de textura proposto para reconhecimento facial por Ahonen, Hadid e
Pietikäinen (2006). Tudo é feito com NumPy vetorizado, sem laços por pixel.

Fluxo: imagem em cinza -> imagem LBP (um código de 0 a 255 por pixel) ->
mapeamento para 59 classes (padrões uniformes) -> histograma de cada bloco da
grade -> vetor único concatenado e normalizado.
"""

import numpy as np

from . import config

# Deslocamentos dos 8 vizinhos, no sentido horário a partir do canto
# superior esquerdo, em coordenadas da imagem sem borda.
_VIZINHOS = [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (1, 0)]


def _tabela_uniformes():
    """Mapeia cada código LBP de 8 bits para uma de 59 classes.

    Um padrão é "uniforme" se, percorrendo os 8 bits em círculo, há no máximo
    duas transições 0->1 ou 1->0. Existem 58 padrões assim (manchas, bordas,
    cantos); os 198 restantes viram uma única classe "outros" (índice 58).
    """
    tabela = np.zeros(256, dtype=np.uint8)
    proximo = 0
    for codigo in range(256):
        bits = [(codigo >> i) & 1 for i in range(8)]
        transicoes = sum(bits[i] != bits[(i + 1) % 8] for i in range(8))
        if transicoes <= 2:
            tabela[codigo] = proximo
            proximo += 1
        else:
            tabela[codigo] = 58
    return tabela


TABELA_UNIFORMES = _tabela_uniformes()
NUM_CLASSES = 59


def imagem_lbp(cinza):
    """Calcula o código LBP (3x3, 8 vizinhos) de cada pixel interno."""
    cinza = np.asarray(cinza, dtype=np.uint8)
    h, w = cinza.shape
    centro = cinza[1:h - 1, 1:w - 1]
    codigo = np.zeros(centro.shape, dtype=np.uint8)
    for posicao, (dy, dx) in enumerate(_VIZINHOS):
        vizinho = cinza[dy:dy + h - 2, dx:dx + w - 2]
        codigo |= ((vizinho >= centro).astype(np.uint8) << (7 - posicao))
    return codigo


def histograma_por_blocos(lbp, grade=None):
    """Divide a imagem LBP em blocos e concatena os histogramas de 59 classes.

    Cada histograma é normalizado pela soma, de modo que o vetor não depende
    do tamanho do bloco. O resultado tem grade[0] * grade[1] * 59 dimensões.
    """
    linhas, colunas = grade or config.GRADE_BLOCOS
    classes = TABELA_UNIFORMES[lbp]
    h, w = classes.shape
    partes = []
    for i in range(linhas):
        for j in range(colunas):
            bloco = classes[i * h // linhas:(i + 1) * h // linhas,
                            j * w // colunas:(j + 1) * w // colunas]
            hist = np.bincount(bloco.ravel(), minlength=NUM_CLASSES).astype(np.float64)
            hist /= max(hist.sum(), 1.0)
            partes.append(hist)
    return np.concatenate(partes)


def extrair(face_normalizada, grade=None):
```

```
78      """Atalho: rosto normalizado -> vetor de características."""
79      return histograma_por_blocos(imagem_lbp(face_normalizada), grade)
```

1.9 cofre/classificacao.py

Fase 5: distância qui-quadrado, decisão em conjunto aberto, votação e integridade do modelo.

```
  1  """Fase 5 - Classificação e interpretação.
  2
  3  O classificador compara o vetor de características do rosto apresentado com os
  4  vetores guardados de cada usuário usando a distância qui-quadrado, adequada a
  5  histogramas. A decisão é em conjunto aberto: se nenhum usuário chega perto o
  6  bastante, o rosto é tratado como desconhecido, nunca como "o mais parecido".
  7  """
  8
  9  import hashlib
 10  from collections import deque
 11  from dataclasses import dataclass
 12  from pathlib import Path
 13
 14  import numpy as np
 15
 16  from . import config
 17  from .erros import ModeloIndisponivel
 18
 19  _EPS = 1e-10
 20
 21
 22  def distancia_qui_quadrado(a, b):
 23      """Distância qui-quadrado simétrica entre histogramas normalizados.
 24
 25      Para vetores de blocos, a soma é dividida pelo número de blocos, o que
 26      mantém o resultado na faixa de 0 a 2 independentemente da grade usada.
 27      """
 28      a = np.asarray(a, dtype=np.float64)
 29      b = np.asarray(b, dtype=np.float64)
 30      blocos = max(1, a.shape[-1] // 59)
 31      return np.sum((a - b) ** 2 / (a + b + _EPS), axis=-1) / blocos
 32
 33
 34  @dataclass
 35  class Previsao:
 36      """Resultado da comparação de um rosto com o modelo."""
 37      usuario_id: int          # id do usuário mais próximo (-1 se modelo vazio)
 38      distancia: float         # distância ao usuário mais próximo
 39      segunda_distancia: float  # distância ao segundo mais próximo (inf se só há um)
 40      aceito: bool             # distancia <= limiar
 41
 42      @property
 43      def margem(self):
 44          """Folga relativa entre o melhor e o segundo colocado."""
 45          if not np.isfinite(self.segunda_distancia) or self.segunda_distancia <= 0:
 46              return 1.0
 47          return (self.segunda_distancia - self.distancia) / self.segunda_distancia
 48
 49
 50  class Reconhecedor:
 51      """Banco de histogramas por usuário + busca do vizinho mais próximo."""
 52
 53      def __init__(self, limiar=None):
 54          self.limiar = config.LIMIAR_DISTANCIA if limiar is None else limiar
 55          self._vetores = np.empty((0, 0))
 56          self._rotulos = np.empty(0, dtype=np.int64)
 57
 58      # -- treino ------------------------------------------------------------
 59      def treinar(self, vetores, rotulos):
 60          vetores = np.asarray(vetores, dtype=np.float64)
 61          rotulos = np.asarray(rotulos, dtype=np.int64)
 62          if len(vetores) == 0 or len(vetores) != len(rotulos):
 63              raise ModeloIndisponivel("Não há amostras suficientes para treinar o
modelo.")
 64          self._vetores, self._rotulos = vetores, rotulos
 65          return self
```

```
66
67      @property
68      def treinado(self):
69          return len(self._rotulos) > 0
70
71      @property
72      def usuarios(self):
73          return sorted(set(int(r) for r in self._rotulos))
74
75      # -- inferência ------------------------------------------------------------
76      def prever(self, vetor):
77          if not self.treinado:
78              raise ModeloIndisponivel("O modelo ainda não foi treinado.")
79          distancias = distancia_qui_quadrado(self._vetores, np.asarray(vetor)[None, :])
80          por_usuario = []
81          for uid in self.usuarios:
82              d = np.sort(distancias[self._rotulos == uid])
83              k = min(config.MELHORES_AMOSTRAS, len(d))
84              por_usuario.append((float(np.mean(d[:k])), uid))
85          por_usuario.sort()
86          melhor_d, melhor_id = por_usuario[0]
87          segunda_d = por_usuario[1][0] if len(por_usuario) > 1 else float("inf")
88          return Previsao(melhor_id, melhor_d, segunda_d, melhor_d <= self.limiar)
89
90      # -- persistência com verificação de integridade ---------------------------
91      def salvar(self, caminho=None):
92          caminho = Path(caminho or config.ARQUIVO_MODELO)
93          caminho.parent.mkdir(parents=True, exist_ok=True)
94          np.savez_compressed(caminho, vetores=self._vetores, rotulos=self._rotulos)
95          return hash_arquivo(caminho)
96
97      @classmethod
98      def carregar(cls, caminho=None, hash_esperado=None, limiar=None):
99          caminho = Path(caminho or config.ARQUIVO_MODELO)
100         if not caminho.is_file():
101             raise ModeloIndisponivel("Nenhum modelo treinado foi encontrado.")
102         if hash_esperado is not None and hash_arquivo(caminho) != hash_esperado:
103             raise ModeloIndisponivel(
104                 "O arquivo do modelo foi alterado ou está corrompido "
105                 "(a impressão digital SHA-256 não confere). Um administrador local deve "
106                 "verificar a máquina e executar: python -m cofre retreinar")
107         try:
108             with np.load(caminho) as dados:
109                 modelo = cls(limiar)
110                 modelo.treinar(dados["vetores"], dados["rotulos"])
111         except ModeloIndisponivel:
112             raise
113         except Exception as exc:
114             raise ModeloIndisponivel(f"Modelo ilegível: {exc}") from exc
115         return modelo
116
117
118 def hash_arquivo(caminho):
119     """SHA-256 do conteúdo de um arquivo."""
120     resumo = hashlib.sha256()
121     with open(caminho, "rb") as arquivo:
122         for bloco in iter(lambda: arquivo.read(65536), b""):
123             resumo.update(bloco)
124     return resumo.hexdigest()
125
126
127 class Confirmador:
128     """Votação temporal: só confirma quando o mesmo usuário vence em N dos
129     últimos M quadros.
130
131     Um único quadro pode ser enganado por ruído, reflexo ou uma semelhança
132     ocasional; exigir consistência no tempo filtra esses erros de um quadro só
133     sem exigir que o usuário fique parado. ATENÇÃO: quadros consecutivos de uma
134     mesma pose são muito parecidos, então 8 votos NÃO são 8 evidências
135     independentes. A votação não protege contra uma pessoa muito parecida com um
136     usuário cadastrado, e o seu efeito sobre a falsa aceitação não foi medido.
137     """
138
139     def __init__(self, janela=None, necessarios=None):
140         self.janela = janela or config.QUADROS_JANELA
141         self.necessarios = necessarios or config.QUADROS_CONFIRMACAO
142         self._votos = deque(maxlen=self.janela)
```

```
143
144      def registrar(self, usuario_id):
145          """usuario_id é None quando o quadro foi rejeitado."""
146          self._votos.append(usuario_id)
147          return self.confirmado()
148
149      def confirmado(self):
150          validos = [v for v in self._votos if v is not None]
151          if not validos:
152              return None
153          candidato = max(set(validos), key=validos.count)
154          return candidato if validos.count(candidato) >= self.necessarios else None
155
156      def reiniciar(self):
157          self._votos.clear()
```

1.10 cofre/pipeline.py

Encadeamento das cinco fases e guarda das imagens intermediárias.

```
1   """Encadeia as cinco fases e guarda as imagens intermediárias.
2
3   Guardar cada etapa permite mostrar, na apresentação, o que o sistema "enxerga"
4   em cada fase, e permite que os testes verifiquem as fases separadamente.
5   """
6
7   from dataclasses import dataclass, field
8   from typing import List, Optional, Tuple
9
10  import numpy as np
11
12  from . import caracteristicas, config, preprocessamento, segmentacao
13
14
15  def _face_final(alinhada):
16      """Normaliza o rosto e, se config.USAR_MASCARA, remove os cantos (cabelo, fundo)."""
17      face = preprocessamento.normalizar_face(alinhada)
18      return segmentacao.mascara_eliptica(face) if config.USAR_MASCARA else face
19
20
21  @dataclass
22  class Resultado:
23      """Tudo o que o pipeline produziu para um quadro."""
24      quadro: Optional[np.ndarray] = None          # fase 1: imagem original (BGR)
25      cinza: Optional[np.ndarray] = None           # fase 2: tons de cinza
26      caixas: List[Tuple[int, int, int, int]] = field(default_factory=list)
27      caixa: Optional[Tuple[int, int, int, int]] = None   # fase 3: maior rosto
28      intruso: bool = False                        # há outro rosto relevante?
29      recorte: Optional[np.ndarray] = None         # fase 3: recorte com margem
30      angulo: Optional[float] = None               # inclinação corrigida (graus)
31      olhos: int = 0                               # olhos abertos detectados
32      normalizada: Optional[np.ndarray] = None     # fase 3/4: rosto 100x100 mascarado
33      lbp: Optional[np.ndarray] = None             # fase 4: imagem LBP
34      vetor: Optional[np.ndarray] = None           # fase 4: histograma por blocos
35      qualidade_ok: bool = False
36      motivos: List[str] = field(default_factory=list)
37
38      @property
39      def tem_rosto(self):
40          return self.caixa is not None
41
42
43  class Pipeline:
44      def __init__(self):
45          self.detector = segmentacao.Detector()
46
47      # -- a partir de um quadro completo (câmera ou foto de corpo/busto) --------
48      def processar(self, quadro_bgr, com_olhos=True):
49          r = Resultado(quadro=quadro_bgr)
50          r.cinza = preprocessamento.para_cinza(quadro_bgr)
51          suave = preprocessamento.reduzir_ruido(r.cinza)
52
53          r.caixas = self.detector.detectar_rostos(suave)
54          if not r.caixas:
55              return r
```

```
56          r.caixa = r.caixas[0]
57          r.intruso = segmentacao.ha_intruso(r.caixas)
58
59          r.recorte = segmentacao.recortar(suave, r.caixa)
60          qual_ok, motivos = preprocessamento.avaliar_qualidade(r.recorte)
61          r.qualidade_ok, r.motivos = qual_ok, motivos
62
63          if com_olhos:
64              olhos = self.detector.detectar_olhos(r.recorte)
65              r.olhos = len(olhos)
66              r.angulo = segmentacao.angulo_entre_olhos(olhos, r.recorte.shape[1])
67          alinhado = segmentacao.alinhar(r.recorte, r.angulo)
68
69          r.normalizada = _face_final(alinhado)
70          r.lbp = caracteristicas.imagem_lbp(r.normalizada)
71          r.vetor = caracteristicas.histograma_por_blocos(r.lbp)
72          return r
73
74      # -- a partir de um recorte já feito (bases públicas, fotos 3x4) -----------
75      def processar_recorte(self, recorte_bgr_ou_cinza):
76          """Pula a detecção: o arquivo já contém só o rosto."""
77          r = Resultado(quadro=recorte_bgr_ou_cinza)
78          r.cinza = preprocessamento.para_cinza(recorte_bgr_ou_cinza)
79          r.recorte = preprocessamento.reduzir_ruido(r.cinza)
80          r.caixa = (0, 0, r.cinza.shape[1], r.cinza.shape[0])
81          r.caixas = [r.caixa]
82          r.qualidade_ok, r.motivos = preprocessamento.avaliar_qualidade(r.recorte)
83          r.normalizada = _face_final(r.recorte)
84          r.lbp = caracteristicas.imagem_lbp(r.normalizada)
85          r.vetor = caracteristicas.histograma_por_blocos(r.lbp)
86          return r
```

1.11 cofre/prova_vida.py

Desafio de piscada (prova de vida).

```
 1  """Prova de vida por desafio de piscada.
 2
 3  Uma foto impressa ou exibida em um celular tem o rosto certo, mas não pisca.
 4  O desafio pede ao usuário que mantenha os olhos abertos, pisque e abra de
 5  novo. O detector de olhos de Haar encontra apenas olhos ABERTOS; portanto uma
 6  piscada aparece como: olhos detectados -> não detectados -> detectados.
 7
 8  Limitações assumidas (discutidas no relatório): (1) um vídeo gravado do
 9  usuário piscando poderia, em tese, passar; (2) o detector de Haar falha de
10  forma intermitente, de modo que uma foto em movimento poderia, por acaso,
11  simular "olhos não detectados". Por isso a piscada exige DOIS quadros
12  consecutivos sem os dois olhos, e a autenticação só se conclui depois que a
13  identidade é reconfirmada nos quadros finais (ver autenticacao.py). A prova de
14  vida é uma camada que encarece o ataque, não uma garantia absoluta
15  (ISO/IEC 30107-1:2016).
16  """
17
18  ABERTOS_INICIAIS = 3     # quadros consecutivos com os dois olhos abertos
19  FECHADOS_MINIMOS = 2     # quadros CONSECUTIVOS com menos de dois olhos detectados
20  ABERTOS_FINAIS = 2       # quadros consecutivos reabertos
21
22
23  class DesafioPiscada:
24      AGUARDA_ABERTOS = 0
25      AGUARDA_PISCADA = 1
26      AGUARDA_REABERTURA = 2
27      CONCLUIDO = 3
28
29      def __init__(self):
30          self.reiniciar()
31
32      def reiniciar(self):
33          self.estado = self.AGUARDA_ABERTOS
34          self._contador = 0
35
36      @property
37      def concluido(self):
38          return self.estado == self.CONCLUIDO
39
```

```
    @property
    def instrucao(self):
        return {
            self.AGUARDA_ABERTOS: "Olhe para a camera com os olhos abertos",
            self.AGUARDA_PISCADA: "Agora PISQUE",
            self.AGUARDA_REABERTURA: "Abra os olhos",
            self.CONCLUIDO: "Prova de vida concluida",
        }[self.estado]

    def registrar(self, olhos_detectados):
        """Alimenta o desafio com o nº de olhos abertos do quadro.

        Passe None quando não houve rosto: o desafio recomeça, pois a pessoa
        saiu de cena e não se pode juntar partes de pessoas diferentes.
        """
        if olhos_detectados is None:
            self.reiniciar()
            return self.estado

        abertos = olhos_detectados >= 2
        if self.estado == self.AGUARDA_ABERTOS:
            self._contador = self._contador + 1 if abertos else 0
            if self._contador >= ABERTOS_INICIAIS:
                self.estado, self._contador = self.AGUARDA_PISCADA, 0
        elif self.estado == self.AGUARDA_PISCADA:
            self._contador = self._contador + 1 if not abertos else 0
            if self._contador >= FECHADOS_MINIMOS:
                self.estado, self._contador = self.AGUARDA_REABERTURA, 0
        elif self.estado == self.AGUARDA_REABERTURA:
            self._contador = self._contador + 1 if abertos else 0
            if self._contador >= ABERTOS_FINAIS:
                self.estado = self.CONCLUIDO
        return self.estado
```

1.12 cofre/autenticacao.py

Sessão de autenticação em tempo real e bloqueio do terminal.

```
"""Lógica de autenticação em tempo real, sem entrada/saída.

A classe SessaoAutenticacao recebe o resultado do pipeline de cada quadro e
devolve o estado da tentativa. Como não depende de câmera nem de janelas, ela
é testada automaticamente com sequências simuladas de quadros.
"""

import time
from collections import Counter
from dataclasses import dataclass
from typing import Optional

from . import config
from .classificacao import Confirmador
from .erros import SistemaBloqueado
from .prova_vida import DesafioPiscada

PROCURANDO = "procurando"
VALIDANDO = "validando"
PROVA_DE_VIDA = "prova_de_vida"
IDENTIFICADO = "identificado"
FALHA = "falha"

_MOTIVOS = {
    "nao_reconhecido": "rosto não reconhecido (usuário desconhecido)",
    "varios_rostos": "mais de um rosto no campo de visão",
    "sem_rosto": "nenhum rosto detectado",
    "qualidade": "qualidade da imagem insuficiente",
}


@dataclass
class Estado:
    fase: str
    mensagem: str
    usuario_id: Optional[int] = None
    distancia: Optional[float] = None
```

```
    motivo_falha: Optional[str] = None

    @property
    def finalizado(self):
        return self.fase in (IDENTIFICADO, FALHA)


class SessaoAutenticacao:
    """Uma tentativa de autenticação, do primeiro quadro ao veredito."""

    def __init__(self, reconhecedor, exigir_vida=None, relogio=time.monotonic,
                 tempo_max=None):
        self.reconhecedor = reconhecedor
        self.exigir_vida = config.EXIGIR_PROVA_DE_VIDA if exigir_vida is None else
exigir_vida
        self._relogio = relogio
        self._tempo_max = tempo_max or config.TEMPO_MAX_AUTENTICACAO
        self._inicio = relogio()
        self._inicio_vida = None
        self._confirmador = Confirmador()
        self._desafio = DesafioPiscada()
        self._ocorrencias = Counter()
        self._usuario = None
        self._distancia = None
        self._outro_seguido = 0
        self._reconfirmados = 0

    # ------------------------------------------------------------------
    def _falha(self, motivo):
        return Estado(FALHA, "Acesso negado: " + motivo, self._usuario,
                      self._distancia, motivo)

    def _motivo_predominante(self):
        if not self._ocorrencias:
            return "tempo esgotado"
        chave = self._ocorrencias.most_common(1)[0][0]
        return _MOTIVOS[chave]

    def _reiniciar_identidade(self):
        self._confirmador.reiniciar()
        self._desafio.reiniciar()
        self._usuario = None
        self._inicio_vida = None
        self._outro_seguido = 0
        self._reconfirmados = 0

    # ------------------------------------------------------------------
    def processar(self, r):
        agora = self._relogio()

        if self._usuario is None and agora - self._inicio > self._tempo_max:
            return self._falha(self._motivo_predominante())
        if (self._usuario is not None and self._inicio_vida is not None
                and agora - self._inicio_vida > config.TEMPO_MAX_PISCADA):
            return self._falha("prova de vida não realizada no tempo previsto")

        if not r.tem_rosto:
            self._ocorrencias["sem_rosto"] += 1
            self._reiniciar_identidade()
            return Estado(PROCURANDO, "Aproxime o rosto da câmera")

        if r.intruso:
            self._ocorrencias["varios_rostos"] += 1
            self._reiniciar_identidade()
            return Estado(PROCURANDO, "Mais de um rosto no campo de visão")

        if not r.qualidade_ok:
            self._ocorrencias["qualidade"] += 1
            if self._usuario is None:
                self._confirmador.registrar(None)
            return Estado(PROCURANDO, "Qualidade ruim: " + ", ".join(r.motivos))

        previsao = self.reconhecedor.prever(r.vetor)
        valido = previsao.aceito and previsao.margem >= config.MARGEM_MINIMA

        if self._usuario is None:
            confirmado = self._confirmador.registrar(
```

```
114                 previsao.usuario_id if valido else None)
115             if not valido:
116                 self._ocorrencias["nao_reconhecido"] += 1
117             if confirmado is None:
118                 texto = "Verificando identidade..." if valido else "Rosto não
reconhecido"
119                 return Estado(VALIDANDO, texto, distancia=previsao.distancia)
120             self._usuario, self._distancia = confirmado, previsao.distancia
121             if not self.exigir_vida:
122                 return Estado(IDENTIFICADO, "Identidade confirmada",
123                               self._usuario, self._distancia)
124             self._inicio_vida = agora
125             self._desafio.reiniciar()
126             self._desafio.registrar(r.olhos)
127             return Estado(PROVA_DE_VIDA, self._desafio.instrucao,
128                           self._usuario, self._distancia)
129
130         # --- fase de prova de vida: a identidade precisa continuar a mesma ----
131         if valido and previsao.usuario_id != self._usuario:
132             self._outro_seguido += 1
133             self._reconfirmados = 0
134             if self._outro_seguido >= 3:
135                 return self._falha("troca de pessoa durante a prova de vida")
136         else:
137             self._outro_seguido = 0
138         reconhecido = valido and previsao.usuario_id == self._usuario
139         # Reconfirmação: só o MESMO usuário reconhecido conta. Sem isso, trocar a foto de
140         # quem foi identificado por um rosto desconhecido que pisca faria a prova de vida
141         # "valer" para a vítima.
142         self._reconfirmados = self._reconfirmados + 1 if reconhecido else 0
143         # Os quadros de olhos ABERTOS (antes e depois da piscada) precisam ser do mesmo
144         # usuário; só durante a piscada, quando o reconhecimento cai de qualidade, o
145         # desafio tolera quadros não reconhecidos. Um rosto desconhecido que apareça
146         # com os olhos abertos reinicia o desafio.
147         if not reconhecido and self._desafio.estado != DesafioPiscada.AGUARDA_PISCADA \
148                 and not self._desafio.concluido:
149             self._desafio.reiniciar()
150         else:
151             self._desafio.registrar(r.olhos)
152         if self._desafio.concluido and self._reconfirmados >=
config.QUADROS_RECONFIRMACAO:
153             return Estado(IDENTIFICADO, "Identidade e prova de vida confirmadas",
154                           self._usuario, self._distancia)
155         return Estado(PROVA_DE_VIDA, self._desafio.instrucao,
156                       self._usuario, self._distancia)
157
158
159 class ControleBloqueio:
160     """Bloqueio temporário do terminal após falhas consecutivas.
161
162     O contador e o horário de liberação ficam no banco, para que fechar e
163     reabrir o programa não zere a punição (ataque de força bruta).
164     """
165
166     def __init__(self, banco, relogio=time.time):
167         self._banco = banco
168         self._relogio = relogio
169
170     def segundos_restantes(self):
171         ate = float(self._banco.obter_parametro("bloqueio_ate", "0"))
172         return max(0, int(round(ate - self._relogio())))
173
174     def exigir_liberado(self):
175         restante = self.segundos_restantes()
176         if restante > 0:
177             raise SistemaBloqueado(restante)
178
179     def registrar_falha(self):
180         """Soma uma falha; devolve True se esta falha ativou o bloqueio."""
181         falhas = int(self._banco.obter_parametro("falhas_consecutivas", "0")) + 1
182         if falhas >= config.TENTATIVAS_ATE_BLOQUEIO:
183             self._banco.definir_parametro(
184                 "bloqueio_ate", self._relogio() + config.SEGUNDOS_BLOQUEIO)
185             self._banco.definir_parametro("falhas_consecutivas", 0)
186             return True
187         self._banco.definir_parametro("falhas_consecutivas", falhas)
188         return False
```

```
189
190      def registrar_sucesso(self):
191          self._banco.definir_parametro("falhas_consecutivas", 0)
```

1.13 cofre/politica.py

Regras dos três níveis de acesso.

```
 1  """Política de acesso aos compartimentos do cofre (três níveis).
 2
 3  Regras, derivadas do enunciado:
 4    Nível 1 (geral)    -> apenas compartimentos de nível mínimo 1.
 5    Nível 2 (diretor)  -> compartimentos de nível 1 e os de nível 2 da PRÓPRIA
 6                          divisão. Diretor de outra divisão é negado.
 7    Nível 3 (ministro) -> todos os compartimentos.
 8
 9  É um controle de acesso em camadas, no espírito do modelo Bell-LaPadula
10  (clearance do sujeito x classificação do objeto), com a restrição adicional por
11  divisão no nível intermediário (necessidade de conhecer).
12  """
13
14  from dataclasses import dataclass
15
16  from . import config
17
18
19  @dataclass
20  class Decisao:
21      permitido: bool
22      motivo: str
23
24
25  def _mesma_divisao(a, b):
26      return (a or "").strip().casefold() == (b or "").strip().casefold()
27
28
29  def decidir(usuario, compartimento):
30      """Decide se `usuario` pode abrir `compartimento` (ambos como mapeamentos)."""
31      if not usuario["ativo"]:
32          return Decisao(False, "usuário desativado")
33
34      nivel = usuario["nivel"]
35      exigido = compartimento["nivel_minimo"]
36
37      if nivel == config.NIVEL_MINISTRO:
38          return Decisao(True, "Nível 3: acesso total")
39
40      if exigido == config.NIVEL_MINISTRO:
41          return Decisao(False, "compartimento exclusivo do ministro (Nível 3)")
42
43      if exigido == config.NIVEL_GERAL:
44          return Decisao(True, "compartimento de permissão geral")
45
46      # compartimento de nível 2
47      if nivel < config.NIVEL_DIRETOR:
48          return Decisao(False, "compartimento restrito a diretores (Nível 2)")
49      if compartimento["divisao"] and not _mesma_divisao(
50              usuario["divisao"], compartimento["divisao"]):
51          return Decisao(False, f"compartimento da divisão '{compartimento['divisao']}', "
52                                f"diferente da divisão do diretor")
53      return Decisao(True, "diretor da divisão responsável")
```

1.14 cofre/banco.py

Banco SQLite: usuários, compartimentos e histórico encadeado por hash.

```
 1  """Persistência em SQLite: usuários, compartimentos do cofre e registro de acessos.
 2
 3  O registro de acessos é encadeado por hash (cada linha inclui o SHA-256 da
 4  anterior), como em uma cadeia de blocos simplificada. Se alguém apagar ou
 5  editar uma linha do histórico, a verificação de integridade acusa a quebra.
 6  """
 7
 8  import hashlib
```

```
 9  import sqlite3
10  from datetime import datetime
11
12  from . import config
13  from .erros import RegraDeNegocio, UsuarioInvalido
14
15  _ESQUEMA = """
16  CREATE TABLE IF NOT EXISTS usuarios (
17      id        INTEGER PRIMARY KEY AUTOINCREMENT,
18      nome      TEXT    NOT NULL,
19      cargo     TEXT    NOT NULL DEFAULT '',
20      nivel     INTEGER NOT NULL CHECK (nivel IN (1, 2, 3)),
21      divisao   TEXT,
22      ativo     INTEGER NOT NULL DEFAULT 1,
23      criado_em TEXT    NOT NULL
24  );
25  CREATE TABLE IF NOT EXISTS compartimentos (
26      id             INTEGER PRIMARY KEY AUTOINCREMENT,
27      nome           TEXT    NOT NULL UNIQUE,
28      nivel_minimo   INTEGER NOT NULL CHECK (nivel_minimo IN (1, 2, 3)),
29      divisao        TEXT
30  );
31  CREATE TABLE IF NOT EXISTS acessos (
32      id              INTEGER PRIMARY KEY AUTOINCREMENT,
33      data_hora       TEXT    NOT NULL,
34      usuario_id      INTEGER,
35      nome            TEXT    NOT NULL,
36      compartimento   TEXT    NOT NULL,
37      resultado       TEXT    NOT NULL,
38      motivo          TEXT    NOT NULL,
39      distancia       REAL,
40      hash_anterior   TEXT    NOT NULL,
41      hash            TEXT    NOT NULL
42  );
43  CREATE TABLE IF NOT EXISTS parametros (
44      chave TEXT PRIMARY KEY,
45      valor TEXT NOT NULL
46  );
47  """
48
49  # Compartimentos padrão do cofre; a divisão só importa no nível 2.
50  COMPARTIMENTOS_PADRAO = [
51      ("Arquivo Geral de Relatórios Ambientais", 1, None),
52      ("Laboratório de Toxinas Agrícolas", 2, "Agrotóxicos e Biocidas"),
53      ("Laboratório de Resíduos Perigosos", 2, "Resíduos Perigosos"),
54      ("Câmara Central de Toxinas de Alto Risco", 3, None),
55  ]
56
57  HASH_INICIAL = "0" * 64
58
59
60  def agora():
61      return datetime.now().isoformat(sep=" ", timespec="seconds")
62
63
64  class Banco:
65      def __init__(self, caminho=None):
66          caminho = caminho or config.ARQUIVO_BANCO
67          config.PASTA_DADOS.mkdir(parents=True, exist_ok=True)
68          self._con = sqlite3.connect(str(caminho))
69          self._con.row_factory = sqlite3.Row
70          self._con.executescript(_ESQUEMA)
71          self._semear_compartimentos()
72
73      def fechar(self):
74          self._con.close()
75
76      def _semear_compartimentos(self):
77          if self._con.execute("SELECT COUNT(*) FROM compartimentos").fetchone()[0] == 0:
78              self._con.executemany(
79                  "INSERT INTO compartimentos (nome, nivel_minimo, divisao) VALUES
(?,?,?)",
80                  COMPARTIMENTOS_PADRAO)
81              self._con.commit()
82
83      # -- usuários ------------------------------------------------------------
84      def validar_novo_usuario(self, nome, cargo, nivel, divisao=None):
```

```
 85          """Aplica as regras de cadastro sem gravar; devolve os dados limpos.
 86
 87          Permite recusar um cadastro inválido ANTES de pedir ao usuário que
 88          posicione o rosto diante da câmera para a captura das amostras.
 89          """
 90          nome = (nome or "").strip()
 91          cargo = (cargo or "").strip()
 92          divisao = (divisao or "").strip() or None
 93          if len(nome) < 3:
 94              raise UsuarioInvalido("O nome deve ter pelo menos 3 caracteres.")
 95          if nivel not in config.NOMES_NIVEIS:
 96              raise UsuarioInvalido("Nível inexistente: use 1, 2 ou 3.")
 97          if nivel == config.NIVEL_DIRETOR and not divisao:
 98              raise UsuarioInvalido("O diretor precisa estar vinculado a uma divisão.")
 99          if nivel != config.NIVEL_DIRETOR:
100              divisao = None
101          if self._con.execute(
102                  "SELECT 1 FROM usuarios WHERE ativo=1 AND lower(nome)=lower(?)",
103                  (nome,)).fetchone():
104              raise UsuarioInvalido(f"Já existe um usuário ativo chamado '{nome}'.")
105          if nivel == config.NIVEL_MINISTRO and self.contar_ativos(config.NIVEL_MINISTRO)
>= 1:
106              raise RegraDeNegocio(
107                  "O Nível 3 é exclusivo do ministro: já existe um ministro ativo. "
108                  "Desative-o antes de cadastrar outro.")
109          return nome, cargo, nivel, divisao
110
111      def criar_usuario(self, nome, cargo, nivel, divisao=None):
112          nome, cargo, nivel, divisao = self.validar_novo_usuario(nome, cargo, nivel,
divisao)
113          cur = self._con.execute(
114              "INSERT INTO usuarios (nome, cargo, nivel, divisao, criado_em) VALUES
(?,?,?,?,?)",
115              (nome, cargo, nivel, divisao, agora()))
116          self._con.commit()
117          return cur.lastrowid
118
119      def obter_usuario(self, usuario_id):
120          return self._con.execute(
121              "SELECT * FROM usuarios WHERE id=?", (usuario_id,)).fetchone()
122
123      def listar_usuarios(self, apenas_ativos=True):
124          sql = "SELECT * FROM usuarios"
125          if apenas_ativos:
126              sql += " WHERE ativo=1"
127          return self._con.execute(sql + " ORDER BY nivel DESC, nome").fetchall()
128
129      def contar_ativos(self, nivel=None):
130          if nivel is None:
131              return self._con.execute(
132                  "SELECT COUNT(*) FROM usuarios WHERE ativo=1").fetchone()[0]
133          return self._con.execute(
134              "SELECT COUNT(*) FROM usuarios WHERE ativo=1 AND nivel=?",
135              (nivel,)).fetchone()[0]
136
137      def reativar_usuario(self, usuario_id):
138          """Desfaz uma desativação (usada para reverter uma troca de ministro que
falhou)."""
139          cur = self._con.execute("UPDATE usuarios SET ativo=1 WHERE id=?", (usuario_id,))
140          self._con.commit()
141          return cur.rowcount > 0
142
143      def desativar_usuario(self, usuario_id):
144          cur = self._con.execute("UPDATE usuarios SET ativo=0 WHERE id=?", (usuario_id,))
145          self._con.commit()
146          return cur.rowcount > 0
147
148      # -- compartimentos ------------------------------------------------------
149      def listar_compartimentos(self):
150          return self._con.execute(
151              "SELECT * FROM compartimentos ORDER BY nivel_minimo, nome").fetchall()
152
153      def obter_compartimento(self, compartimento_id):
154          return self._con.execute(
155              "SELECT * FROM compartimentos WHERE id=?", (compartimento_id,)).fetchone()
156
157      def listar_divisoes(self):
```

```
158          linhas = self._con.execute(
159              "SELECT DISTINCT divisao FROM compartimentos "
160              "WHERE divisao IS NOT NULL ORDER BY divisao").fetchall()
161          return [l[0] for l in linhas]
162
163      # -- parâmetros (modelo, bloqueio) ------------------------------------------
164      def obter_parametro(self, chave, padrao=None):
165          linha = self._con.execute(
166              "SELECT valor FROM parametros WHERE chave=?", (chave,)).fetchone()
167          return linha[0] if linha else padrao
168
169      def definir_parametro(self, chave, valor):
170          self._con.execute(
171              "INSERT INTO parametros (chave, valor) VALUES (?,?) "
172              "ON CONFLICT(chave) DO UPDATE SET valor=excluded.valor", (chave, str(valor)))
173          self._con.commit()
174
175      # -- registro de acessos com hash encadeado ---------------------------------
176      @staticmethod
177      def _calcular_hash(anterior, data_hora, usuario_id, nome, comp, resultado, motivo,
dist):
178          texto = "|".join(str(v) for v in
179                           (anterior, data_hora, usuario_id, nome, comp, resultado, motivo,
dist))
180          return hashlib.sha256(texto.encode("utf-8")).hexdigest()
181
182      def registrar_acesso(self, usuario_id, nome, compartimento, resultado, motivo,
183                           distancia=None):
184          ultimo = self._con.execute(
185              "SELECT hash FROM acessos ORDER BY id DESC LIMIT 1").fetchone()
186          anterior = ultimo[0] if ultimo else HASH_INICIAL
187          data_hora = agora()
188          h = self._calcular_hash(anterior, data_hora, usuario_id, nome, compartimento,
189                                  resultado, motivo, distancia)
190          # Linha + âncora na MESMA transação: uma queda no meio não deixa a âncora
191          # desencontrada (o que viraria falso alarme).
192          # Âncora: total de linhas e último hash. Detecta a remoção das ÚLTIMAS
193          # linhas, que o encadeamento sozinho não vê (a cadeia restante segue válida).
194          # Limite: a âncora mora no mesmo banco; quem edita o banco inteiro refaz as duas.
195          with self._con:
196              self._con.execute(
197                  "INSERT INTO acessos (data_hora, usuario_id, nome, compartimento,
resultado, "
198                  "motivo, distancia, hash_anterior, hash) VALUES (?,?,?,?,?,?,?,?,?)",
199                  (data_hora, usuario_id, nome, compartimento, resultado, motivo,
200                   distancia, anterior, h))
201              total = self._con.execute("SELECT COUNT(*) FROM acessos").fetchone()[0]
202              for chave, valor in (("log_total", total), ("log_ultimo_hash", h)):
203                  self._con.execute(
204                      "INSERT INTO parametros (chave, valor) VALUES (?,?) "
205                      "ON CONFLICT(chave) DO UPDATE SET valor=excluded.valor", (chave,
str(valor)))
206
207      def registrar_evento(self, responsavel, descricao, usuario_id=None):
208          """Registra uma operação administrativa (cadastro, remoção, retreino)."""
209          self.registrar_acesso(usuario_id, responsavel, "Administração do sistema",
210                                "ADMIN", descricao, None)
211
212      def listar_acessos(self, desde=None):
213          if desde:
214              return self._con.execute(
215                  "SELECT * FROM acessos WHERE data_hora >= ? ORDER BY id",
(desde,)).fetchall()
216          return self._con.execute("SELECT * FROM acessos ORDER BY id").fetchall()
217
218      def verificar_integridade(self):
219          """Percorre o histórico; devolve (ok, id_da_primeira_linha_suspeita)."""
220          anterior = HASH_INICIAL
221          ultimo_id, contagem = 0, 0
222          for linha in self._con.execute("SELECT * FROM acessos ORDER BY id"):
223              esperado = self._calcular_hash(
224                  anterior, linha["data_hora"], linha["usuario_id"], linha["nome"],
225                  linha["compartimento"], linha["resultado"], linha["motivo"],
226                  linha["distancia"])
227              if linha["hash_anterior"] != anterior or linha["hash"] != esperado:
228                  return False, linha["id"]
229              anterior = linha["hash"]
```

```
230             ultimo_id, contagem = linha["id"], contagem + 1
231         # confere a âncora (truncagem do fim do histórico)
232         total = self.obter_parametro("log_total")
233         ultimo = self.obter_parametro("log_ultimo_hash")
234         if total is None or ultimo is None:
235             # Falha FECHADA: com linhas no histórico, a âncora tem de existir; sem ela
236             # (apagada junto com o fim do histórico) não dá para afirmar que está
íntegro.
237             if contagem > 0:
238                 return False, 0
239         elif int(total) != contagem or ultimo != anterior:
240             return False, ultimo_id + 1            # linha(s) ausente(s) no fim
241         return True, None
```

1.15 cofre/servico.py

Operações de negócio que unem os módulos (cadastro, treino, decisão, registro).

```
  1  """Camada de serviço: junta banco, pipeline, modelo, política e registro.
  2
  3  A interface (menus e janelas) só chama métodos desta classe; assim a mesma
  4  lógica serve ao uso interativo, à autenticação por arquivo e aos testes.
  5  """
  6
  7  import shutil
  8  from datetime import datetime
  9
 10  from . import caracteristicas, config, politica
 11  from .aquisicao import carregar_imagem, salvar_imagem
 12  from .autenticacao import ControleBloqueio
 13  from .banco import Banco
 14  from .classificacao import Reconhecedor
 15  from .erros import ModeloIndisponivel, RegraDeNegocio, UsuarioInvalido
 16  from .pipeline import Pipeline
 17
 18  DESCONHECIDO = "(desconhecido)"
 19  ORIGEM_ARQUIVO = "por arquivo, sem prova de vida"
 20
 21  # "Compartimento" virtual: operações administrativas (cadastrar, remover,
 22  # relatórios) exigem o mesmo critério do Nível 3, exclusivo do ministro.
 23  ADMINISTRACAO = {"id": None, "nome": "Administração do sistema",
 24                   "nivel_minimo": config.NIVEL_MINISTRO, "divisao": None}
 25
 26
 27  class Cofre:
 28      def __init__(self, banco=None, pipeline=None):
 29          self.banco = banco or Banco()
 30          self._pipeline = pipeline
 31          self.bloqueio = ControleBloqueio(self.banco)
 32          self.reconhecedor = None
 33
 34      @property
 35      def pipeline(self):
 36          if self._pipeline is None:
 37              self._pipeline = Pipeline()
 38          return self._pipeline
 39
 40      # -- modelo ------------------------------------------------------------
 41      def carregar_modelo(self):
 42          """Carrega o modelo conferindo a impressão digital SHA-256 guardada.
 43
 44          Falha FECHADA: se o banco não tiver a impressão digital, o modelo não é
 45          usado (aceitar "sem hash" permitiria burlar a verificação apagando-o).
 46          """
 47          if not config.ARQUIVO_MODELO.is_file():
 48              raise ModeloIndisponivel("Nenhum modelo treinado foi encontrado.")
 49          esperado = self.banco.obter_parametro("hash_modelo")
 50          if not esperado:
 51              raise ModeloIndisponivel(
 52                  "O modelo não tem impressão digital (SHA-256) registrada no banco. "
 53                  "Um administrador local deve executar: python -m cofre retreinar")
 54          self.reconhecedor = Reconhecedor.carregar(hash_esperado=esperado)
 55          return self.reconhecedor
 56
 57      def retreinar_modelo(self):
```

```
 58          """Recuperação local: refaz o modelo a partir das amostras guardadas.
 59
 60          É uma ação de administrador da máquina (linha de comando), pois o modelo
 61          adulterado impede a autenticação do ministro, e é registrada no histórico.
 62          """
 63          n = self.treinar()
 64          self.banco.registrar_evento("(administrador local)",
 65                                      f"modelo retreinado a partir de {n} amostras
guardadas")
 66          return n
 67
 68      def modelo_pronto(self):
 69          try:
 70              self.carregar_modelo()
 71              return True
 72          except ModeloIndisponivel:
 73              return False
 74
 75      def treinar(self):
 76          """Reconstrói o modelo a partir das amostras dos usuários ativos."""
 77          vetores, rotulos = [], []
 78          for usuario in self.banco.listar_usuarios():
 79              pasta = config.PASTA_AMOSTRAS / str(usuario["id"])
 80              for arquivo in sorted(pasta.glob("*.png")):
 81                  vetores.append(caracteristicas.extrair(carregar_imagem(arquivo,
cinza=True)))
 82                  rotulos.append(usuario["id"])
 83          if not vetores:
 84              raise ModeloIndisponivel("Não há amostras de usuários ativos para treinar.")
 85          modelo = Reconhecedor().treinar(vetores, rotulos)
 86          self.banco.definir_parametro("hash_modelo", modelo.salvar())
 87          self.reconhecedor = modelo
 88          return len(vetores)
 89
 90      # -- usuários ------------------------------------------------------------
 91      def cadastrar(self, nome, cargo, nivel, divisao, faces_normalizadas,
 92                    responsavel="(administrador)"):
 93          """Grava usuário e amostras e retreina o modelo (e registra no histórico).
 94
 95          `faces_normalizadas` são os recortes 100x100 já processados. Se algo
 96          falhar no meio, o usuário recém-criado é desfeito para não deixar um
 97          cadastro sem biometria no banco.
 98          """
 99          if len(faces_normalizadas) < config.AMOSTRAS_MINIMAS:
100              raise UsuarioInvalido(
101                  f"Amostras insuficientes: {len(faces_normalizadas)} (mínimo
{config.AMOSTRAS_MINIMAS}). "
102                  "Um cadastro sem biometria nunca poderia ser reconhecido.")
103          usuario_id = self.banco.criar_usuario(nome, cargo, nivel, divisao)
104          try:
105              pasta = config.PASTA_AMOSTRAS / str(usuario_id)
106              for i, face in enumerate(faces_normalizadas, start=1):
107                  salvar_imagem(pasta / f"{i:03d}.png", face)
108              self.treinar()
109          except Exception:
110              self.banco.desativar_usuario(usuario_id)
111              shutil.rmtree(config.PASTA_AMOSTRAS / str(usuario_id), ignore_errors=True)
112              raise
113          self.banco.registrar_evento(
114              responsavel, f"usuário '{nome.strip()}' cadastrado (nível {nivel}) com "
115                           f"{len(faces_normalizadas)} amostras", usuario_id)
116          return usuario_id
117
118      def trocar_ministro(self, antigo_id, nome, cargo, faces_normalizadas,
119                          responsavel="(administrador local)"):
120          """Substitui o ministro de forma ATÔMICA: se o cadastro do novo falhar, o antigo
121          volta ao estado anterior (ativo, com biometria e modelo), e o sistema nunca fica
122          sem ministro. A biometria do antigo só é apagada depois que o novo está
cadastrado."""
123          antigo = self.banco.obter_usuario(antigo_id)
124          self.banco.desativar_usuario(antigo_id)          # libera a vaga única do nível 3
125          try:
126              novo_id = self.cadastrar(nome, cargo, config.NIVEL_MINISTRO, None,
127                                       faces_normalizadas, responsavel)
128          except Exception:
129              self.banco.reativar_usuario(antigo_id)
130              self.treinar()
```

```
131             raise
132         shutil.rmtree(config.PASTA_AMOSTRAS / str(antigo_id), ignore_errors=True)
133         self.treinar()
134         self.banco.registrar_evento(responsavel, f"ministro '{antigo['nome']}'
substituído por "
135                                                  f"'{nome.strip()}'; biometria do
anterior apagada",
136                                     antigo_id)
137         return novo_id
138
139     def remover_usuario(self, usuario_id, permitir_ministro=False,
responsavel="(administrador)"):
140         """Desativa o usuário, apaga as amostras biométricas e retreina.
141
142         O ministro não pode ser removido pelo menu: se o fosse, ninguém mais
143         conseguiria autenticar-se como administrador. A troca de ministro é uma
144         ação local (comando `trocar-ministro`) que passa permitir_ministro=True.
145         O histórico de acessos é mantido (finalidade de auditoria).
146         """
147         usuario = self.banco.obter_usuario(usuario_id)
148         if usuario is None or not usuario["ativo"]:
149             return False
150         if usuario["nivel"] == config.NIVEL_MINISTRO and not permitir_ministro:
151             raise RegraDeNegocio(
152                 "O ministro não pode ser removido pelo menu. Para substituí-lo, um
administrador "
153                 "local deve executar: python -m cofre trocar-ministro")
154         self.banco.desativar_usuario(usuario_id)
155         self.banco.registrar_evento(responsavel, f"usuário '{usuario['nome']}' removido;
"
156                                                  "amostras biométricas apagadas",
usuario_id)
157         shutil.rmtree(config.PASTA_AMOSTRAS / str(usuario_id), ignore_errors=True)
158         try:
159             self.treinar()
160         except ModeloIndisponivel:  # era o último usuário
161             config.ARQUIVO_MODELO.unlink(missing_ok=True)
162             self.banco.definir_parametro("hash_modelo", "")
163             self.reconhecedor = None
164         return True
165
166     # -- decisão e registro -------------------------------------------------------
167     def resolver_compartimento(self, referencia):
168         """Aceita o id de um compartimento ou o próprio registro (ou ADMINISTRACAO)."""
169         if isinstance(referencia, int):
170             registro = self.banco.obter_compartimento(referencia)
171             if registro is None:
172                 raise UsuarioInvalido(f"Compartimento {referencia} não existe.")
173             return registro
174         return referencia
175
176     def concluir_identificacao(self, usuario_id, compartimento, distancia, origem=None):
177         """Aplica a política ao usuário identificado e registra o resultado.
178
179         O bloqueio por falhas protege o RECONHECIMENTO contra tentativas em série
180         (rosto desconhecido, falha na prova de vida). Quem foi reconhecido mas não
181         tem permissão para aquele compartimento é negado e registrado, porém não
182         é bloqueado: errar de compartimento não é força bruta contra a biometria.
183         """
184         usuario = self.banco.obter_usuario(usuario_id)
185         compartimento = self.resolver_compartimento(compartimento)
186         decisao = politica.decidir(usuario, compartimento)
187         resultado = "PERMITIDO" if decisao.permitido else "NEGADO"
188         motivo = decisao.motivo + (f" [{origem}]" if origem else "")
189         self.banco.registrar_acesso(usuario["id"], usuario["nome"],
compartimento["nome"],
190                                     resultado, motivo, distancia)
191         self.bloqueio.registrar_sucesso()
192         return decisao, usuario, False
193
194     def registrar_falha_de_identificacao(self, compartimento, motivo, quadro=None,
195                                          distancia=None, usuario_id=None, origem=None):
196         """Registra tentativa negada e guarda a foto como evidência.
197
198         `usuario_id` é preenchido quando a pessoa chegou a ser identificada
199         mas falhou depois (ex.: prova de vida); caso contrário o registro
200         sai como "(desconhecido)".
```

```
201          """
202          compartimento = self.resolver_compartimento(compartimento)
203          usuario = self.banco.obter_usuario(usuario_id) if usuario_id else None
204          if origem:
205              motivo = f"{motivo} [{origem}]"
206          self.banco.registrar_acesso(usuario["id"] if usuario else None,
207                                      usuario["nome"] if usuario else DESCONHECIDO,
208                                      compartimento["nome"], "NEGADO", motivo, distancia)
209          if quadro is not None:
210              nome = datetime.now().strftime("%Y%m%d_%H%M%S_%f") + ".jpg"
211              salvar_imagem(config.PASTA_TENTATIVAS / nome, quadro)
212          return self.bloqueio.registrar_falha()
213
214      def autenticar_arquivo(self, caminho, compartimento_id, so_recorte=False):
215          """Autentica a partir de um arquivo de imagem (scanner, foto).
216
217          ATENÇÃO: não há prova de vida em arquivo (uma foto do titular serve). É
218          um caminho de demonstração e de teste, e os registros que ele gera levam
219          a marca "por arquivo, sem prova de vida" no motivo. Em produção, deve ser
220          desativado ou restrito ao administrador. Devolve um dicionário com o veredito.
221          """
222          self.bloqueio.exigir_liberado()
223          if self.reconhecedor is None:
224              self.carregar_modelo()
225          imagem = carregar_imagem(caminho)
226          # mesmo caminho do cadastro por câmera (com alinhamento pelos olhos)
227          r = (self.pipeline.processar_recorte(imagem) if so_recorte
228               else self.pipeline.processar(imagem, com_olhos=True))
229          if not r.tem_rosto:
230              self.registrar_falha_de_identificacao(compartimento_id, "nenhum rosto
detectado", origem=ORIGEM_ARQUIVO)
231              return {"permitido": False, "motivo": "nenhum rosto detectado", "usuario":
None}
232          if r.intruso:
233              self.registrar_falha_de_identificacao(compartimento_id, "mais de um rosto na
imagem", origem=ORIGEM_ARQUIVO)
234              return {"permitido": False, "motivo": "mais de um rosto na imagem",
"usuario": None}
235          previsao = self.reconhecedor.prever(r.vetor)
236          if not (previsao.aceito and previsao.margem >= config.MARGEM_MINIMA):
237              self.registrar_falha_de_identificacao(
238                  compartimento_id, "rosto não reconhecido", r.quadro, previsao.distancia,
239                  origem=ORIGEM_ARQUIVO)
240              return {"permitido": False, "motivo": "rosto não reconhecido",
241                      "usuario": None, "distancia": previsao.distancia}
242          decisao, usuario, _ = self.concluir_identificacao(
243              previsao.usuario_id, compartimento_id, previsao.distancia,
244              origem=ORIGEM_ARQUIVO)
245          return {"permitido": decisao.permitido, "motivo": decisao.motivo,
246                  "usuario": usuario["nome"], "distancia": previsao.distancia}
```

1.16 cofre/relatorios.py

Relatórios de acessos e de usuários.

```
 1  """Relatórios de auditoria: acessos, usuários e resumo estatístico.
 2
 3  Gera arquivos CSV (abrem no Excel) e um resumo em texto. O relatório de
 4  acessos sempre começa informando se o histórico está íntegro.
 5  """
 6
 7  import csv
 8  from collections import Counter
 9  from datetime import datetime
10
11  from . import config
12
13
14  def _carimbo():
15      return datetime.now().strftime("%Y%m%d_%H%M%S")
16
17
18  def relatorio_acessos(banco, desde=None):
19      """Grava CSV com os acessos e devolve (caminho_csv, texto_do_resumo)."""
20      acessos = banco.listar_acessos(desde)
```

```
21      integro, linha_ruim = banco.verificar_integridade()
22      config.PASTA_RELATORIOS.mkdir(parents=True, exist_ok=True)
23      caminho = config.PASTA_RELATORIOS / f"acessos_{_carimbo()}.csv"
24
25      with open(caminho, "w", newline="", encoding="utf-8-sig") as arquivo:
26          escritor = csv.writer(arquivo, delimiter=";")
27          escritor.writerow(["data_hora", "usuario", "compartimento", "resultado",
28                             "motivo", "distancia"])
29          for a in acessos:
30              dist = "" if a["distancia"] is None else f"{a['distancia']:.4f}".replace(".",
","")
31              escritor.writerow([a["data_hora"], a["nome"], a["compartimento"],
32                                 a["resultado"], a["motivo"], dist])
33
34      # operações administrativas (cadastro, remoção, retreino) não são tentativas de acesso
35      administrativos = sum(1 for a in acessos if a["resultado"] == "ADMIN")
36      tentativas = [a for a in acessos if a["resultado"] != "ADMIN"]
37      total = len(tentativas)
38      permitidos = sum(1 for a in tentativas if a["resultado"] == "PERMITIDO")
39      negados = total - permitidos
40      por_comp = Counter(a["compartimento"] for a in tentativas)
41      por_motivo = Counter(a["motivo"] for a in tentativas if a["resultado"] == "NEGADO")
42      por_usuario = Counter(a["nome"] for a in tentativas)
43
44      linhas = ["RELATÓRIO DE ACESSOS AO COFRE",
45                f"Gerado em: {datetime.now():%d/%m/%Y %H:%M:%S}"]
46      if desde:
47          linhas.append(f"Período: a partir de {desde}")
48      linhas.append("Integridade do histórico: " +
49                    ("ÍNTEGRO" if integro else f"VIOLADO (primeira linha suspeita:
{linha_ruim})"))
50      linhas += ["", f"Total de tentativas: {total}",
51                 f"  Permitidas: {permitidos}", f"  Negadas:    {negados}"]
52      if total:
53          linhas.append(f"  Taxa de negação: {100.0 * negados / total:.1f}%")
54      linhas.append(f"Operações administrativas registradas: {administrativos}")
55      if por_comp:
56          linhas += ["", "Tentativas por compartimento:"]
57          linhas += [f"  {n:3d}  {c}" for c, n in por_comp.most_common()]
58      if por_motivo:
59          linhas += ["", "Motivos de negação:"]
60          linhas += [f"  {n:3d}  {m}" for m, n in por_motivo.most_common()]
61      if por_usuario:
62          linhas += ["", "Tentativas por usuário:"]
63          linhas += [f"  {n:3d}  {u}" for u, n in por_usuario.most_common()]
64      linhas += ["", f"Arquivo CSV: {caminho}"]
65      return caminho, "\n".join(linhas)
66
67
68  def relatorio_usuarios(banco):
69      """Grava CSV com os usuários (sem qualquer dado biométrico)."""
70      config.PASTA_RELATORIOS.mkdir(parents=True, exist_ok=True)
71      caminho = config.PASTA_RELATORIOS / f"usuarios_{_carimbo()}.csv"
72      with open(caminho, "w", newline="", encoding="utf-8-sig") as arquivo:
73          escritor = csv.writer(arquivo, delimiter=";")
74          escritor.writerow(["id", "nome", "cargo", "nivel", "divisao", "ativo",
"criado_em"])
75          for u in banco.listar_usuarios(apenas_ativos=False):
76              escritor.writerow([u["id"], u["nome"], u["cargo"], u["nivel"],
77                                 u["divisao"] or "", "sim" if u["ativo"] else "não",
78                                 u["criado_em"]])
79      return caminho
```

1.17 cofre/avaliacao.py

Medição de FAR, FRR e EER em uma base de imagens.

```
1  """Avaliação de desempenho do reconhecedor em uma base de imagens.
2
3  Espera uma pasta com uma subpasta por pessoa (ex.: base/s1/1.pgm, 2.pgm...).
4  Protocolo em conjunto aberto, que imita o cofre real:
5
6    * metade das pessoas é "cadastrada" (as primeiras k imagens de cada uma);
7    * a outra metade é de "intrusos": nunca aparece no treino;
8    * as demais imagens são o teste: as de cadastrados medem a taxa de falsa
```

```
     rejeição (FRR) e as dos intrusos medem a taxa de falsa aceitação (FAR).

Varrendo o limiar de distância obtém-se a curva FAR x FRR e o ponto de erro
igual (EER), que orienta a escolha de config.LIMIAR_DISTANCIA.
"""

import csv
from dataclasses import dataclass
from pathlib import Path

import numpy as np

from . import caracteristicas, config
from .aquisicao import carregar_imagem
from .classificacao import Reconhecedor
from .erros import ErroCofre
from .pipeline import Pipeline

EXTENSOES = {".pgm", ".png", ".jpg", ".jpeg", ".bmp"}


@dataclass
class Relatorio:
    limiares: np.ndarray
    far: np.ndarray
    frr: np.ndarray
    eer: float
    limiar_eer: float
    acuracia_identificacao: float   # entre cadastrados, no limiar do EER
    n_genuinos: int
    n_impostores: int
    n_cadastrados: int
    n_intrusos: int


def carregar_base(pasta):
    pasta = Path(pasta)
    if not pasta.is_dir():
        raise ErroCofre(f"Pasta da base não encontrada: {pasta}")
    base = {}
    for sub in sorted(p for p in pasta.iterdir() if p.is_dir()):
        arquivos = sorted(f for f in sub.iterdir() if f.suffix.lower() in EXTENSOES)
        if arquivos:
            base[sub.name] = arquivos
    if len(base) < 4:
        raise ErroCofre("A base precisa de pelo menos 4 pessoas (2 cadastradas e 2
intrusos).")
    return base


def _vetor(pipeline, caminho, detectar):
    imagem = carregar_imagem(caminho, cinza=True)
    r = (pipeline.processar(imagem, com_olhos=True) if detectar
         else pipeline.processar_recorte(imagem))
    return r.vetor if r.tem_rosto and r.vetor is not None else None


def avaliar(pasta, amostras_cadastro=5, detectar=False, passo_limiar=0.01,
            progresso=None):
    base = carregar_base(pasta)
    pessoas = list(base)
    cadastrados = pessoas[::2]            # alternância determinística
    intrusos = [p for p in pessoas if p not in cadastrados]
    pipeline = Pipeline()

    vetores, rotulos, teste = [], [], []   # teste: (vetor, rotulo_real ou -1)
    for indice, pessoa in enumerate(pessoas):
        for pos, arquivo in enumerate(base[pessoa]):
            v = _vetor(pipeline, arquivo, detectar)
            if v is None:
                continue
            if pessoa in cadastrados and pos < amostras_cadastro:
                vetores.append(v)
                rotulos.append(cadastrados.index(pessoa))
            elif pessoa in cadastrados:
                teste.append((v, cadastrados.index(pessoa)))
            else:
```

```
                teste.append((v, -1))
        if progresso:
            progresso(indice + 1, len(pessoas))

    modelo = Reconhecedor(limiar=float("inf")).treinar(vetores, rotulos)
    previsoes = [(modelo.prever(v), real) for v, real in teste]

    genuinos = [(p.distancia, p.usuario_id == real) for p, real in previsoes if real >=
0]
    impostores = [p.distancia for p, real in previsoes if real < 0]
    if not genuinos or not impostores:
        raise ErroCofre("A base não gerou amostras de teste suficientes.")

    dist_gen = np.array([d for d, _ in genuinos])
    acertou = np.array([ok for _, ok in genuinos])
    dist_imp = np.array(impostores)

    limiares = np.arange(0.0, max(dist_gen.max(), dist_imp.max()) + passo_limiar,
passo_limiar)
    # FRR: genuíno rejeitado por distância alta OU identificado como outra pessoa
    frr = np.array([np.mean((dist_gen > t) | (~acertou)) for t in limiares])
    far = np.array([np.mean(dist_imp <= t) for t in limiares])
    # EER pelos limiares EXATOS (as próprias distâncias medidas), e não pela grade acima,
    # para coincidir com ferramentas/experimentos.py; a grade serve só para a curva e o
CSV.
    cand = np.unique(np.concatenate([dist_gen, dist_imp]))
    far_c = np.array([np.mean(dist_imp <= t) for t in cand])
    frr_c = np.array([np.mean((dist_gen > t) | (~acertou)) for t in cand])
    i = int(np.argmin(np.abs(far_c - frr_c)))
    eer = float((far_c[i] + frr_c[i]) / 2.0)
    acuracia = float(np.mean(acertou & (dist_gen <= cand[i])))

    return Relatorio(limiares, far, frr, eer, float(cand[i]), acuracia,
                     len(dist_gen), len(dist_imp), len(cadastrados), len(intrusos))


def calibrar_com_cadastros(banco):
    """Mede distâncias entre as amostras de cadastro REAIS (câmera) dos usuários.

    Serve para conferir o limiar com os rostos do próprio grupo, sem pedir novas
    capturas. Dois vieses otimistas, que o relatório deve declarar:
      * legítimos: as amostras são da mesma sessão (ignoram-se a própria e as
        vizinhas na sequência, que são quase idênticas), então a FRR real, em
        outro dia e outra luz, tende a ser PIOR do que a medida aqui;
      * impostores: só existem os outros usuários cadastrados, poucas pessoas.
    Devolve um dicionário com as estatísticas.
    """
    from .classificacao import distancia_qui_quadrado

    por_usuario = {}
    for usuario in banco.listar_usuarios():
        pasta = config.PASTA_AMOSTRAS / str(usuario["id"])
        vetores = [caracteristicas.extrair(carregar_imagem(f, cinza=True))
                   for f in sorted(pasta.glob("*.png"))]
        if len(vetores) >= 5:
            por_usuario[usuario["nome"]] = np.array(vetores)
    if len(por_usuario) < 2:
        raise ErroCofre("A calibração precisa de pelo menos 2 usuários cadastrados com
amostras.")

    genuinos, impostores = [], []
    for nome, V in por_usuario.items():
        for i, v in enumerate(V):
            d = distancia_qui_quadrado(V, v[None, :])
            longe = np.abs(np.arange(len(V)) - i) > 2       # fora da vizinhança temporal
            if longe.any():
                genuinos.append(float(d[longe].min()))
        for outro, W in por_usuario.items():
            if outro != nome:
                for v in W:
                    impostores.append(float(distancia_qui_quadrado(V, v[None, :]).min()))
    gen, imp = np.array(genuinos), np.array(impostores)
    limiar = config.LIMIAR_DISTANCIA
    return {
        "usuarios": len(por_usuario), "n_genuinos": len(gen), "n_impostores": len(imp),
        "gen_mediana": float(np.median(gen)), "gen_p90": float(np.percentile(gen, 90)),
        "gen_max": float(gen.max()), "imp_min": float(imp.min()),
```

```
158          "imp_p5": float(np.percentile(imp, 5)), "imp_mediana": float(np.median(imp)),
159          "frr_no_limiar": float(np.mean(gen > limiar)), "far_no_limiar": float(np.mean(imp
<= limiar)),
160          "limiar": limiar,
161      }
162
163
164  def resumo_calibracao(r):
165      return "\n".join([
166          "CALIBRAÇÃO COM AS AMOSTRAS DE CADASTRO (câmera)",
167          f"  Usuários: {r['usuarios']}   Pares legítimos: {r['n_genuinos']}   Pares de
impostor: {r['n_impostores']}",
168          f"  Legítimos (distância): mediana {r['gen_mediana']:.3f}, percentil 90
{r['gen_p90']:.3f}, máxima {r['gen_max']:.3f}",
169          f"  Impostores (distância): mínima {r['imp_min']:.3f}, percentil 5
{r['imp_p5']:.3f}, mediana {r['imp_mediana']:.3f}",
170          f"  No limiar atual ({r['limiar']}): FRR {100 * r['frr_no_limiar']:.1f}%  FAR
{100 * r['far_no_limiar']:.1f}%",
171          "  Leitura: um limiar seguro fica ABAIXO da menor distância de impostor. Se a
máxima dos",
172          "  legítimos estiver acima dela, as faixas se sobrepõem e é preciso melhorar a
captura",
173          "  (luz, posição) antes de mexer no limiar. Estes números são OTIMISTAS: mesma
sessão,",
174          "  poucos impostores. Confirme com tentativas reais em outro momento.",
175      ])
176
177
178  def salvar_csv(relatorio, caminho):
179      caminho = Path(caminho)
180      caminho.parent.mkdir(parents=True, exist_ok=True)
181      with open(caminho, "w", newline="", encoding="utf-8-sig") as arquivo:
182          escritor = csv.writer(arquivo, delimiter=";")
183          escritor.writerow(["limiar", "FAR", "FRR"])
184          for t, a, f in zip(relatorio.limiares, relatorio.far, relatorio.frr):
185              escritor.writerow([f"{t:.3f}".replace(".", ","),
186                                 f"{a:.4f}".replace(".", ","), f"{f:.4f}".replace(".",
",")])
187      return caminho
188
189
190  def resumo(relatorio):
191      return "\n".join([
192          "AVALIAÇÃO DE DESEMPENHO (conjunto aberto)",
193          f"  Pessoas cadastradas: {relatorio.n_cadastrados}   Intrusos:
{relatorio.n_intrusos}",
194          f"  Testes genuínos: {relatorio.n_genuinos}   Testes de impostor:
{relatorio.n_impostores}",
195          f"  Limiar no ponto de erro igual (EER): {relatorio.limiar_eer:.3f}",
196          f"  EER: {100 * relatorio.eer:.2f}%",
197          f"  Identificação correta de cadastrados nesse limiar: "
198          f"{100 * relatorio.acuracia_identificacao:.2f}%",
199          f"  Limiar configurado hoje: {config.LIMIAR_DISTANCIA}",
200      ])
```

1.18 cofre/interface.py

Menus, janelas em tempo real, painel das fases e telas de resultado.

```
 1  """Interface com o usuário: janelas em tempo real (OpenCV) e menus de texto.
 2
 3  Os textos desenhados nas janelas não têm acentos porque as fontes do OpenCV
 4  só cobrem ASCII; os menus do terminal usam o português completo.
 5  """
 6
 7  import time
 8
 9  import cv2
10  import numpy as np
11
12  from . import autenticacao as aut
13  from . import config, relatorios
14  from .aquisicao import Camera
15  from .erros import (CameraIndisponivel, ErroCaptura, ErroCofre, ModeloIndisponivel,
16                      SistemaBloqueado, UsuarioInvalido)
```

```
 17  from .servico import ADMINISTRACAO, Cofre
 18
 19  VERDE, VERMELHO, AMARELO, AZUL, BRANCO = (60, 200, 60), (50, 50, 230), (0, 200, 255),
(230, 160, 40), (255, 255, 255)
 20  JANELA = "Cofre biometrico - ESC cancela"
 21  _DURACAO_ADMIN = 300  # segundos de validade da sessão de administrador
 22
 23
 24  # ---------------------------------------------------------------------------
 25  # desenho
 26  # ---------------------------------------------------------------------------
 27  def _texto(img, texto, pos, cor=BRANCO, escala=0.6, espessura=2):
 28      cv2.putText(img, texto, pos, cv2.FONT_HERSHEY_SIMPLEX, escala, (0, 0, 0),
 29                  espessura + 2, cv2.LINE_AA)
 30      cv2.putText(img, texto, pos, cv2.FONT_HERSHEY_SIMPLEX, escala, cor, espessura,
cv2.LINE_AA)
 31
 32
 33  def _sem_acentos(texto):
 34      tabela = str.maketrans("áàâãäéèêëíìîïóòôõöúùûüçÁÀÂÃÄÉÈÊËÍÌÎÏÓÒÔÕÖÚÙÛÜÇ",
 35                             "aaaaaeeeeiiiioooooouuuucAAAAAEEEEIIIIOOOOOUUUUC")
 36      return texto.translate(tabela)
 37
 38
 39  def _desenhar_estado(quadro, r, estado, restante=None):
 40      saida = quadro.copy()
 41      cor = {aut.PROCURANDO: AMARELO, aut.VALIDANDO: AZUL, aut.PROVA_DE_VIDA: AZUL,
 42             aut.IDENTIFICADO: VERDE, aut.FALHA: VERMELHO}[estado.fase]
 43      for caixa in r.caixas:
 44          x, y, w, h = caixa
 45          espessura = 3 if caixa == r.caixa else 1
 46          cv2.rectangle(saida, (x, y), (x + w, y + h), cor if caixa == r.caixa else
VERMELHO,
 47                        espessura)
 48      barra = np.zeros((70, saida.shape[1], 3), np.uint8)
 49      _texto(barra, _sem_acentos(estado.mensagem), (10, 28), cor, 0.7)
 50      info = []
 51      if estado.distancia is not None:
 52          info.append(f"distancia {estado.distancia:.3f} (limiar
{config.LIMIAR_DISTANCIA})")
 53      if restante is not None:
 54          info.append(f"tempo {restante:0.0f}s")
 55      _texto(barra, "  |  ".join(info), (10, 56), BRANCO, 0.5, 1)
 56      return np.vstack([saida, barra])
 57
 58
 59  def _mostrar_resultado(quadro, permitido, linhas, segundos=2.5):
 60      tela = quadro.copy()
 61      cor = VERDE if permitido else VERMELHO
 62      cv2.rectangle(tela, (0, 0), (tela.shape[1] - 1, tela.shape[0] - 1), cor, 14)
 63      _texto(tela, "ACESSO PERMITIDO" if permitido else "ACESSO NEGADO", (30, 70), cor,
1.2, 3)
 64      for i, linha in enumerate(linhas):
 65          _texto(tela, _sem_acentos(linha), (30, 120 + 34 * i), BRANCO, 0.65)
 66      cv2.imshow(JANELA, tela)
 67      cv2.waitKey(int(segundos * 1000))
 68
 69
 70  def montar_painel(r, previsao=None):
 71      """Imagem com as cinco fases lado a lado (usada na apresentação)."""
 72      lado = 220
 73
 74      def bloco(img, titulo):
 75          if img.ndim == 2:
 76              img = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
 77          img = cv2.resize(img, (lado, lado), interpolation=cv2.INTER_NEAREST)
 78          _texto(img, titulo, (6, 20), AMARELO, 0.5, 1)
 79          return img
 80
 81      vazio = np.zeros((lado, lado, 3), np.uint8)
 82      original = r.quadro if r.quadro.ndim == 3 else cv2.cvtColor(r.quadro,
cv2.COLOR_GRAY2BGR)
 83      original = original.copy()
 84      if r.caixa:
 85          x, y, w, h = r.caixa
 86          cv2.rectangle(original, (x, y), (x + w, y + h), VERDE, 2)
 87      t1 = bloco(original, "1 Aquisicao")
```

```
 88      t2 = bloco(r.recorte, "2 Pre-proc.") if r.recorte is not None else bloco(vazio, "2
Pre-proc.")
 89      t3 = bloco(r.normalizada, "3 Segmentacao") if r.normalizada is not None else
bloco(vazio, "3 Segmentacao")
 90      t4 = bloco(r.lbp, "4 LBP") if r.lbp is not None else bloco(vazio, "4 LBP")
 91
 92      t5 = np.zeros((lado, lado, 3), np.uint8)
 93      _texto(t5, "5 Classificacao", (6, 20), AMARELO, 0.5, 1)
 94      if previsao is not None:
 95          cor = VERDE if previsao.aceito else VERMELHO
 96          _texto(t5, f"dist {previsao.distancia:.3f}", (10, 70), BRANCO, 0.6)
 97          _texto(t5, f"limiar {config.LIMIAR_DISTANCIA}", (10, 100), BRANCO, 0.6)
 98          _texto(t5, "RECONHECIDO" if previsao.aceito else "DESCONHECIDO", (10, 150), cor,
0.7)
 99      elif r.vetor is not None:
100          pontos = np.clip(r.vetor[:lado] / max(r.vetor[:lado].max(), 1e-9), 0, 1)
101          for i, v in enumerate(pontos):
102              cv2.line(t5, (i, lado - 1), (i, lado - 1 - int(v * 150)), AZUL, 1)
103      return np.hstack([t1, t2, t3, t4, t5])
104
105
106  # ---------------------------------------------------------------------------
107  # operações com câmera
108  # ---------------------------------------------------------------------------
109  def _janela_aberta():
110      try:
111          return cv2.getWindowProperty(JANELA, cv2.WND_PROP_VISIBLE) >= 1
112      except cv2.error:
113          return False
114
115
116  def capturar_amostras(cofre, quantidade=None):
117      """Captura rostos de qualidade para o cadastro. Devolve a lista de faces
118      normalizadas, ou None se o usuário cancelar."""
119      quantidade = quantidade or config.AMOSTRAS_POR_CADASTRO
120      faces, ultimo, inicio = [], 0.0, time.monotonic()
121      dicas = ["Olhe de frente", "Vire levemente para a esquerda", "Vire levemente para a
direita",
122               "Incline a cabeca um pouco", "Sorria", "Expressao neutra"]
123      with Camera() as camera:
124          while len(faces) < quantidade:
125              if time.monotonic() - inicio > 120:
126                  raise ErroCofre("Tempo esgotado na captura. Tente de novo em local mais
iluminado.")
127              quadro = camera.ler()
128              r = cofre.pipeline.processar(quadro)
129              estado = "Posicione o rosto na camera"
130              cor = AMARELO
131              if r.tem_rosto and r.intruso:
132                  estado = "Apenas uma pessoa no cadastro"
133              elif r.tem_rosto and not r.qualidade_ok:
134                  estado = "Qualidade ruim: " + ", ".join(r.motivos)
135              elif r.tem_rosto:
136                  cor = VERDE
137                  estado = dicas[(len(faces) * len(dicas)) // quantidade]
138                  if time.monotonic() - ultimo >= config.INTERVALO_ENTRE_AMOSTRAS:
139                      faces.append(r.normalizada)
140                      ultimo = time.monotonic()
141              tela = quadro.copy()
142              if r.caixa:
143                  x, y, w, h = r.caixa
144                  cv2.rectangle(tela, (x, y), (x + w, y + h), cor, 3)
145              _texto(tela, f"Amostras {len(faces)}/{quantidade}", (10, 28), BRANCO, 0.8)
146              _texto(tela, estado, (10, 460), cor, 0.7)
147              cv2.imshow(JANELA, tela)
148              tecla = cv2.waitKey(1) & 0xFF
149              if tecla == 27 or not _janela_aberta():
150                  cv2.destroyAllWindows()
151                  return None
152      cv2.destroyAllWindows()
153      return faces
154
155
156  def autenticar_ao_vivo(cofre, compartimento):
157      """Autentica uma pessoa pela câmera. Devolve (permitido, usuario, motivo).
158
159      `usuario` é None quando a identidade não foi confirmada.
```

```
    """
    cofre.bloqueio.exigir_liberado()
    if cofre.reconhecedor is None:
        cofre.carregar_modelo()
    compartimento = cofre.resolver_compartimento(compartimento)
    sessao = aut.SessaoAutenticacao(cofre.reconhecedor)
    inicio = time.monotonic()
    try:
        with Camera() as camera:
            while True:
                quadro = camera.ler()
                r = cofre.pipeline.processar(quadro)
                estado = sessao.processar(r)
                restante = max(0, config.TEMPO_MAX_AUTENTICACAO - (time.monotonic() -
inicio))
                cv2.imshow(JANELA, _desenhar_estado(quadro, r, estado, restante))
                tecla = cv2.waitKey(1) & 0xFF
                if tecla == 27 or not _janela_aberta():
                    return False, None, "cancelado pelo usuário"
                if estado.finalizado:
                    break
            if estado.fase == aut.FALHA:
                bloqueou = cofre.registrar_falha_de_identificacao(
                    compartimento, estado.motivo_falha, quadro, estado.distancia,
                    estado.usuario_id)
                _mostrar_resultado(quadro, False, [estado.motivo_falha] +
                                   (["Terminal bloqueado por 60 s"] if bloqueou else []))
                return False, None, estado.motivo_falha
            decisao, usuario, bloqueou = cofre.concluir_identificacao(
                estado.usuario_id, compartimento, estado.distancia)
            linhas = [usuario["nome"], config.NOMES_NIVEIS[usuario["nivel"]],
                      compartimento["nome"], decisao.motivo]
            if bloqueou:
                linhas.append("Terminal bloqueado por 60 s")
            _mostrar_resultado(quadro, decisao.permitido, linhas)
            return decisao.permitido, usuario, decisao.motivo
    finally:
        cv2.destroyAllWindows()


def demonstrar_fases(cofre):
    """Mostra ao vivo as cinco fases; se houver modelo, mostra também o veredito."""
    previsor = None
    try:
        previsor = cofre.carregar_modelo()
    except ModeloIndisponivel:
        pass
    try:
        with Camera() as camera:
            while True:
                r = cofre.pipeline.processar(camera.ler())
                previsao = previsor.prever(r.vetor) if (previsor and r.vetor is not None)
else None
                painel = montar_painel(r, previsao)
                cv2.imshow(JANELA, painel)
                if cv2.waitKey(1) & 0xFF == 27 or not _janela_aberta():
                    break
    finally:
        cv2.destroyAllWindows()


def diagnosticar_camera(cofre, segundos=12):
    """Mede, durante alguns segundos, a detecção de rosto e olhos e quantas
    piscadas o desafio reconhece. Não grava nenhuma imagem."""
    from .prova_vida import DesafioPiscada
    desafio = DesafioPiscada()
    n = com_rosto = qualidade = dois_olhos = piscadas = 0
    tempos = []
    fim = time.monotonic() + segundos
    try:
        with Camera() as camera:
            while time.monotonic() < fim:
                quadro = camera.ler()
                t0 = time.perf_counter()
                r = cofre.pipeline.processar(quadro)
                tempos.append(time.perf_counter() - t0)
                n += 1
```

```
235                 if r.tem_rosto:
236                     com_rosto += 1
237                     qualidade += int(r.qualidade_ok)
238                     dois_olhos += int(r.olhos >= 2)
239                     desafio.registrar(r.olhos)
240                     if desafio.concluido:
241                         piscadas += 1
242                         desafio.reiniciar()
243                 else:
244                     desafio.registrar(None)
245                 tela = quadro.copy()
246                 if r.caixa:
247                     x, y, w, h = r.caixa
248                     cv2.rectangle(tela, (x, y), (x + w, y + h), VERDE, 2)
249                 _texto(tela, "Olhe para a camera e PISQUE varias vezes", (10, 28),
AMARELO, 0.7)
250                 _texto(tela, f"olhos detectados: {r.olhos}   piscadas: {piscadas}   "
251                              f"restam {max(0, fim - time.monotonic()):.0f}s", (10, 460),
BRANCO, 0.6)
252                 cv2.imshow(JANELA, tela)
253                 if cv2.waitKey(1) & 0xFF == 27 or not _janela_aberta():
254                     break
255     finally:
256         cv2.destroyAllWindows()
257     if n == 0:
258         return "Nenhum quadro foi processado."
259     pct = lambda v: 100.0 * v / n
260     return "\n".join([
261         f"Quadros processados: {n}  ({1.0 / np.mean(tempos):.0f} quadros/s de
processamento)",
262         f"Rosto detectado:       {pct(com_rosto):5.1f}% dos quadros",
263         f"Qualidade aprovada:    {pct(qualidade):5.1f}% dos quadros",
264         f"Dois olhos abertos:    {pct(dois_olhos):5.1f}% dos quadros",
265         f"Piscadas reconhecidas: {piscadas}",
266     ])
267
268
269 # ---------------------------------------------------------------------------
270 # menus
271 # ---------------------------------------------------------------------------
272 def _perguntar(texto, padrao=None):
273     resposta = input(texto).strip()
274     return resposta or padrao
275
276
277 def _escolher_nivel():
278     print("\nNíveis de segurança:")
279     for n, nome in config.NOMES_NIVEIS.items():
280         print(f"  {n} - {nome}")
281     while True:
282         valor = _perguntar("Nível (1-3): ")
283         if valor in ("1", "2", "3"):
284             return int(valor)
285         print("Digite 1, 2 ou 3.")
286
287
288 def _escolher_divisao(cofre):
289     divisoes = cofre.banco.listar_divisoes()
290     print("\nDivisões:")
291     for i, d in enumerate(divisoes, start=1):
292         print(f"  {i} - {d}")
293     while True:
294         valor = _perguntar("Divisão: ")
295         if valor and valor.isdigit() and 1 <= int(valor) <= len(divisoes):
296             return divisoes[int(valor) - 1]
297         print("Escolha o número de uma divisão da lista.")
298
299
300 def _escolher_compartimento(cofre):
301     compartimentos = cofre.banco.listar_compartimentos()
302     print("\nCompartimentos do cofre:")
303     for c in compartimentos:
304         extra = f" - divisão {c['divisao']}" if c["divisao"] else ""
305         print(f"  {c['id']} - {c['nome']} (nível mínimo {c['nivel_minimo']}{extra})")
306     ids = {c["id"] for c in compartimentos}
307     while True:
308         valor = _perguntar("Compartimento que deseja abrir: ")
```

```
309          if valor and valor.isdigit() and int(valor) in ids:
310              return int(valor)
311          print("Número inválido.")
312
313
314  class Menu:
315      def __init__(self, cofre):
316          self.cofre = cofre
317          self._admin_ate = 0.0
318          self._admin_nome = "(administrador)"
319
320      # -- administração protegida pelo ministro ----------------------------------
321      def _exigir_ministro(self):
322          """True se o ministro estiver autenticado (sessão de 5 min)."""
323          if time.monotonic() < self._admin_ate:
324              return True
325          print("\nEsta operação exige a autenticação facial do ministro (Nível 3).")
326          permitido, usuario, motivo = autenticar_ao_vivo(self.cofre, ADMINISTRACAO)
327          if permitido:
328              self._admin_ate = time.monotonic() + _DURACAO_ADMIN
329              self._admin_nome = usuario["nome"]
330              return True
331          print(f"Operação negada: {motivo}.")
332          return False
333
334      # -- opções -----------------------------------------------------------------
335      def cadastrar(self, local=False, substituindo=None):
336          """Cadastra um usuário pela câmera.
337
338          Regras de quem pode cadastrar (todas verificadas no código):
339            * banco vazio: o primeiro cadastro é o do ministro, sem autenticação (não há
340              ninguém para autenticar);
341            * há ministro: ele precisa se autenticar (sessão de 5 min);
342            * há usuários mas NÃO há ministro: o menu recusa. Cadastrar o ministro nesse
343              estado, sem autenticação, deixaria qualquer um com acesso ao menu virar
344              ministro; só o administrador da máquina, pelo comando local
345              `trocar-ministro` (local=True), pode fazê-lo;
346            * `substituindo`: troca atômica do ministro (o antigo volta se o novo falhar).
347          """
348          banco = self.cofre.banco
349          sem_ministro = banco.contar_ativos(config.NIVEL_MINISTRO) == 0
350          primeiro = sem_ministro and banco.contar_ativos() == 0
351          como_ministro = False
352          if substituindo is not None or (sem_ministro and local):
353              como_ministro = True
354          elif primeiro:
355              como_ministro = True
356              print("\nNenhum usuário cadastrado. O primeiro cadastro é o do ministro
(Nível 3).")
357          elif sem_ministro:
358              print("Não há ministro ativo. Para recuperar a administração, um
administrador da "
359                    "máquina deve executar: python -m cofre trocar-ministro")
360              return
361          elif not self._exigir_ministro():
362              return
363          nome = _perguntar("Nome completo: ")
364          cargo = _perguntar("Cargo: ", "")
365          nivel = config.NIVEL_MINISTRO if como_ministro else _escolher_nivel()
366          divisao = _escolher_divisao(self.cofre) if nivel == config.NIVEL_DIRETOR else
None
367          if substituindo is None:
368              banco.validar_novo_usuario(nome, cargo, nivel, divisao)  # falha cedo
369          elif len((nome or "").strip()) < 3:
370              raise UsuarioInvalido("O nome deve ter pelo menos 3 caracteres.")
371          print("\nA janela da câmera vai abrir. Mantenha apenas o seu rosto no quadro e "
372                "siga as instruções. ESC cancela.")
373          faces = capturar_amostras(self.cofre)
374          if faces is None:
375              print("Cadastro cancelado. Nada foi alterado.")
376              return
377          if substituindo is not None:
378              uid = self.cofre.trocar_ministro(substituindo["id"], nome, cargo, faces)
379              print(f"Ministro substituído (novo id {uid}). A biometria do anterior foi
apagada.")
380              return
381          if primeiro:
```

```
382              responsavel = "(primeiro cadastro do ministro)"
383          elif local:
384              responsavel = "(administrador local)"
385          else:
386              responsavel = self._admin_nome
387          uid = self.cofre.cadastrar(nome, cargo, nivel, divisao, faces, responsavel)
388          print(f"Usuário cadastrado (id {uid}) com {len(faces)} amostras. Modelo
atualizado.")
389
390      def autenticar(self):
391          if self.cofre.banco.contar_ativos() == 0:
392              print("Ainda não há usuários cadastrados.")
393              return
394          # Se o modelo estiver ausente ou adulterado, carregar_modelo explica o motivo
395          # (a mensagem chega ao usuário pelo tratamento de ModeloIndisponivel do menu).
396          self.cofre.carregar_modelo()
397          comp = _escolher_compartimento(self.cofre)
398          permitido, usuario, motivo = autenticar_ao_vivo(self.cofre, comp)
399          print(("ACESSO PERMITIDO" if permitido else "ACESSO NEGADO") +
400                (f" - {usuario['nome']}" if usuario else "") + f" ({motivo})")
401
402      def listar(self):
403          usuarios = self.cofre.banco.listar_usuarios()
404          if not usuarios:
405              print("Nenhum usuário cadastrado.")
406          for u in usuarios:
407              div = f" | divisão {u['divisao']}" if u["divisao"] else ""
408              print(f"  [{u['id']}] {u['nome']} - {u['cargo']} |
{config.NOMES_NIVEIS[u['nivel']]}{div}")
409
410      def remover(self):
411          if not self._exigir_ministro():
412              return
413          self.listar()
414          valor = _perguntar("Id do usuário a remover: ")
415          if not (valor and valor.isdigit()):
416              print("Id inválido.")
417              return
418          usuario = self.cofre.banco.obter_usuario(int(valor))
419          if usuario is None or not usuario["ativo"]:
420              print("Usuário não encontrado.")
421              return
422          if _perguntar(f"Remover '{usuario['nome']}' e apagar a biometria? (s/N): ",
"n").lower() == "s":
423              self.cofre.remover_usuario(usuario["id"], responsavel=self._admin_nome)
424              print("Usuário removido (amostras biométricas apagadas) e modelo atualizado.
"
425                    "O histórico de acessos é mantido.")
426
427      def relatorio(self):
428          if not self._exigir_ministro():
429              return
430          _, resumo = relatorios.relatorio_acessos(self.cofre.banco)
431          print("\n" + resumo)
432          print("Lista de usuários:", relatorios.relatorio_usuarios(self.cofre.banco))
433
434      def integridade(self):
435          ok, linha = self.cofre.banco.verificar_integridade()
436          if ok:
437              print("Histórico ÍNTEGRO.")
438          elif linha == 0:
439              print("HISTÓRICO SUSPEITO: a âncora (total de linhas e último hash) está
ausente.")
440          else:
441              print(f"HISTÓRICO VIOLADO a partir da linha {linha}!")
442
443      def executar(self):
444          opcoes = {
445              "1": ("Cadastrar usuário", self.cadastrar),
446              "2": ("Autenticar e abrir compartimento", self.autenticar),
447              "3": ("Listar usuários", self.listar),
448              "4": ("Remover usuário", self.remover),
449              "5": ("Relatórios de acesso", self.relatorio),
450              "6": ("Verificar integridade do histórico", self.integridade),
451              "7": ("Demonstração das 5 fases (ao vivo)", lambda:
demonstrar_fases(self.cofre)),
452              "8": ("Diagnóstico da câmera e da prova de vida",
```

```
453                     lambda: print(diagnosticar_camera(self.cofre))),
454             "0": ("Sair", None),
455         }
456         while True:
457             print("\n=== COFRE DE SEGURANÇA MÁXIMA - Ministério do Meio Ambiente ===")
458             for chave, (titulo, _) in opcoes.items():
459                 print(f"  {chave} - {titulo}")
460             try:
461                 escolha = input("Opção: ").strip()
462             except (EOFError, KeyboardInterrupt):
463                 print()
464                 return
465             if escolha == "0":
466                 return
467             if escolha not in opcoes:
468                 print("Opção inválida.")
469                 continue
470             try:
471                 opcoes[escolha][1]()
472             except SistemaBloqueado as e:
473                 print(f"Terminal bloqueado por falhas consecutivas. Aguarde
{e.segundos_restantes} s.")
474             except CameraIndisponivel as e:
475                 print(f"Câmera indisponível: {e}")
476             except ErroCaptura as e:
477                 print(f"Falha na captura: {e}")
478             except ModeloIndisponivel as e:
479                 print(f"Modelo: {e}")
480             except ErroCofre as e:
481                 print(f"Erro: {e}")
482             except KeyboardInterrupt:
483                 cv2.destroyAllWindows()
484                 print("\nOperação interrompida.")
```

2 TESTES AUTOMATIZADOS

2.1 testes/_ambiente.py

Isola os testes em uma pasta temporária.

```
 1  """Isola os testes: aponta COFRE_DADOS para uma pasta temporária ANTES de o
 2  pacote `cofre` ser importado, para nunca tocar nos dados reais."""
 3
 4  import atexit
 5  import os
 6  import shutil
 7  import tempfile
 8  from pathlib import Path
 9
10  if "COFRE_TESTE_DIR" not in os.environ:
11      _tmp = tempfile.mkdtemp(prefix="cofre_teste_")
12      os.environ["COFRE_TESTE_DIR"] = _tmp
13      os.environ["COFRE_DADOS"] = _tmp
14      atexit.register(shutil.rmtree, _tmp, ignore_errors=True)
15
16  PASTA_DADOS = Path(os.environ["COFRE_DADOS"])
17  BASE_ORL = Path(__file__).resolve().parent.parent.parent / "base_teste" / "orl"
```

2.2 testes/test_fases.py

Testes das fases de pré-processamento, segmentação, características e classificação.

```
  1  """Testes das fases de pré-processamento, segmentação, características e
classificação."""
  2
  3  import unittest
  4
  5  import numpy as np
  6
  7  import _ambiente  # noqa: F401  (deve vir antes dos imports de cofre)
  8  from cofre import caracteristicas, classificacao, config, preprocessamento, segmentacao
  9  from cofre.erros import ModeloIndisponivel
 10
 11
 12  class TestCaracteristicas(unittest.TestCase):
 13      def test_codigo_lbp_de_bloco_conhecido(self):
 14          # Centro 50; vizinhos no sentido horário a partir do canto superior
 15          # esquerdo: 60 10 60 / 10 . 60 / 10 60 10 -> bits 1,0,1,1,0,1,0,0
 16          bloco = np.array([[60, 10, 60],
 17                            [10, 50, 60],
 18                            [10, 60, 10]], dtype=np.uint8)
 19          # ordem dos vizinhos: (0,0)=60 (0,1)=10 (0,2)=60 (1,2)=60 (2,2)=10 (2,1)=60
(2,0)=10 (1,0)=10
 20          # bits: 1 0 1 1 0 1 0 0 -> 10110100 = 180
 21          self.assertEqual(int(caracteristicas.imagem_lbp(bloco)[0, 0]), 0b10110100)
 22
 23      def test_tabela_tem_58_uniformes_e_uma_classe_outros(self):
 24          tabela = caracteristicas.TABELA_UNIFORMES
 25          self.assertEqual(len(set(tabela.tolist())), 59)
 26          self.assertEqual(int((tabela == 58).sum()), 256 - 58)
 27          self.assertEqual(int(tabela[0b00000000]), int(tabela[0b00000000]))  #
determinismo
 28          self.assertNotEqual(int(tabela[0b00000000]), 58)   # só zeros é uniforme
 29          self.assertEqual(int(tabela[0b01010101]), 58)      # muitas transições
 30
 31      def test_vetor_tem_dimensao_e_soma_esperadas(self):
 32          rng = np.random.default_rng(0)
 33          face = rng.integers(0, 256, config.TAMANHO_FACE[::-1], dtype=np.uint8)
 34          v = caracteristicas.extrair(face)
 35          blocos = config.GRADE_BLOCOS[0] * config.GRADE_BLOCOS[1]
 36          self.assertEqual(v.shape, (blocos * 59,))
 37          self.assertAlmostEqual(float(v.sum()), blocos, places=6)  # cada bloco soma 1
 38
 39      def test_lbp_e_invariante_a_brilho_aditivo(self):
```

```
 40          rng = np.random.default_rng(1)
 41          face = rng.integers(0, 150, (60, 60), dtype=np.uint8)
 42          np.testing.assert_array_equal(caracteristicas.imagem_lbp(face),
 43                                        caracteristicas.imagem_lbp(face + 50))
 44
 45
 46  class TestPreProcessamento(unittest.TestCase):
 47      def test_normalizar_devolve_tamanho_padrao(self):
 48          face = np.full((150, 130), 120, np.uint8)
 49          saida = preprocessamento.normalizar_face(face)
 50          self.assertEqual(saida.shape, config.TAMANHO_FACE[::-1])
 51
 52      def test_qualidade_reprova_escuro_claro_e_desfocado(self):
 53          rng = np.random.default_rng(2)
 54          bom = rng.integers(40, 200, (120, 120), dtype=np.uint8)
 55          self.assertTrue(preprocessamento.avaliar_qualidade(bom)[0])
 56          escuro = (bom // 8).astype(np.uint8)
 57          self.assertIn("imagem muito escura",
preprocessamento.avaliar_qualidade(escuro)[1])
 58          claro = np.full((120, 120), 250, np.uint8)
 59          self.assertIn("imagem muito clara", preprocessamento.avaliar_qualidade(claro)[1])
 60          liso = np.full((120, 120), 120, np.uint8)
 61          self.assertIn("imagem desfocada", preprocessamento.avaliar_qualidade(liso)[1])
 62
 63
 64  class TestSegmentacao(unittest.TestCase):
 65      def test_intruso_so_quando_segundo_rosto_e_comparavel(self):
 66          self.assertFalse(segmentacao.ha_intruso([(0, 0, 200, 200)]))
 67          self.assertFalse(segmentacao.ha_intruso([(0, 0, 200, 200), (0, 0, 90, 90)]))
 68          self.assertTrue(segmentacao.ha_intruso([(0, 0, 200, 200), (0, 0, 180, 180)]))
 69
 70      def test_angulo_entre_olhos(self):
 71          olhos = [(10, 40, 20, 20), (70, 40, 20, 20)]          # mesma altura
 72          self.assertAlmostEqual(segmentacao.angulo_entre_olhos(olhos, 100), 0.0, places=5)
 73          inclinado = [(10, 40, 20, 20), (70, 50, 20, 20)]      # direito mais baixo
 74          self.assertGreater(segmentacao.angulo_entre_olhos(inclinado, 100), 5)
 75          colados = [(40, 40, 20, 20), (45, 40, 20, 20)]
 76          self.assertIsNone(segmentacao.angulo_entre_olhos(colados, 100))
 77          self.assertIsNone(segmentacao.angulo_entre_olhos(olhos[:1], 100))
 78
 79      def test_mascara_zera_cantos_e_preserva_centro(self):
 80          face = np.full((100, 100), 200, np.uint8)
 81          m = segmentacao.mascara_eliptica(face)
 82          self.assertEqual(int(m[0, 0]), 0)
 83          self.assertEqual(int(m[50, 50]), 200)
 84
 85      def test_recortar_respeita_bordas(self):
 86          img = np.zeros((100, 100), np.uint8)
 87          r = segmentacao.recortar(img, (80, 80, 40, 40))
 88          self.assertLessEqual(r.shape[0], 100)
 89          self.assertLessEqual(r.shape[1], 100)
 90
 91
 92  class TestClassificacao(unittest.TestCase):
 93      def _modelo(self):
 94          rng = np.random.default_rng(3)
 95          centros = [rng.dirichlet(np.ones(59 * 64) * 0.3) for _ in range(3)]
 96          vetores, rotulos = [], []
 97          for uid, c in enumerate(centros, start=1):
 98              for _ in range(5):
 99                  ruido = np.abs(c + rng.normal(0, 0.00003, c.shape))
100                  vetores.append(ruido / ruido.sum() * 64)
101                  rotulos.append(uid)
102          vetores = np.array(vetores)
103          return classificacao.Reconhecedor(limiar=0.2).treinar(vetores, rotulos), centros
104
105      def test_distancia_zero_para_iguais_e_simetrica(self):
106          a = np.random.default_rng(4).dirichlet(np.ones(59))
107          b = np.random.default_rng(5).dirichlet(np.ones(59))
108          self.assertAlmostEqual(float(classificacao.distancia_qui_quadrado(a, a)), 0.0)
109          self.assertAlmostEqual(float(classificacao.distancia_qui_quadrado(a, b)),
110                                 float(classificacao.distancia_qui_quadrado(b, a)))
111
112      def test_reconhece_usuario_cadastrado_e_rejeita_desconhecido(self):
113          modelo, centros = self._modelo()
114          consulta = centros[1] / centros[1].sum() * 64
115          previsao = modelo.prever(consulta)
```

```
116          self.assertEqual(previsao.usuario_id, 2)
117          self.assertTrue(previsao.aceito)
118          estranho = np.random.default_rng(99).dirichlet(np.ones(59 * 64) * 0.3) * 64
119          self.assertFalse(modelo.prever(estranho).aceito)
120
121      def test_modelo_nao_treinado_levanta_erro(self):
122          with self.assertRaises(ModeloIndisponivel):
123              classificacao.Reconhecedor().prever(np.zeros(59))
124
125      def test_salvar_carregar_e_deteccao_de_adulteracao(self):
126          import tempfile
127          from pathlib import Path
128          modelo, _ = self._modelo()
129          with tempfile.TemporaryDirectory() as tmp:
130              caminho = Path(tmp) / "m.npz"
131              h = modelo.salvar(caminho)
132              carregado = classificacao.Reconhecedor.carregar(caminho, hash_esperado=h)
133              self.assertEqual(carregado.usuarios, [1, 2, 3])
134              with open(caminho, "ab") as f:
135                  f.write(b"x")                    # adultera o arquivo
136              with self.assertRaises(ModeloIndisponivel):
137                  classificacao.Reconhecedor.carregar(caminho, hash_esperado=h)
138
139      def test_confirmador_exige_consistencia(self):
140          c = classificacao.Confirmador(janela=5, necessarios=3)
141          for voto in (1, None, 1, 2):
142              self.assertIsNone(c.registrar(voto))
143          self.assertEqual(c.registrar(1), 1)
144          c.reiniciar()
145          self.assertIsNone(c.confirmado())
146
147
148  if __name__ == "__main__":
149      unittest.main()
```

2.3 testes/test_integracao.py

Teste de ponta a ponta sem câmera, com a base ORL.

```
  1  """Teste de ponta a ponta sem câmera, com fotos da base ORL (se estiver na pasta).
  2
  3  Cadastra três pessoas (uma por nível), autentica por arquivo e confere
  4  permissões, recusa de intruso, registro de histórico e bloqueio do terminal.
  5  """
  6
  7  import unittest
  8
  9  import _ambiente  # noqa: F401
 10  from cofre import config
 11  from cofre.aquisicao import carregar_imagem
 12  from cofre.banco import Banco
 13  from cofre.erros import SistemaBloqueado
 14  from cofre.servico import Cofre
 15
 16  PESSOAS = {"ministro": "s01", "diretor": "s03", "geral": "s05", "intruso": "s02"}
 17  CADASTRO = range(1, 8)       # imagens 1..7 para cadastro
 18  SONDA = "08.png"             # imagem não usada no cadastro
 19
 20
 21  def _caminho(pessoa, arquivo):
 22      return _ambiente.BASE_ORL / pessoa / arquivo
 23
 24
 25  @unittest.skipUnless(_ambiente.BASE_ORL.is_dir(), "base ORL não encontrada")
 26  class TestIntegracao(unittest.TestCase):
 27      @classmethod
 28      def setUpClass(cls):
 29          cls.cofre = Cofre(Banco(_ambiente.PASTA_DADOS / "integracao.db"))
 30          pip = cls.cofre.pipeline
 31
 32          def faces(pessoa):
 33              return [pip.processar_recorte(carregar_imagem(_caminho(pessoa,
f"{i:02d}.png"))).normalizada
 34                      for i in CADASTRO]
 35
```

```
 36          cls.cofre.cadastrar("Marina Ministra", "Ministra do Meio Ambiente", 3, None,
 37                              faces(PESSOAS["ministro"]))
 38          cls.cofre.cadastrar("Dora Diretora", "Diretora", 2, "Agrotóxicos e Biocidas",
 39                              faces(PESSOAS["diretor"]))
 40          cls.cofre.cadastrar("Gabriel Geral", "Analista", 1, None,
faces(PESSOAS["geral"]))
 41          comps = {c["nome"]: c["id"] for c in cls.cofre.banco.listar_compartimentos()}
 42          cls.geral = comps["Arquivo Geral de Relatórios Ambientais"]
 43          cls.agro = comps["Laboratório de Toxinas Agrícolas"]
 44          cls.residuos = comps["Laboratório de Resíduos Perigosos"]
 45          cls.central = comps["Câmara Central de Toxinas de Alto Risco"]
 46
 47      def _entrar(self, quem, comp):
 48          return self.cofre.autenticar_arquivo(_caminho(PESSOAS[quem], SONDA), comp,
so_recorte=True)
 49
 50      def setUp(self):
 51          # A base ORL tem muito mais variação de pose do que uma webcam na mesma
 52          # sessão: as distâncias legítimas ficam em 0,30-0,51 e as de intrusos em
 53          # 0,56-0,59. O teste verifica o FLUXO (política, histórico, bloqueio),
 54          # não a taxa de acerto, que é medida em avaliacao.py; por isso usa um
 55          # limiar que separa as duas faixas em vez do limiar de produção.
 56          if self.cofre.reconhecedor is not None:
 57              self.cofre.reconhecedor.limiar = 0.53
 58          self.cofre.banco.definir_parametro("bloqueio_ate", 0)
 59          self.cofre.banco.definir_parametro("falhas_consecutivas", 0)
 60
 61      def test_modelo_foi_treinado_e_tem_tres_usuarios(self):
 62          self.assertTrue(self.cofre.modelo_pronto())
 63          self.assertEqual(len(self.cofre.reconhecedor.usuarios), 3)
 64
 65      def test_ministro_entra_em_todos_os_compartimentos(self):
 66          for comp in (self.geral, self.agro, self.residuos, self.central):
 67              r = self._entrar("ministro", comp)
 68              self.assertTrue(r["permitido"], r)
 69              self.assertEqual(r["usuario"], "Marina Ministra")
 70
 71      def test_diretor_so_na_propria_divisao(self):
 72          self.assertTrue(self._entrar("diretor", self.agro)["permitido"])
 73          self.assertFalse(self._entrar("diretor", self.residuos)["permitido"])
 74          self.assertFalse(self._entrar("diretor", self.central)["permitido"])
 75
 76      def test_geral_so_no_arquivo_geral(self):
 77          self.assertTrue(self._entrar("geral", self.geral)["permitido"])
 78          self.assertFalse(self._entrar("geral", self.agro)["permitido"])
 79          self.assertFalse(self._entrar("geral", self.central)["permitido"])
 80
 81      def test_intruso_e_negado_e_sem_identidade(self):
 82          r = self._entrar("intruso", self.central)
 83          self.assertFalse(r["permitido"])
 84          self.assertIsNone(r["usuario"])
 85
 86      def test_bloqueio_apos_falhas_consecutivas(self):
 87          for _ in range(config.TENTATIVAS_ATE_BLOQUEIO):
 88              self.assertFalse(self._entrar("intruso", self.central)["permitido"])
 89          with self.assertRaises(SistemaBloqueado):
 90              self._entrar("ministro", self.central)
 91
 92      def test_historico_registra_tudo_e_continua_integro(self):
 93          self._entrar("ministro", self.central)
 94          self._entrar("intruso", self.central)
 95          acessos = self.cofre.banco.listar_acessos()
 96          self.assertGreater(len(acessos), 0)
 97          self.assertEqual(self.cofre.banco.verificar_integridade(), (True, None))
 98
 99      def test_negacao_de_politica_nao_bloqueia_o_terminal(self):
100          # quem é reconhecido mas pede um compartimento proibido é negado e registrado,
101          # sem acionar o bloqueio (que protege o reconhecimento contra tentativas em
série)
102          for comp in (self.residuos, self.central, self.residuos, self.central):
103              self.assertFalse(self._entrar("diretor", comp)["permitido"])
104          self.cofre.bloqueio.exigir_liberado()                    # não levanta
105          self.assertTrue(self._entrar("diretor", self.agro)["permitido"])
106
107      def test_motivo_por_arquivo_fica_marcado_no_historico(self):
108          self._entrar("ministro", self.central)
109          ultimo = self.cofre.banco.listar_acessos()[-1]
```

```
110          self.assertIn("sem prova de vida", ultimo["motivo"])
111
112      def test_ministro_nao_pode_ser_removido_pelo_menu(self):
113          from cofre.erros import RegraDeNegocio
114          ministro = next(u for u in self.cofre.banco.listar_usuarios() if u["nivel"] == 3)
115          with self.assertRaises(RegraDeNegocio):
116              self.cofre.remover_usuario(ministro["id"])
117          self.assertTrue(self.cofre.banco.obter_usuario(ministro["id"])["ativo"])
118
119      def test_cadastro_e_remocao_aparecem_no_historico(self):
120          eventos = [a for a in self.cofre.banco.listar_acessos() if a["resultado"] ==
"ADMIN"]
121          self.assertTrue(any("cadastrado" in e["motivo"] for e in eventos))
122
123      def test_modelo_adulterado_e_recuperavel_por_comando_local(self):
124          from cofre import config as cfg
125          from cofre.erros import ModeloIndisponivel
126          try:
127              with open(cfg.ARQUIVO_MODELO, "ab") as f:
128                  f.write(b"x")                                   # adultera o arquivo
129              with self.assertRaises(ModeloIndisponivel) as ctx:
130                  self.cofre.carregar_modelo()
131              self.assertIn("retreinar", str(ctx.exception))      # a mensagem diz o que
fazer
132          finally:
133              self.cofre.retreinar_modelo()
134          self.assertTrue(self.cofre.modelo_pronto())
135          self.assertTrue(any("retreinado" in a["motivo"] for a in
self.cofre.banco.listar_acessos()))
136
137      def test_modelo_sem_impressao_digital_falha_fechado(self):
138          from cofre.erros import ModeloIndisponivel
139          hash_ok = self.cofre.banco.obter_parametro("hash_modelo")
140          try:
141              self.cofre.banco.definir_parametro("hash_modelo", "")
142              with self.assertRaises(ModeloIndisponivel):
143                  self.cofre.carregar_modelo()
144          finally:
145              self.cofre.banco.definir_parametro("hash_modelo", hash_ok)
146          self.assertTrue(self.cofre.modelo_pronto())
147
148      def test_troca_de_ministro_e_atomica(self):
149          # se o cadastro do novo ministro falhar, o antigo volta (ativo, com biometria e
modelo)
150          ministro = next(u for u in self.cofre.banco.listar_usuarios() if u["nivel"] == 3)
151          antes = len(self.cofre.reconhecedor.usuarios) if self.cofre.reconhecedor else 3
152          with self.assertRaises(Exception):
153              self.cofre.trocar_ministro(ministro["id"], "Novo Ministro", "x", [])   # sem
amostras
154          depois = self.cofre.banco.obter_usuario(ministro["id"])
155          self.assertTrue(depois["ativo"])
156          self.assertTrue(self.cofre.modelo_pronto())
157          self.assertEqual(len(self.cofre.reconhecedor.usuarios), antes)
158
159      def test_remover_usuario_retira_o_acesso(self):
160          uid = self.cofre.banco.criar_usuario("Temporario Teste", "x", 1)
161          self.cofre.remover_usuario(uid)
162          self.assertFalse(self.cofre.banco.obter_usuario(uid)["ativo"])
163
164
165  if __name__ == "__main__":
166      unittest.main()
```

2.4 testes/test_seguranca.py

Testes de política, banco, prova de vida, sessão e bloqueio.

```
  1  """Testes de política de acesso, banco, prova de vida, sessão e bloqueio."""
  2
  3  import tempfile
  4  import unittest
  5  from pathlib import Path
  6
  7  import numpy as np
  8
```

```
import _ambiente  # noqa: F401
from cofre import autenticacao as aut
from cofre import config, politica
from cofre.autenticacao import ControleBloqueio, SessaoAutenticacao
from cofre.banco import Banco
from cofre.classificacao import Previsao
from cofre.erros import RegraDeNegocio, SistemaBloqueado, UsuarioInvalido
from cofre.pipeline import Resultado
from cofre.prova_vida import DesafioPiscada


def _banco():
    tmp = tempfile.mkdtemp(dir=_ambiente.PASTA_DADOS)
    return Banco(Path(tmp) / "t.db")


class TestPolitica(unittest.TestCase):
    def setUp(self):
        self.banco = _banco()
        self.comp = {c["nome"]: c for c in self.banco.listar_compartimentos()}
        self.geral = self.comp["Arquivo Geral de Relatórios Ambientais"]
        self.agro = self.comp["Laboratório de Toxinas Agrícolas"]
        self.residuos = self.comp["Laboratório de Resíduos Perigosos"]
        self.central = self.comp["Câmara Central de Toxinas de Alto Risco"]

    def _usuario(self, nome, nivel, divisao=None):
        uid = self.banco.criar_usuario(nome, "cargo", nivel, divisao)
        return self.banco.obter_usuario(uid)

    def test_matriz_de_permissoes(self):
        geral = self._usuario("Maria Geral", 1)
        diretor = self._usuario("Dora Diretora", 2, "Agrotóxicos e Biocidas")
        ministro = self._usuario("Marina Ministra", 3)
        esperado = {
            # (usuário, compartimento): permitido
            ("geral", "geral"): True, ("geral", "agro"): False,
            ("geral", "residuos"): False, ("geral", "central"): False,
            ("diretor", "geral"): True, ("diretor", "agro"): True,
            ("diretor", "residuos"): False, ("diretor", "central"): False,
            ("ministro", "geral"): True, ("ministro", "agro"): True,
            ("ministro", "residuos"): True, ("ministro", "central"): True,
        }
        usuarios = {"geral": geral, "diretor": diretor, "ministro": ministro}
        comps = {"geral": self.geral, "agro": self.agro,
                 "residuos": self.residuos, "central": self.central}
        for (u, c), quer in esperado.items():
            decisao = politica.decidir(usuarios[u], comps[c])
            self.assertEqual(decisao.permitido, quer, f"{u} -> {c}: {decisao.motivo}")

    def test_usuario_desativado_e_negado(self):
        u = self._usuario("Pedro Antigo", 1)
        self.banco.desativar_usuario(u["id"])
        self.assertFalse(politica.decidir(self.banco.obter_usuario(u["id"]),
self.geral).permitido)


class TestBanco(unittest.TestCase):
    def setUp(self):
        self.banco = _banco()

    def test_validacoes_de_cadastro(self):
        with self.assertRaises(UsuarioInvalido):
            self.banco.criar_usuario("Al", "x", 1)
        with self.assertRaises(UsuarioInvalido):
            self.banco.criar_usuario("Fulano de Tal", "x", 9)
        with self.assertRaises(UsuarioInvalido):
            self.banco.criar_usuario("Diretor Sem Divisao", "x", 2)
        self.banco.criar_usuario("Ana Souza", "x", 1)
        with self.assertRaises(UsuarioInvalido):          # nome duplicado (sem caixa)
            self.banco.criar_usuario("ana souza", "x", 1)

    def test_nivel_3_e_exclusivo_do_ministro(self):
        uid = self.banco.criar_usuario("Marina Ministra", "Ministra", 3)
        with self.assertRaises(RegraDeNegocio):
            self.banco.criar_usuario("Outro Ministro", "x", 3)
        self.banco.desativar_usuario(uid)
        self.banco.criar_usuario("Outro Ministro", "x", 3)   # agora é permitido
```

```
 85
 86      def test_historico_encadeado_detecta_adulteracao(self):
 87          for i in range(4):
 88              self.banco.registrar_acesso(1, "Ana", "Arquivo", "PERMITIDO", "ok", 0.1 + i /
100)
 89          self.assertEqual(self.banco.verificar_integridade(), (True, None))
 90          self.banco._con.execute("UPDATE acessos SET resultado='NEGADO' WHERE id=2")   #
era PERMITIDO
 91          self.banco._con.execute("UPDATE acessos SET motivo='editado' WHERE id=3")
 92          self.banco._con.commit()
 93          ok, linha = self.banco.verificar_integridade()
 94          self.assertFalse(ok)
 95          self.assertEqual(linha, 2)      # a PRIMEIRA linha adulterada é a 2 (a 3 também
foi)
 96
 97      def test_apagar_as_ultimas_linhas_e_detectado_pela_ancora(self):
 98          for i in range(4):
 99              self.banco.registrar_acesso(None, "x", "c", "NEGADO", "m")
100          self.assertTrue(self.banco.verificar_integridade()[0])
101          self.banco._con.execute("DELETE FROM acessos WHERE id IN (3, 4)")   # corta o fim
102          self.banco._con.commit()
103          ok, linha = self.banco.verificar_integridade()
104          self.assertFalse(ok)
105          self.assertEqual(linha, 3)      # restam as linhas 1 e 2: a ausente é a 3
106
107      def test_ancora_ausente_com_historico_falha_fechado(self):
108          # apagar o fim do histórico E as chaves da âncora não pode parecer "íntegro"
109          for i in range(4):
110              self.banco.registrar_acesso(None, "x", "c", "NEGADO", "m")
111          self.banco._con.execute("DELETE FROM acessos WHERE id IN (3, 4)")
112          self.banco._con.execute("DELETE FROM parametros WHERE chave IN ('log_total',
'log_ultimo_hash')")
113          self.banco._con.commit()
114          self.assertFalse(self.banco.verificar_integridade()[0])
115
116      def test_historico_vazio_e_integro(self):
117          self.assertEqual(self.banco.verificar_integridade(), (True, None))
118
119      def test_eventos_administrativos_entram_no_historico(self):
120          self.banco.registrar_evento("Marina", "usuário 'X' cadastrado")
121          linha = self.banco.listar_acessos()[0]
122          self.assertEqual(linha["resultado"], "ADMIN")
123          self.assertTrue(self.banco.verificar_integridade()[0])
124
125      def test_apagar_linha_quebra_a_cadeia(self):
126          for i in range(3):
127              self.banco.registrar_acesso(None, "x", "c", "NEGADO", "m")
128          self.banco._con.execute("DELETE FROM acessos WHERE id=2")
129          self.banco._con.commit()
130          self.assertFalse(self.banco.verificar_integridade()[0])
131
132
133  class TestProvaDeVida(unittest.TestCase):
134      def test_piscada_completa_conclui(self):
135          d = DesafioPiscada()
136          for olhos in (2, 2, 2, 0, 0, 2, 2):
137              d.registrar(olhos)
138          self.assertTrue(d.concluido)
139
140      def test_um_unico_quadro_sem_olhos_nao_e_piscada(self):
141          # falha isolada do detector de Haar não pode valer como piscada
142          d = DesafioPiscada()
143          for olhos in (2, 2, 2, 0, 2, 2, 2, 2, 0, 2, 2, 2):
144              d.registrar(olhos)
145          self.assertFalse(d.concluido)
146
147      def test_olhos_sempre_abertos_nao_conclui(self):
148          d = DesafioPiscada()
149          for _ in range(50):
150              d.registrar(2)
151          self.assertFalse(d.concluido)           # foto parada: nunca pisca
152
153      def test_perder_o_rosto_reinicia_o_desafio(self):
154          d = DesafioPiscada()
155          for olhos in (2, 2, 2, 0):
156              d.registrar(olhos)
157          d.registrar(None)
```

```
158          self.assertEqual(d.estado, DesafioPiscada.AGUARDA_ABERTOS)
159
160
161  class _ReconhecedorFalso:
162      """Devolve previsões pré-programadas: (usuario_id, distancia, segunda)."""
163      def __init__(self, roteiro):
164          self.roteiro = list(roteiro)
165
166      def prever(self, vetor):
167          uid, dist, seg = self.roteiro.pop(0) if len(self.roteiro) > 1 else
self.roteiro[0]
168          return Previsao(uid, dist, seg, dist <= config.LIMIAR_DISTANCIA)
169
170
171  def _resultado(olhos=2, rosto=True, intruso=False, ok=True):
172      r = Resultado()
173      if rosto:
174          r.caixa, r.caixas = (0, 0, 100, 100), [(0, 0, 100, 100)]
175      r.intruso, r.qualidade_ok, r.olhos = intruso, ok, olhos
176      r.motivos = [] if ok else ["imagem desfocada"]
177      r.vetor = np.zeros(59)
178      return r
179
180
181  class _Relogio:
182      def __init__(self):
183          self.t = 0.0
184
185      def __call__(self):
186          return self.t
187
188
189  class TestSessao(unittest.TestCase):
190      def _sessao(self, roteiro, vida=False, relogio=None):
191          return SessaoAutenticacao(_ReconhecedorFalso(roteiro), exigir_vida=vida,
192                                    relogio=relogio or _Relogio())
193
194      def test_identifica_apos_quadros_consistentes(self):
195          s = self._sessao([(7, 0.20, 0.50)])
196          estados = [s.processar(_resultado()) for _ in range(config.QUADROS_CONFIRMACAO)]
197          self.assertEqual(estados[-1].fase, aut.IDENTIFICADO)
198          self.assertEqual(estados[-1].usuario_id, 7)
199          self.assertNotEqual(estados[0].fase, aut.IDENTIFICADO)   # um quadro só não basta
200
201      def test_distancia_alta_nunca_identifica_e_estoura_tempo(self):
202          rel = _Relogio()
203          s = self._sessao([(7, 0.90, 1.0)], relogio=rel)
204          for _ in range(5):
205              self.assertNotEqual(s.processar(_resultado()).fase, aut.IDENTIFICADO)
206          rel.t = config.TEMPO_MAX_AUTENTICACAO + 1
207          e = s.processar(_resultado())
208          self.assertEqual(e.fase, aut.FALHA)
209          self.assertIn("não reconhecido", e.motivo_falha)
210
211      def test_dois_usuarios_parecidos_geram_ambiguidade(self):
212          s = self._sessao([(7, 0.30, 0.305)])       # margem ~1,6% < mínima
213          for _ in range(config.QUADROS_JANELA):
214              self.assertNotEqual(s.processar(_resultado()).fase, aut.IDENTIFICADO)
215
216      def test_segundo_rosto_impede_a_autenticacao(self):
217          rel = _Relogio()
218          s = self._sessao([(7, 0.20, 0.50)], relogio=rel)
219          for _ in range(20):
220              self.assertNotEqual(s.processar(_resultado(intruso=True)).fase,
aut.IDENTIFICADO)
221          rel.t = 100
222          self.assertIn("mais de um rosto",
s.processar(_resultado(intruso=True)).motivo_falha)
223
224      def test_prova_de_vida_exige_piscada(self):
225          rel = _Relogio()
226          s = self._sessao([(7, 0.20, 0.50)], vida=True, relogio=rel)
227          fase = None
228          for _ in range(config.QUADROS_CONFIRMACAO):
229              fase = s.processar(_resultado(olhos=2)).fase
230          self.assertEqual(fase, aut.PROVA_DE_VIDA)
231          for olhos in (2, 2, 0, 0, 2, 2):
```

```
232             estado = s.processar(_resultado(olhos=olhos))
233         self.assertEqual(estado.fase, aut.IDENTIFICADO)
234
235     def test_rosto_desconhecido_que_pisca_apos_a_identificacao_nao_conclui(self):
236         # troca de foto: identidade confirmada, depois entra um rosto NÃO reconhecido
237         # que pisca. A prova de vida concluída não pode valer para a identidade antiga.
238         rel = _Relogio()
239         roteiro = [(7, 0.20, 0.50)] * config.QUADROS_CONFIRMACAO + [(7, 0.90, 1.0)]
240         s = self._sessao(roteiro, vida=True, relogio=rel)
241         for _ in range(config.QUADROS_CONFIRMACAO):
242             s.processar(_resultado(olhos=2))
243         for olhos in (2, 2, 2, 0, 0, 2, 2, 2, 2, 2):
244             estado = s.processar(_resultado(olhos=olhos))
245             self.assertNotEqual(estado.fase, aut.IDENTIFICADO)
246         rel.t = config.TEMPO_MAX_PISCADA + 1
247         self.assertEqual(s.processar(_resultado(olhos=2)).fase, aut.FALHA)
248
249     def test_foto_rosto_proprio_piscando_foto_de_novo_nao_conclui(self):
250         # foto da vítima identifica; um rosto DESCONHECIDO aparece de olhos abertos e
pisca;
251         # a foto volta. Os quadros de olhos abertos precisam ser do mesmo usuário, então
a
252         # presença do desconhecido reinicia o desafio e a sequência não conclui.
253         n = config.QUADROS_CONFIRMACAO
254         roteiro = ([(7, 0.20, 0.50)] * n            # foto: identifica
255                    + [(7, 0.90, 1.0)] * 7           # desconhecido: 3 abertos, 2
fechados, 2 abertos
256                    + [(7, 0.20, 0.50)] * 3)         # foto de volta
257         rel = _Relogio()
258         s = self._sessao(roteiro, vida=True, relogio=rel)
259         for _ in range(n):
260             s.processar(_resultado(olhos=2))
261         for olhos in (2, 2, 2, 0, 0, 2, 2, 2, 2, 2):
262             estado = s.processar(_resultado(olhos=olhos))
263             self.assertNotEqual(estado.fase, aut.IDENTIFICADO)
264
265     def test_foto_parada_falha_na_prova_de_vida(self):
266         rel = _Relogio()
267         s = self._sessao([(7, 0.20, 0.50)], vida=True, relogio=rel)
268         for _ in range(config.QUADROS_CONFIRMACAO + 5):
269             s.processar(_resultado(olhos=2))
270         rel.t = config.TEMPO_MAX_PISCADA + 1
271         estado = s.processar(_resultado(olhos=2))
272         self.assertEqual(estado.fase, aut.FALHA)
273         self.assertIn("prova de vida", estado.motivo_falha)
274
275     def test_troca_de_pessoa_durante_a_prova_de_vida(self):
276         roteiro = [(7, 0.20, 0.50)] * config.QUADROS_CONFIRMACAO + [(8, 0.20, 0.50)]
277         s = self._sessao(roteiro, vida=True)
278         for _ in range(config.QUADROS_CONFIRMACAO):
279             s.processar(_resultado(olhos=2))
280         estados = [s.processar(_resultado(olhos=2)) for _ in range(4)]
281         self.assertEqual(estados[-1].fase, aut.FALHA)
282         self.assertIn("troca de pessoa", estados[-1].motivo_falha)
283
284
285 class TestBloqueio(unittest.TestCase):
286     def test_bloqueia_apos_tres_falhas_e_libera_depois(self):
287         banco = _banco()
288         relogio = _Relogio()
289         relogio.t = 1000.0
290         ctl = ControleBloqueio(banco, relogio=relogio)
291         self.assertFalse(ctl.registrar_falha())
292         self.assertFalse(ctl.registrar_falha())
293         self.assertTrue(ctl.registrar_falha())
294         with self.assertRaises(SistemaBloqueado):
295             ctl.exigir_liberado()
296         relogio.t += config.SEGUNDOS_BLOQUEIO + 1
297         ctl.exigir_liberado()                       # liberado
298
299     def test_sucesso_zera_o_contador(self):
300         ctl = ControleBloqueio(_banco(), relogio=_Relogio())
301         ctl.registrar_falha()
302         ctl.registrar_falha()
303         ctl.registrar_sucesso()
304         self.assertFalse(ctl.registrar_falha())
305
```

```
306
307  if __name__ == "__main__":
308       unittest.main()
```

3 FERRAMENTAS DE EXPERIMENTOS E FIGURAS

3.1 ferramentas/adulterar_historico.py

```
"""Adultera DE PROPÓSITO uma linha do histórico, para demonstrar a verificação de integridade.

Só roda se a variável COFRE_DADOS apontar para uma pasta de TESTE (nunca os dados da apresentação). Uso (PowerShell, dentro da pasta `codigo`):

    $env:COFRE_DADOS = "C:\\temp\\dados_teste"
    python ferramentas\\adulterar_historico.py editar      # muda o resultado da linha 1
    python ferramentas\\adulterar_historico.py apagar      # apaga a ÚLTIMA linha

Depois, no menu, a opção 6 (verificar integridade) deve acusar violação.
"""

import os
import sqlite3
import sys
from pathlib import Path


def main():
    pasta = os.environ.get("COFRE_DADOS")
    if not pasta:
        print("Recusado: defina COFRE_DADOS apontando para uma pasta de teste.")
        return 1
    banco = Path(pasta) / "cofre.db"
    if not banco.is_file():
        print(f"Banco não encontrado: {banco}")
        return 1
    acao = sys.argv[1] if len(sys.argv) > 1 else "editar"
    con = sqlite3.connect(str(banco))
    if acao == "editar":
        con.execute("UPDATE acessos SET resultado='PERMITIDO' WHERE id=(SELECT MIN(id) FROM acessos)")
    elif acao == "apagar":
        con.execute("DELETE FROM acessos WHERE id=(SELECT MAX(id) FROM acessos)")
    else:
        print("Use: editar | apagar")
        return 1
    con.commit()
    print(f"Histórico em {banco} adulterado ({acao}). Rode a opção 6 do menu.")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

3.2 ferramentas/baixar_orl.py

```
"""Baixa a base ORL (Olivetti faces, AT&T) e grava uma subpasta por pessoa.

Uso (dentro da pasta `codigo`):
    python ferramentas/baixar_orl.py ..\\base_teste\\orl

A base tem 40 pessoas e 10 imagens de cada, em tons de cinza, 64x64 pixels
(versão distribuída pela biblioteca scikit-learn). Requer a internet na
primeira execução; depois fica em cache.
"""

import sys
from pathlib import Path

import cv2
import numpy as np
from sklearn.datasets import fetch_olivetti_faces
```

```
17
18
19  def main():
20      destino = Path(sys.argv[1] if len(sys.argv) > 1 else "../base_teste/orl")
21      dados = fetch_olivetti_faces(data_home=str(destino.parent / "_cache"), shuffle=False)
22      imagens = (dados.images * 255).astype(np.uint8)
23      contagem = {}
24      for imagem, pessoa in zip(imagens, dados.target):
25          contagem[pessoa] = contagem.get(pessoa, 0) + 1
26          pasta = destino / f"s{pessoa + 1:02d}"
27          pasta.mkdir(parents=True, exist_ok=True)
28          cv2.imencode(".png", imagem)[1].tofile(str(pasta / f"{contagem[pessoa]:02d}.png"))
29      print(f"{len(imagens)} imagens de {len(contagem)} pessoas gravadas em {destino}")
30
31
32  if __name__ == "__main__":
33      main()
```

3.3 ferramentas/experimentos.py

Experimentos reproduzíveis do capítulo de resultados.

```
  1  """Experimentos reproduzíveis com a base ORL (usados no capítulo de resultados).
  2
  3  Uso (dentro da pasta `codigo`):
  4      python ferramentas/experimentos.py ..\\base_teste\\orl  ..\\figuras
..\\documentos\\resultados.json
  5
  6  Protocolo de identificação em CONJUNTO ABERTO: metade das pessoas é cadastrada (as
  7  primeiras n imagens), a outra metade são intrusos que nunca aparecem no cadastro.
  8
  9  Duas formas de relatar cada configuração:
 10    * divisão de referência: pessoas de índice par cadastradas, ímpar intrusos, imagens
 11      em ordem (a mesma usada em todo o relatório);
 12    * 30 divisões aleatórias (pessoas e imagens sorteadas): média e desvio-padrão.
 13      Como todas vêm das mesmas 40 pessoas, o desvio-padrão mede a sensibilidade à
 14      divisão, NÃO a incerteza de generalizar para outras pessoas (que é maior).
 15
 16  Métricas (nomes da ISO/IEC 19795-1 para identificação em conjunto aberto):
 17    FAR  = FPIR: fração de impostores aceitos como alguém;
 18    FRR  = FNIR: fração de legítimos recusados OU identificados como outra pessoa;
 19    EER  = ponto em que FAR = FRR (pode passar de 50% porque a FRR inclui a troca de
identidade);
 20    FRR@FAR1% = FRR no limiar mais permissivo que mantém FAR <= 1%.
 21  """
 22
 23  import json
 24  import sys
 25  from pathlib import Path
 26
 27  import cv2
 28  import numpy as np
 29
 30  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 31
 32  from cofre import caracteristicas, config, preprocessamento, segmentacao  # noqa: E402
 33  from cofre.aquisicao import carregar_imagem  # noqa: E402
 34  from cofre.classificacao import distancia_qui_quadrado  # noqa: E402
 35
 36  SEMENTES = range(30)
 37
 38
 39  def carregar(pasta):
 40      pessoas = sorted(p for p in Path(pasta).iterdir() if p.is_dir())
 41      return [[carregar_imagem(f, cinza=True) for f in sorted(p.glob("*.png"))] for p in
pessoas]
 42
 43
 44  def preparar(img, mascara=True, equalizacao="clahe", tamanho=(100, 100)):
 45      """Mesma cadeia do pipeline (suavização, normalização, máscara) com opções."""
 46      suave = preprocessamento.reduzir_ruido(img)
 47      r = cv2.resize(suave, tamanho, interpolation=cv2.INTER_AREA)
 48      if equalizacao == "clahe":
 49          r = preprocessamento.equalizar_adaptativa(r)
 50      elif equalizacao == "global":
```

```
51          r = cv2.equalizeHist(r)
52      return segmentacao.mascara_eliptica(r) if mascara else r
53
54
55  # ---------------------------------------------------------------------------
56  # métricas
57  # ---------------------------------------------------------------------------
58  def calcular_eer(dist_gen, acertou, dist_imp):
59      """(EER, limiar no EER, FRR@FAR1%) varrendo o limiar."""
60      melhor, frr_far1 = None, 1.0
61      for t in np.unique(np.concatenate([dist_gen, dist_imp])):
62          far = float(np.mean(dist_imp <= t))
63          frr = float(np.mean((dist_gen > t) | (~acertou)))
64          if melhor is None or abs(far - frr) < melhor[0]:
65              melhor = (abs(far - frr), (far + frr) / 2.0, float(t))
66          if far <= 0.01:
67              frr_far1 = frr          # o último t com FAR <= 1% é o mais permissivo
68      return melhor[1], melhor[2], frr_far1
69
70
71  def prever(ref, rot, v, distancia, k, n_usuarios):
72      d = distancia(ref, v[None, :])
73      por = [np.sort(d[rot == u])[:k].mean() for u in range(n_usuarios)]
74      return int(np.argmin(por)), float(np.min(por))
75
76
77  def avaliar_divisao(F_ref, rot, F_gen, rot_gen, F_imp, distancia, k, n_usuarios):
78      gen, ok = [], []
79      for v, real in zip(F_gen, rot_gen):
80          ident, d = prever(F_ref, rot, v, distancia, k, n_usuarios)
81          gen.append(d)
82          ok.append(ident == real)
83      imp = [prever(F_ref, rot, v, distancia, k, n_usuarios)[1] for v in F_imp]
84      return np.array(gen), np.array(ok), np.array(imp)
85
86
87  def dividir(rng, n_pessoas, n_imgs, n_cad):
88      """Sorteia pessoas cadastradas/intrusos e imagens de cadastro/teste."""
89      pessoas = rng.permutation(n_pessoas)
90      cad, intr = pessoas[: n_pessoas // 2], pessoas[n_pessoas // 2:]
91      plano = [(int(p), *np.split(rng.permutation(n_imgs), [n_cad])) for p in cad]
92      return plano, [int(p) for p in intr]
93
94
95  def dividir_referencia(n_pessoas, n_imgs, n_cad):
96      cad = list(range(0, n_pessoas, 2))
97      intr = list(range(1, n_pessoas, 2))
98      ordem = np.arange(n_imgs)
99      return [(p, ordem[:n_cad], ordem[n_cad:]) for p in cad], intr
100
101
102 # ---------------------------------------------------------------------------
103 # execução de uma configuração
104 # ---------------------------------------------------------------------------
105 def rodar(F_de, distancia, n_cad=5, k=1, n_pessoas=40, n_imgs=10, aleatorio=True):
106     """F_de(plano) -> (F_ref, F_gen, F_imp) para uma divisão. Devolve métricas."""
107     resultados = []
108     divisoes = ([dividir(np.random.default_rng(s), n_pessoas, n_imgs, n_cad) for s in
SEMENTES]
109                 if aleatorio else [dividir_referencia(n_pessoas, n_imgs, n_cad)])
110     for plano, intr in divisoes:
111         F_ref, rot, F_gen, rot_gen, F_imp = F_de(plano, intr)
112         gen, ok, imp = avaliar_divisao(F_ref, rot, F_gen, rot_gen, F_imp, distancia, k,
len(plano))
113         eer, lim, frr1 = calcular_eer(gen, ok, imp)
114         resultados.append({"eer": eer, "limiar": lim, "frr_far1": frr1,
115                            "far_cfg": float(np.mean(imp <= config.LIMIAR_DISTANCIA)),
116                            "frr_cfg": float(np.mean((gen > config.LIMIAR_DISTANCIA) |
(~ok))),
117                            "gen": gen, "imp": imp, "ok": ok})
118     return resultados
119
120
121 def extrator_arrays(F):
122     """F: array (pessoas, imagens, D) já calculado -> função de divisão."""
123     def f(plano, intr):
124         ref, rot, gen, rot_gen = [], [], [], []
```

```
125          for u, (p, j_cad, j_teste) in enumerate(plano):
126              for j in j_cad:
127                  ref.append(F[p, j]); rot.append(u)
128              for j in j_teste:
129                  gen.append(F[p, j]); rot_gen.append(u)
130          imp = [F[p, j] for p in intr for j in range(F.shape[1])]
131          return np.array(ref), np.array(rot), np.array(gen), np.array(rot_gen),
np.array(imp)
132      return f
133
134
135  def extrator_pca(X, componentes):
136      """PCA (Turk e Pentland, 1991) treinada SÓ com as imagens de cadastro da divisão."""
137      def f(plano, intr):
138          treino = np.array([X[p, j] for p, j_cad, _ in plano for j in j_cad])
139          media = treino.mean(axis=0)
140          _, _, vt = np.linalg.svd(treino - media, full_matrices=False)
141          base = vt[:componentes]
142          proj = lambda x: base @ (x - media)
143          ref, rot, gen, rot_gen = [], [], [], []
144          for u, (p, j_cad, j_teste) in enumerate(plano):
145              for j in j_cad:
146                  ref.append(proj(X[p, j])); rot.append(u)
147              for j in j_teste:
148                  gen.append(proj(X[p, j])); rot_gen.append(u)
149          imp = [proj(X[p, j]) for p in intr for j in range(X.shape[1])]
150          return np.array(ref), np.array(rot), np.array(gen), np.array(rot_gen),
np.array(imp)
151      return f
152
153
154  def euclidiana(a, b):
155      return np.sqrt(np.sum((a - b) ** 2, axis=-1))
156
157
158  def coseno(a, b):
159      num = np.sum(a * b, axis=-1)
160      den = np.linalg.norm(a, axis=-1) * np.linalg.norm(b, axis=-1) + 1e-12
161      return 1.0 - num / den
162
163
164  def intersecao(a, b):
165      return 1.0 - np.sum(np.minimum(a, b), axis=-1) / (a.shape[-1] // 59)
166
167
168  def resumo(ref, aleat):
169      eers = np.array([r["eer"] for r in aleat])
170      frr1 = np.array([r["frr_far1"] for r in aleat])
171      return {"eer": ref[0]["eer"], "limiar": ref[0]["limiar"], "frr_far1":
ref[0]["frr_far1"],
172              "eer_media": float(eers.mean()), "eer_dp": float(eers.std(ddof=1)),
173              "frr_far1_media": float(frr1.mean()), "frr_far1_dp": float(frr1.std(ddof=1)),
174              "acuracia": 1.0 - ref[0]["eer"]}       # mantido só para compatibilidade com
as figuras
175
176
177  def pareados(imagens, resultados):
178      """Diferenças PAREADAS (mesmas 30 divisões) entre variantes, com IC de 95% da média.
179
180      diferença = EER da variante - EER do LBP padrão (ou da referência indicada);
181      positivo = a variante é pior. O IC usa t de Student (29 gl); vale para a
182      variação ENTRE DIVISÕES das mesmas 40 pessoas, não para outras pessoas.
183      """
184      n_p, n_i = len(imagens), len(imagens[0])
185      chi = distancia_qui_quadrado
186
187      def eers(F_de, d, k=1):
188          return np.array([r["eer"] for r in rodar(F_de, d, 5, k, n_p, n_i,
aleatorio=True)])
189
190      F_pad = np.array([[caracteristicas.extrair(preparar(i)) for i in ims] for ims in
imagens])
191      F_nu = np.array([[caracteristicas.extrair(preparar(i, mascara=False,
equalizacao="nenhuma")) for i in ims]
192                       for ims in imagens])
193      X_cl = np.array([[preparar(i, mascara=False).ravel().astype(np.float64) for i in ims]
for ims in imagens])
```

```
194       X_nc = np.array([[preparar(i, mascara=False,
equalizacao="nenhuma").ravel().astype(np.float64) for i in ims]
195                        for ims in imagens])
196       base = eers(extrator_arrays(F_pad), chi)                        # LBP padrão (CLAHE +
máscara)
197       sem = eers(extrator_arrays(F_nu), chi)                          # LBP sem CLAHE e sem
máscara
198       pca_cl = eers(extrator_pca(X_cl, 80), coseno)                   # melhor PCA, com
CLAHE
199       pca_nc = eers(extrator_pca(X_nc, 80), coseno)                   # o mesmo PCA, sem
CLAHE
200       euc = eers(extrator_arrays(F_pad), euclidiana)
201       k2 = eers(extrator_arrays(F_pad), chi, k=2)
202       comparacoes = {
203           "LBP padrão (CLAHE e máscara) contra LBP sem CLAHE e sem máscara": base - sem,
204           "PCA 80 + cosseno (com CLAHE) contra LBP padrão (com CLAHE e máscara)": pca_cl -
base,
205           "PCA 80 + cosseno (sem CLAHE) contra LBP sem CLAHE e sem máscara": pca_nc - sem,
206           "Distância euclidiana contra qui-quadrado (LBP padrão)": euc - base,
207           "Decisão por k = 2 contra k = 1 (LBP padrão)": k2 - base,
208       }
209       t = 2.045   # t de Student, 29 graus de liberdade, bilateral 95%
210       saida = {}
211       for nome, d in comparacoes.items():
212           m, dp = float(d.mean()), float(d.std(ddof=1))
213           h = t * dp / np.sqrt(len(d))
214           saida[nome] = {"media": m, "ic_inf": m - h, "ic_sup": m + h, "fracao_positiva":
float(np.mean(d > 0))}
215           print(f"{nome}: diferença média {100 * m:+.1f} pp  IC95% [{100 * (m - h):+.1f};
{100 * (m + h):+.1f}]  "
216                 f"(variante pior em {100 * np.mean(d > 0):.0f}% das divisões)")
217       resultados["pareados"] = saida
218       # linhas extras para a tabela de métodos
219       resumo_pca_nc = {"eer_media": float(pca_nc.mean()), "eer_dp":
float(pca_nc.std(ddof=1))}
220       resultados["metodo"]["Eigenfaces sem CLAHE (PCA 80 + cosseno)"] = {
221           **resumo_pca_nc, "eer": float(pca_nc[0]), "frr_far1_media": float("nan"),
"frr_far1_dp": float("nan")}
222       return resultados
223
224
225   def main():
226       if len(sys.argv) > 4 and sys.argv[4] == "--so-pareados":
227           # roda apenas o bloco de diferenças pareadas e atualiza o JSON existente
228           pasta, saida_json = sys.argv[1], Path(sys.argv[3])
229           res = json.loads(saida_json.read_text(encoding="utf-8"))
230           pareados(carregar(pasta), res)
231           saida_json.write_text(json.dumps(res, indent=2, ensure_ascii=False),
encoding="utf-8")
232           print("Pareados gravados em", saida_json)
233           return
234       pasta, saida_fig, saida_json = sys.argv[1], Path(sys.argv[2]), Path(sys.argv[3])
235       saida_fig.mkdir(parents=True, exist_ok=True)
236       imagens = carregar(pasta)
237       n_p, n_i = len(imagens), len(imagens[0])
238       chi = distancia_qui_quadrado
239       resultados = {"pessoas": n_p, "imagens_por_pessoa": n_i, "divisoes_aleatorias":
len(list(SEMENTES))}
240
241       def LBP(grade=(8, 8), **opc):
242           return np.array([[caracteristicas.extrair(preparar(i, **opc), grade) for i in
ims] for ims in imagens])
243
244       def linha(nome, F_de, distancia, n_cad=5, k=1):
245           ref = rodar(F_de, distancia, n_cad, k, n_p, n_i, aleatorio=False)
246           alea = rodar(F_de, distancia, n_cad, k, n_p, n_i, aleatorio=True)
247           r = resumo(ref, alea)
248           print(f"{nome:44s} ref {100 * r['eer']:5.1f}%  30 divisões {100 *
r['eer_media']:5.1f} ± {100 * r['eer_dp']:4.1f}"
249                 f"  FRR@FAR1% {100 * r['frr_far1_media']:5.1f} ± {100 *
r['frr_far1_dp']:4.1f}")
250           return r, ref, alea
251
252       F_padrao = LBP()
253       padrao = extrator_arrays(F_padrao)
254
255       print("--- grade de blocos ---")
```

```
256      resultados["grade"] = {}
257      for g in (1, 3, 4, 5, 6, 7, 8, 10):
258          F = F_padrao if g == 8 else LBP((g, g))
259          resultados["grade"][f"{g}x{g}"] = linha(f"grade {g}x{g}", extrator_arrays(F),
chi)[0]
260
261      print("--- pré-processamento ---")
262      resultados["preprocessamento"] = {}
263      for nome, opc in {"CLAHE + máscara (padrão)": {}, "sem máscara": {"mascara": False},
264                        "equalização global": {"equalizacao": "global"},
265                        "sem equalização": {"equalizacao": "nenhuma"},
266                        "sem equalização e sem máscara": {"equalizacao": "nenhuma",
"mascara": False}}.items():
267          F = F_padrao if not opc else LBP(**opc)
268          resultados["preprocessamento"][nome] = linha(nome, extrator_arrays(F), chi)[0]
269
270      print("--- medida de distância e k ---")
271      resultados["distancia"] = {}
272      for nome, d in {"qui-quadrado": chi, "euclidiana": euclidiana, "interseção de
histogramas": intersecao}.items():
273          resultados["distancia"][nome] = linha(nome, padrao, d)[0]
274      resultados["k"] = {}
275      for k in (1, 2, 3):
276          resultados["k"][str(k)] = linha(f"k = {k}", padrao, chi, k=k)[0]
277
278      print("--- imagens de cadastro por pessoa ---")
279      resultados["cadastro"] = {}
280      for n in (2, 3, 5, 7):
281          resultados["cadastro"][str(n)] = linha(f"{n} imagens de cadastro", padrao, chi,
n_cad=n)[0]
282
283      print("--- método: LBP x Eigenfaces (várias configurações de PCA) ---")
284      resultados["metodo"] = {"LBP 8x8 + qui-quadrado": resultados["distancia"]["qui-
quadrado"]}
285      X = np.array([[preparar(i, mascara=False).ravel().astype(np.float64) for i in ims]
for ims in imagens])
286      pca = {}
287      for comp in (10, 20, 40, 60, 80):
288          for nome_d, d in (("euclidiana", euclidiana), ("cosseno", coseno)):
289              pca[(comp, nome_d)] = linha(f"Eigenfaces PCA {comp} + {nome_d}",
extrator_pca(X, comp), d)[0]
290      melhor = min(pca, key=lambda c: pca[c]["eer_media"])
291      resultados["metodo"]["Eigenfaces (PCA 40) + euclidiana"] = pca[(40, "euclidiana")]
292      resultados["metodo"][f"Eigenfaces, melhor configuração testada (PCA {melhor[0]} +
{melhor[1]})"] = pca[melhor]
293      resultados["pca_todas"] = {f"{c}+{d}": v for (c, d), v in pca.items()}
294
295      # distâncias da divisão de referência (para as figuras) e limiar de produção em 30
divisões
296      ref = rodar(padrao, chi, 5, 1, n_p, n_i, aleatorio=False)[0]
297      alea = rodar(padrao, chi, 5, 1, n_p, n_i, aleatorio=True)
298      np.savez(saida_fig / "distancias_orl.npz", gen=ref["gen"], imp=ref["imp"])
299      gen, imp = ref["gen"], ref["imp"]
300      far_cfg = np.array([r["far_cfg"] for r in alea])
301      frr_cfg = np.array([r["frr_cfg"] for r in alea])
302      resultados["padrao"] = {
303          "eer": ref["eer"], "limiar_eer": ref["limiar"], "acuracia": 1.0 - ref["eer"],
304          "n_genuinos": int(len(gen)), "n_impostores": int(len(imp)),
305          "far_no_limiar_config": float(np.mean(imp <= config.LIMIAR_DISTANCIA)),
306          "frr_no_limiar_config": float(np.mean((gen > config.LIMIAR_DISTANCIA) |
(~ref["ok"]))),
307          "gen_media": float(gen.mean()), "imp_media": float(imp.mean()),
308          "gen_mediana": float(np.median(gen)), "imp_mediana": float(np.median(imp)),
309          "imp_min": float(imp.min()),
310          "limiar_cfg_far_media_30": float(far_cfg.mean()), "limiar_cfg_far_dp_30":
float(far_cfg.std(ddof=1)),
311          "limiar_cfg_far_max_30": float(far_cfg.max()),
312          "limiar_cfg_frr_media_30": float(frr_cfg.mean()), "limiar_cfg_frr_dp_30":
float(frr_cfg.std(ddof=1)),
313      }
314      print(f"\nLimiar {config.LIMIAR_DISTANCIA} em 30 divisões: FAR {100 *
far_cfg.mean():.2f}% ± {100 * far_cfg.std(ddof=1):.2f} "
315            f"(máx {100 * far_cfg.max():.1f}%), FRR {100 * frr_cfg.mean():.1f}% ± {100 *
frr_cfg.std(ddof=1):.1f}")
316      print("--- diferenças pareadas ---")
317      pareados(imagens, resultados)
318      saida_json.parent.mkdir(parents=True, exist_ok=True)
```

```
319      saida_json.write_text(json.dumps(resultados, indent=2, ensure_ascii=False),
encoding="utf-8")
320      print("Resultados gravados em", saida_json)
321
322
323  if __name__ == "__main__":
324      main()
```

3.4 ferramentas/gerar_figuras.py

Geração das figuras do relatório.

```
  1  """Gera as figuras do relatório a partir do código e da base ORL.
  2
  3  Uso (dentro da pasta `codigo`):
  4      python ferramentas/gerar_figuras.py ..\\base_teste\\orl ..\\figuras
..\\documentos\\resultados.json
  5
  6  As imagens de rosto vêm da base ORL (AT&T Laboratories Cambridge); as figuras de
  7  processamento são produzidas pelas próprias funções do sistema, não desenhadas.
  8  """
  9
 10  import json
 11  import sys
 12  from pathlib import Path
 13
 14  import cv2
 15  import matplotlib
 16
 17  matplotlib.use("Agg")
 18  import matplotlib.pyplot as plt  # noqa: E402
 19  import numpy as np  # noqa: E402
 20  from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Rectangle  # noqa: E402
 21
 22  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 23  from cofre import caracteristicas, config, interface, preprocessamento, segmentacao  #
noqa: E402
 24  from cofre.aquisicao import carregar_imagem  # noqa: E402
 25  from cofre.classificacao import Reconhecedor  # noqa: E402
 26  from cofre.pipeline import Pipeline  # noqa: E402
 27
 28  plt.rcParams.update({"font.family": ["Arial", "DejaVu Sans"], "font.size": 9,
 29                       "axes.spines.top": False, "axes.spines.right": False,
 30                       "savefig.dpi": 200, "savefig.bbox": "tight", "figure.dpi": 100})
 31  CINZA, ESCURO, DESTAQUE, CLARO = "#6b6b6b", "#222222", "#1f6f8b", "#e6eef2"
 32
 33
 34  def salvar(fig, pasta, nome):
 35      fig.savefig(pasta / nome, facecolor="white")
 36      plt.close(fig)
 37      print("ok", nome)
 38
 39
 40  def caixa(ax, x, y, w, h, texto, cor=CLARO, borda=ESCURO, tam=8.5, negrito=False):
 41      ax.add_patch(FancyBboxPatch((x, y), w, h,
boxstyle="round,pad=0.02,rounding_size=0.08",
 42                                  fc=cor, ec=borda, lw=1.1))
 43      ax.text(x + w / 2, y + h / 2, texto, ha="center", va="center", fontsize=tam,
 44              fontweight="bold" if negrito else "normal", color=ESCURO)
 45
 46
 47  def seta(ax, p, q, curva=0.0):
 48      ax.add_patch(FancyArrowPatch(p, q, arrowstyle="-|>", mutation_scale=11, lw=1.1,
 49                                   color=ESCURO, connectionstyle=f"arc3,rad={curva}"))
 50
 51
 52  def fig_cinco_fases(pasta):
 53      fig, ax = plt.subplots(figsize=(6.3, 2.5))
 54      ax.set_xlim(0, 10); ax.set_ylim(0, 4.0); ax.axis("off")
 55      nomes = ["1. Aquisição", "2. Pré-\nprocessamento", "3. Segmen-\ntação",
 56               "4. Extração de\ncaracterísticas", "5. Classificação"]
 57      detalhe = ["câmera ou\narquivo de\nimagem", "cinza, ruído,\nequalização",
"detecção,\nrecorte, alinha-\nmento, máscara",
 58                 "LBP uniforme\ne histograma\npor blocos", "distância\nqui-
quadrado,\nlimiar, política"]
```

```
 59      xs = [0.05, 2.05, 4.05, 6.05, 8.05]
 60      for x, n, d in zip(xs, nomes, detalhe):
 61          caixa(ax, x, 2.35, 1.65, 1.3, n, tam=7.4, negrito=True)
 62          ax.text(x + 0.825, 1.55, d, ha="center", va="center", fontsize=6.4, color=CINZA)
 63      for a, b in zip(xs, xs[1:]):
 64          seta(ax, (a + 1.7, 3.0), (b - 0.04, 3.0))
 65      ax.text(5, 0.35, "Fluxo do sistema: da imagem bruta até a decisão de acesso",
ha="center",
 66              fontsize=7.5, style="italic", color=CINZA)
 67      salvar(fig, pasta, "fig01_cinco_fases.png")
 68
 69
 70  def fig_arquitetura(pasta):
 71      """Arquitetura em camadas: interface -> serviço -> três grupos de módulos."""
 72      fig, ax = plt.subplots(figsize=(6.3, 4.4))
 73      ax.set_xlim(0, 10); ax.set_ylim(0, 7.2); ax.axis("off")
 74      caixa(ax, 1.6, 6.3, 6.8, 0.75, "Interface: interface.py e __main__.py\n(menus,
janelas em tempo real, linha de comando)", tam=7.8)
 75      caixa(ax, 2.2, 4.95, 5.6, 0.85, "Serviço: servico.py (classe Cofre)\ncadastro,
treino, decisão e registro", cor="#d5e6ec", tam=7.8, negrito=True)
 76      seta(ax, (5, 6.3), (5, 5.83))
 77      colunas = [
 78          (0.1, "Fases 1 a 4 (imagem)", ["aquisicao.py", "preprocessamento.py",
"segmentacao.py", "caracteristicas.py", "pipeline.py"]),
 79          (3.5, "Fase 5 e autenticação", ["classificacao.py", "autenticacao.py",
"prova_vida.py", "avaliacao.py"]),
 80          (6.9, "Segurança e auditoria", ["politica.py", "banco.py", "relatorios.py"]),
 81      ]
 82      for x, titulo, modulos in colunas:
 83          ax.text(x + 1.5, 4.45, titulo, ha="center", fontsize=8, fontweight="bold")
 84          seta(ax, (min(max(x + 1.5, 3.0), 7.0), 4.93), (x + 1.5, 4.68))
 85          for i, nome in enumerate(modulos):
 86              caixa(ax, x, 3.5 - 0.78 * i, 3.0, 0.58, nome, tam=8)
 87      salvar(fig, pasta, "fig02_arquitetura.png")
 88
 89
 90  def fig_lbp_exemplo(pasta):
 91      bloco = np.array([[60, 10, 60], [10, 50, 60], [10, 60, 10]])
 92      centro = 50
 93      bits = (bloco >= centro).astype(int)
 94      ordem = [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 1), (2, 0), (1, 0)]
 95      cod = "".join(str(bits[p]) for p in ordem)
 96      fig, axs = plt.subplots(1, 3, figsize=(6.3, 2.4))
 97      for ax, dados, titulo in zip(axs[:2], (bloco, bits), ("Vizinhança 3x3 (tons de
cinza)", "Limiarização: vizinho >= centro")):
 98          ax.imshow(np.zeros((3, 3)), cmap="Greys", vmin=0, vmax=1)
 99          for i in range(3):
100              for j in range(3):
101                  ax.text(j, i, str(dados[i, j]), ha="center", va="center", fontsize=12,
102                          fontweight="bold" if (i, j) == (1, 1) else "normal")
103          ax.add_patch(Rectangle((0.5, 0.5), 1, 1, fill=False, ec=DESTAQUE, lw=2))
104          ax.set_xticks(range(3)); ax.set_yticks(range(3)); ax.tick_params(length=0,
labelbottom=False, labelleft=False)
105          ax.set_title(titulo, fontsize=8)
106          for s in ax.spines.values():
107              s.set_visible(True)
108      axs[2].axis("off")
109      axs[2].text(0, 0.82, "Leitura no sentido horário\na partir do canto
superior\nesquerdo:", fontsize=8.5, va="top")
110      axs[2].text(0, 0.42, " ".join(cod), fontsize=11, family="monospace",
fontweight="bold")
111      axs[2].text(0, 0.2, f"= {cod} (binário)\n= {int(cod, 2)} (decimal)", fontsize=9,
va="top")
112      axs[2].text(0, -0.05, "O pixel central recebe\no código " + str(int(cod, 2)),
fontsize=8.5, va="top", color=CINZA)
113      salvar(fig, pasta, "fig03_lbp_exemplo.png")
114
115
116  def fig_etapas(pasta, imagens):
117      pipe = Pipeline()
118      img = imagens[0][2]
119      r = pipe.processar_recorte(img)
120      suave = preprocessamento.reduzir_ruido(img)
121      clahe = preprocessamento.normalizar_face(suave)
122      classes = caracteristicas.TABELA_UNIFORMES[r.lbp]
123      fig, axs = plt.subplots(1, 6, figsize=(6.4, 1.9))
124      pares = [(img, "Original\n(64x64)"), (suave, "Pré-processada\n(gaussiano 3x3)"),
```

```
125               (clahe, "Normalizada\n(100x100 + CLAHE)"), (r.normalizada,
"Máscara\nelíptica"),
126               (r.lbp, "Imagem LBP\n(códigos 0-255)"), (classes, "Classes uniformes\n(0-
58)")]
127      for ax, (im, tit) in zip(axs, pares):
128          ax.imshow(im, cmap="gray"); ax.set_title(tit, fontsize=6.2); ax.axis("off")
129      salvar(fig, pasta, "fig04_etapas.png")
130
131      fig, axs = plt.subplots(1, 2, figsize=(6.3, 2.7), gridspec_kw={"width_ratios": [1,
1.7]})
132      axs[0].imshow(r.normalizada, cmap="gray"); axs[0].axis("off")
133      h, w = r.normalizada.shape
134      for k in range(1, 8):
135          axs[0].axhline(k * h / 8, color="#ffcc00", lw=0.5); axs[0].axvline(k * w / 8,
color="#ffcc00", lw=0.5)
136      axs[0].set_title("Grade de 8x8 blocos", fontsize=8)
137      vetor = r.vetor
138      bloco = vetor[27 * 59:28 * 59]
139      axs[1].bar(range(59), bloco, color=DESTAQUE, width=0.85)
140      axs[1].set_title("Histograma LBP de um bloco (59 classes)", fontsize=8)
141      axs[1].set_xlabel("classe LBP (0-57 uniformes, 58 = outros)");
axs[1].set_ylabel("frequência relativa")
142      salvar(fig, pasta, "fig05_grade_histograma.png")
143
144
145  def fig_variacao_pose(pasta, imagens):
146      fig, axs = plt.subplots(2, 5, figsize=(6.0, 2.7))
147      for ax, im in zip(axs.ravel(), imagens[5]):
148          ax.imshow(im, cmap="gray"); ax.axis("off")
149      fig.suptitle("Dez imagens de uma mesma pessoa na base ORL (variação de pose e
expressão)", fontsize=8)
150      salvar(fig, pasta, "fig06_variacao_orl.png")
151
152
153  def fig_distribuicoes(pasta):
154      dados = np.load(pasta / "distancias_orl.npz")
155      gen, imp = dados["gen"], dados["imp"]
156      fig, axs = plt.subplots(1, 2, figsize=(6.4, 2.8))
157      bins = np.linspace(0.15, 0.8, 30)
158      axs[0].hist(gen, bins=bins, alpha=0.8, color=DESTAQUE, label="legítimos")
159      axs[0].hist(imp, bins=bins, alpha=0.6, color="#b5651d", label="intrusos")
160      axs[0].axvline(config.LIMIAR_DISTANCIA, color="black", ls="--", lw=1)
161      axs[0].text(config.LIMIAR_DISTANCIA + 0.01, axs[0].get_ylim()[1] * 0.9,
"limiar\n0,38", fontsize=7)
162      axs[0].set_xlabel("distância qui-quadrado"); axs[0].set_ylabel("n° de testes");
axs[0].legend(fontsize=7)
163      axs[0].set_title("Distribuição das distâncias", fontsize=8)
164      ts = np.linspace(0, 0.8, 300)
165      far = [np.mean(imp <= t) for t in ts]
166      frr = [np.mean(gen > t) for t in ts]
167      axs[1].plot(ts, np.array(far) * 100, color="#b5651d", label="FAR (falsa aceitação)")
168      axs[1].plot(ts, np.array(frr) * 100, color=DESTAQUE, label="FRR (falsa rejeição)")
169      axs[1].axvline(config.LIMIAR_DISTANCIA, color="black", ls="--", lw=1)
170      axs[1].set_ylim(0, 125)
171      axs[1].set_xlabel("limiar de distância"); axs[1].set_ylabel("taxa (%)")
172      axs[1].legend(fontsize=7, loc="upper center", ncol=2, frameon=False)
173      axs[1].set_title("FAR e FRR em função do limiar", fontsize=8)
174      fig.tight_layout()
175      salvar(fig, pasta, "fig07_far_frr.png")
176
177
178  def fig_barras(pasta, res):
179      fig, axs = plt.subplots(1, 2, figsize=(6.4, 2.7))
180      g = res["grade"]
181      chaves = list(g)
182      axs[0].bar(chaves, [100 * g[k]["eer_media"] for k in chaves], yerr=[100 *
g[k]["eer_dp"] for k in chaves],
183                 color=DESTAQUE, capsize=2, error_kw={"lw": 0.8})
184      axs[0].set_ylabel("EER médio (%)"); axs[0].set_xlabel("grade de blocos")
185      axs[0].tick_params(axis="x", labelsize=6.5)
186      axs[0].set_title("Efeito da grade de blocos", fontsize=8)
187      pca_melhor = [v for n, v in res["metodo"].items() if n.startswith("Eigenfaces,
melhor")][0]
188      metodos = {"LBP\nqui-quad.": res["metodo"]["LBP 8x8 + qui-quadrado"],
189                 "LBP\neuclid.": res["distancia"]["euclidiana"],
190                 "LBP\ninters.": res["distancia"]["interseção de histogramas"],
191                 "PCA 40\neuclid.": res["metodo"]["Eigenfaces (PCA 40) + euclidiana"],
```

```
192                 "PCA\nmelhor": pca_melhor}
193      axs[1].bar(list(metodos), [100 * v["eer_media"] for v in metodos.values()],
194                 yerr=[100 * v["eer_dp"] for v in metodos.values()], capsize=2,
error_kw={"lw": 0.8},
195                 color=[DESTAQUE, "#6aa0b4", "#6aa0b4", "#b5651d", "#b5651d"])
196      axs[1].set_ylabel("EER médio (%)"); axs[1].tick_params(axis="x", labelsize=6.5)
197      axs[1].set_title("Métodos e medidas de distância", fontsize=8)
198      fig.tight_layout()
199      salvar(fig, pasta, "fig08_ablacao.png")
200
201
202  def fig_cadastro(pasta, res):
203      c = res["cadastro"]
204      fig, ax = plt.subplots(figsize=(4.2, 2.5))
205      ns = list(c)
206      ax.errorbar(ns, [100 * c[n]["eer_media"] for n in ns], yerr=[100 * c[n]["eer_dp"] for
n in ns],
207                  fmt="o-", color=DESTAQUE, capsize=3, label="EER (média ± desvio-padrão)")
208      ax.errorbar(ns, [100 * c[n]["frr_far1_media"] for n in ns], yerr=[100 *
c[n]["frr_far1_dp"] for n in ns],
209                  fmt="s--", color="#b5651d", capsize=3, label="FRR com FAR de cerca de
1%")
210      ax.set_xlabel("imagens de cadastro por pessoa"); ax.set_ylabel("%");
ax.legend(fontsize=7)
211      ax.set_title("Mais amostras no cadastro, menos erro (30 divisões)", fontsize=8)
212      salvar(fig, pasta, "fig09_cadastro.png")
213
214
215  def fig_estados(pasta):
216      fig, ax = plt.subplots(figsize=(6.3, 3.7))
217      ax.set_xlim(0, 10); ax.set_ylim(0, 6.2); ax.axis("off")
218      caixa(ax, 0.1, 4.3, 2.3, 1.0, "PROCURANDO\nsem rosto, intruso\nou qualidade ruim",
tam=7)
219      caixa(ax, 3.7, 4.3, 2.3, 1.0, "VALIDANDO\nvotação: 8 votos\nem 12 quadros", tam=7)
220      caixa(ax, 7.3, 4.3, 2.5, 1.0, "PROVA DE VIDA\npiscada em\naté 8 segundos", tam=7)
221      caixa(ax, 7.3, 1.6, 2.5, 1.0, "IDENTIFICADO\naplica a política", cor="#d5e6ec",
tam=7.2, negrito=True)
222      caixa(ax, 3.2, 0.2, 3.3, 1.0, "FALHA\nacesso negado e registro", cor="#f0d9c8",
tam=7.2, negrito=True)
223      seta(ax, (2.45, 4.8), (3.65, 4.8)); seta(ax, (6.05, 4.8), (7.25, 4.8))
224      seta(ax, (8.55, 4.28), (8.55, 2.65))
225      ax.text(8.7, 3.45, "piscada\nconcluída", fontsize=6.5, color=CINZA, va="center")
226      ax.text(0.1, 5.85, "início da tentativa", fontsize=6.8, color=CINZA)
227      seta(ax, (1.0, 5.8), (1.0, 5.35))
228      seta(ax, (3.7, 4.55), (2.45, 4.55), 0.0)
229      ax.text(3.05, 3.85, "rosto some ou há vários\nrostos: recomeça", fontsize=6.5,
color=CINZA, ha="center")
230      seta(ax, (4.85, 4.28), (4.85, 1.25))
231      ax.text(4.98, 2.75, "15 s sem\nconfirmar", fontsize=6.5, color=CINZA, va="center")
232      seta(ax, (7.7, 4.28), (6.55, 1.0), -0.12)
233      ax.text(7.35, 2.0, "sem piscada ou\ntroca de pessoa", fontsize=6.5, color=CINZA,
ha="right", va="center")
234      salvar(fig, pasta, "fig10_estados_autenticacao.png")
235
236
237  def fig_banco(pasta):
238      fig, ax = plt.subplots(figsize=(6.3, 3.9))
239      ax.set_xlim(0, 10); ax.set_ylim(-1.1, 5.6); ax.axis("off")
240      def tabela(x, y, w, titulo, campos):
241          h = 0.42 * (len(campos) + 1)
242          ax.add_patch(Rectangle((x, y), w, h, fc="white", ec=ESCURO, lw=1.1))
243          ax.add_patch(Rectangle((x, y + h - 0.42), w, 0.42, fc=CLARO, ec=ESCURO, lw=1.1))
244          ax.text(x + w / 2, y + h - 0.21, titulo, ha="center", va="center", fontsize=8,
fontweight="bold")
245          for i, c in enumerate(campos):
246              ax.text(x + 0.1, y + h - 0.42 - 0.21 - 0.42 * i, c, va="center", fontsize=7)
247      tabela(0.1, 1.5, 2.9, "usuarios", ["id (PK)", "nome", "cargo", "nivel (1 a 3)",
"divisao", "ativo", "criado_em"])
248      tabela(3.55, 3.0, 2.9, "compartimentos", ["id (PK)", "nome (único)", "nivel_minimo",
"divisao"])
249      tabela(3.55, 1.2, 2.9, "parametros", ["chave (PK)", "valor"])
250      tabela(7.0, 0.5, 2.9, "acessos", ["id (PK)", "data_hora", "usuario_id", "nome",
"compartimento", "resultado", "motivo", "distancia", "hash_anterior", "hash"])
251      ax.text(0.1, -0.25,
252              "A tabela acessos guarda o nome do usuário e do compartimento (texto), e não
só chaves, para que o\n"
```

```
253              "histórico sobreviva à remoção de cadastros. A tabela parametros guarda o
hash do modelo, o contador\n"
254              "de falhas e o horário de liberação do bloqueio.",
255              fontsize=6.6, color=CINZA, ha="left", va="top")
256      salvar(fig, pasta, "fig11_banco.png")
257
258
259  def fig_haar(pasta):
260      fig, axs = plt.subplots(1, 4, figsize=(6.0, 1.7))
261      def desenhar(ax, regioes, titulo):
262          ax.set_xlim(0, 4); ax.set_ylim(0, 3); ax.set_aspect("equal"); ax.axis("off")
263          ax.add_patch(Rectangle((0.3, 0.3), 3.4, 2.4, fc="white", ec=ESCURO))
264          for (x, y, w, h, cor) in regioes:
265              ax.add_patch(Rectangle((x, y), w, h, fc=cor, ec=ESCURO))
266          ax.set_title(titulo, fontsize=7)
267      desenhar(axs[0], [(0.3, 0.3, 1.7, 2.4, "white"), (2.0, 0.3, 1.7, 2.4, "#444444")],
"borda vertical")
268      desenhar(axs[1], [(0.3, 1.5, 3.4, 1.2, "white"), (0.3, 0.3, 3.4, 1.2, "#444444")],
"borda horizontal")
269      desenhar(axs[2], [(0.3, 0.3, 1.13, 2.4, "white"), (1.43, 0.3, 1.14, 2.4, "#444444"),
(2.57, 0.3, 1.13, 2.4, "white")], "linha")
270      desenhar(axs[3], [(0.3, 0.3, 1.7, 1.2, "white"), (2.0, 1.5, 1.7, 1.2, "white"), (2.0,
0.3, 1.7, 1.2, "#444444"), (0.3, 1.5, 1.7, 1.2, "#444444")], "diagonal")
271      salvar(fig, pasta, "fig12_haar.png")
272
273
274  def fig_telas(pasta, imagens):
275      """Reproduz as telas do sistema com rostos da ORL (mesmas funções de desenho)."""
276      pipe = Pipeline()
277      treino = [pipe.processar_recorte(im).vetor for i in (0, 2, 4) for im in
imagens[i][:7]]
278      rotulos = [i for i in (1, 2, 3) for _ in range(7)]
279      modelo = Reconhecedor(limiar=0.53).treinar(treino, rotulos)
280      probe = imagens[2][8]
281      r = pipe.processar_recorte(probe)
282      previsao = modelo.prever(r.vetor)
283      painel = interface.montar_painel(r, previsao)
284      cv2.imencode(".png", painel)[1].tofile(str(pasta / "fig13_painel_fases.png"))
285      print("ok fig13_painel_fases.png")
286
287      def tela(permitido, linhas):
288          base = cv2.resize(cv2.cvtColor(probe, cv2.COLOR_GRAY2BGR), (480, 480),
interpolation=cv2.INTER_CUBIC)
289          canvas = np.zeros((480, 640, 3), np.uint8)
290          canvas[:, 80:560] = base
291          cor = interface.VERDE if permitido else interface.VERMELHO
292          cv2.rectangle(canvas, (0, 0), (639, 479), cor, 14)
293          interface._texto(canvas, "ACESSO PERMITIDO" if permitido else "ACESSO NEGADO",
(30, 70), cor, 1.2, 3)
294          for i, linha in enumerate(linhas):
295              interface._texto(canvas, linha, (30, 120 + 34 * i), interface.BRANCO, 0.65)
296          return canvas
297      a = tela(True, ["Dora Diretora", "Nivel 2 - Diretor de divisao", "Laboratorio de
Toxinas Agricolas", "diretor da divisao responsavel"])
298      b = tela(False, ["Dora Diretora", "Nivel 2 - Diretor de divisao", "Camara Central de
Toxinas de Alto Risco", "compartimento exclusivo do ministro"])
299      cv2.imencode(".png", np.hstack([a, b]))[1].tofile(str(pasta /
"fig14_telas_resultado.png"))
300      print("ok fig14_telas_resultado.png")
301
302
303  def main():
304      pasta_base, pasta_fig, arq_json = Path(sys.argv[1]), Path(sys.argv[2]),
Path(sys.argv[3])
305      pasta_fig.mkdir(parents=True, exist_ok=True)
306      pessoas = sorted(p for p in pasta_base.iterdir() if p.is_dir())
307      imagens = [[carregar_imagem(f, cinza=True) for f in sorted(p.glob("*.png"))] for p in
pessoas]
308      res = json.loads(arq_json.read_text(encoding="utf-8"))
309      fig_cinco_fases(pasta_fig); fig_arquitetura(pasta_fig); fig_lbp_exemplo(pasta_fig)
310      fig_etapas(pasta_fig, imagens); fig_variacao_pose(pasta_fig, imagens)
311      fig_distribuicoes(pasta_fig); fig_barras(pasta_fig, res); fig_cadastro(pasta_fig,
res)
312      fig_estados(pasta_fig); fig_banco(pasta_fig); fig_haar(pasta_fig);
fig_telas(pasta_fig, imagens)
313
314
```

```
315  if __name__ == "__main__":
316      main()
```

3.5 ferramentas/simulacao_interface.py

```
  1  """Roda os fluxos de CÂMERA da interface (cadastro, autenticação, prova de vida)
  2  com uma câmera simulada, sem janelas e sem pessoas.
  3
  4  A câmera simulada entrega quadros 640x480 montados com rostos da base ORL
  5  ampliados. Serve para pegar erros de programação nos caminhos que só rodam com
  6  câmera (que os testes automatizados não cobrem). Não mede desempenho de
  7  reconhecimento: a ORL não tem a aparência de uma foto de webcam.
  8
  9  Uso (dentro da pasta `codigo`):  python ferramentas/simulacao_interface.py
..\\base_teste\\orl
 10  """
 11
 12  import os
 13  import sys
 14  import tempfile
 15  from pathlib import Path
 16
 17  DADOS = tempfile.mkdtemp(prefix="cofre_sim_")
 18  os.environ["COFRE_DADOS"] = DADOS
 19  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
 20
 21  import cv2  # noqa: E402
 22  import numpy as np  # noqa: E402
 23
 24  from cofre import config, interface  # noqa: E402
 25  from cofre.aquisicao import carregar_imagem  # noqa: E402
 26  from cofre.servico import ADMINISTRACAO, Cofre  # noqa: E402
 27
 28  BASE = Path(sys.argv[1] if len(sys.argv) > 1 else "../base_teste/orl")
 29
 30
 31  def quadro_de_rosto(imagem64, olhos_fechados=False, lado=260):
 32      """Monta um quadro 640x480 com o rosto ampliado no centro."""
 33      rosto = cv2.resize(imagem64, (lado, lado), interpolation=cv2.INTER_CUBIC)
 34      if olhos_fechados:
 35          faixa = rosto[int(lado * 0.28):int(lado * 0.46), :]
 36          faixa[:] = int(np.median(rosto[int(lado * 0.2):int(lado * 0.6), :]))
 37          cv2.line(rosto, (int(lado * 0.2), int(lado * 0.37)), (int(lado * 0.8), int(lado *
0.37)), 40, 2)
 38      fundo = np.full((480, 640), 120, np.uint8)
 39      y0, x0 = (480 - lado) // 2, (640 - lado) // 2
 40      fundo[y0:y0 + lado, x0:x0 + lado] = rosto
 41      return cv2.cvtColor(fundo, cv2.COLOR_GRAY2BGR)
 42
 43
 44  class CameraFalsa:
 45      """Entrega quadros de uma lista (repete a última ao acabar)."""
 46      quadros = []
 47
 48      def __init__(self, *a, **k):
 49          self._i = 0
 50
 51      def __enter__(self):
 52          return self
 53
 54      def __exit__(self, *_):
 55          return False
 56
 57      def ler(self):
 58          q = self.quadros[min(self._i, len(self.quadros) - 1)]
 59          self._i += 1
 60          return q
 61
 62
 63  def carregar_pessoa(n):
 64      pasta = BASE / f"s{n:02d}"
 65      return [carregar_imagem(f, cinza=True) for f in sorted(pasta.glob("*.png"))]
 66
```

```
 67
 68  def silenciar_janelas():
 69      interface.cv2.imshow = lambda *a, **k: None
 70      interface.cv2.waitKey = lambda *a, **k: -1
 71      interface.cv2.destroyAllWindows = lambda *a, **k: None
 72      interface.cv2.getWindowProperty = lambda *a, **k: 1.0
 73      interface.Camera = CameraFalsa
 74
 75
 76  def main():
 77      silenciar_janelas()
 78      cofre = Cofre()
 79      pip = cofre.pipeline
 80
 81      pessoas = {"ministro": carregar_pessoa(1), "diretor": carregar_pessoa(3), "geral":
carregar_pessoa(5),
 82                 "intruso": carregar_pessoa(2)}
 83      # 1) detecção de rosto nos quadros simulados
 84      detectados = sum(pip.processar(quadro_de_rosto(im)).tem_rosto for im in
pessoas["ministro"])
 85      print(f"1) quadros simulados com rosto detectado: {detectados}/10")
 86      if detectados < 7:
 87          print("   A detecção falhou nos quadros simulados; a simulação não é aplicável.")
 88          return 1
 89
 90      # 2) cadastro pelo fluxo de câmera (25 amostras)
 91      cadastros = {"ministro": ("Marina Ministra", "Ministra", 3, None),
 92                   "diretor": ("Dora Diretora", "Diretora", 2, "Agrotóxicos e Biocidas"),
 93                   "geral": ("Gabriel Geral", "Analista", 1, None)}
 94      for chave, dados in cadastros.items():
 95          CameraFalsa.quadros = [quadro_de_rosto(im) for im in pessoas[chave][:8]] * 4
 96          faces = interface.capturar_amostras(cofre)
 97          assert faces is not None and len(faces) == config.AMOSTRAS_POR_CADASTRO,
f"captura {chave}: {faces and len(faces)}"
 98          uid = cofre.cadastrar(*dados, faces)
 99          print(f"2) {chave}: cadastrado (id {uid}) com {len(faces)} amostras pelo fluxo de
câmera")
100
101      comp = {c["nome"]: c["id"] for c in cofre.banco.listar_compartimentos()}
102      central = comp["Câmara Central de Toxinas de Alto Risco"]
103      agro = comp["Laboratório de Toxinas Agrícolas"]
104      cofre.reconhecedor.limiar = 0.53   # ORL tem mais variação que a webcam (ver testes
de integração)
105
106      def tentar(chave, compartimento, vida, fechar_olhos=False):
107          config.EXIGIR_PROVA_DE_VIDA = vida
108          aberto = [quadro_de_rosto(im) for im in pessoas[chave][7:10]]
109          seq = aberto * 6
110          if fechar_olhos:   # piscada simulada no meio da sequência
111              fechado = [quadro_de_rosto(im, olhos_fechados=True) for im in
pessoas[chave][7:10]]
112              seq = aberto * 5 + fechado[:2] + aberto * 8
113          CameraFalsa.quadros = seq
114          cofre.banco.definir_parametro("bloqueio_ate", 0)
115          cofre.banco.definir_parametro("falhas_consecutivas", 0)
116          return interface.autenticar_ao_vivo(cofre, compartimento)
117
118      # 3) autenticação sem prova de vida
119      ok, usuario, motivo = tentar("ministro", central, vida=False)
120      print(f"3) ministro -> câmara central (sem prova de vida): permitido={ok}
({motivo})")
121      assert ok and usuario["nome"] == "Marina Ministra"
122      ok, usuario, motivo = tentar("diretor", central, vida=False)
123      print(f"   diretor -> câmara central: permitido={ok} ({motivo})")
124      assert not ok
125      ok, usuario, motivo = tentar("diretor", agro, vida=False)
126      print(f"   diretor -> laboratório da divisão: permitido={ok} ({motivo})")
127      assert ok
128      ok, usuario, motivo = tentar("intruso", central, vida=False)
129      print(f"   intruso -> câmara central: permitido={ok} ({motivo})")
130      assert not ok and usuario is None
131
132      # 4) prova de vida
133      ok, usuario, motivo = tentar("ministro", central, vida=True, fechar_olhos=False)
134      print(f"4) com prova de vida e SEM piscada (foto parada): permitido={ok} ({motivo})")
135      ok_piscou, usuario, motivo2 = tentar("ministro", central, vida=True,
fechar_olhos=True)
```

```
136      print(f"   com prova de vida e piscada simulada: permitido={ok_piscou} ({motivo2})")
137
138      # 5) cadastro/remoção
139      cofre.remover_usuario(cofre.banco.listar_usuarios()[-1]["id"])
140      print("5) remoção de usuário: ok; ativos agora:", cofre.banco.contar_ativos())
141      print("6) integridade do histórico:", cofre.banco.verificar_integridade())
142      from cofre import avaliacao
143      print("7) calibração com as amostras de cadastro:")
144      print(avaliacao.resumo_calibracao(avaliacao.calibrar_com_cadastros(cofre.banco)))
145      config.EXIGIR_PROVA_DE_VIDA = True
146      return 0
147
148
149  if __name__ == "__main__":
150      sys.exit(main())
```